From Simulation to Deployment: A Critical Review of Fog-Based IoT Smart Parking Systems

Pratyush Kumar Srivastava
Lead Author
Roll No. 2401330120132
Dept. of CS, *NIET*
Greater Noida, India

Vedant Pandey
System Analysis
Roll No. 2401330120211
Dept. of CS, *NIET*
Greater Noida, India

Piyush Diwedi
Evaluation Planning
Roll No. 2401330120121
Dept. of CS, *NIET*
Greater Noida, India

Abstract—Internet of Things (IoT) based smart parking promises shorter searches for free bays, lower congestion, and better use of parking space. A recent fog-computing design combines LoRaWAN sensing, shortest-path bay selection, Markovchain availability estimation, and hybrid pricing, and reports shorter arrival times and higher revenue in simulation. Its evidence base, however, is narrow. The prototype covers a small part of one parking lot, camera-based congestion sensing was left out, occupancy forecasting rests on fixed-rate assumptions, the treatment of security and privacy is brief, and the pricing scheme was tested with randomly generated bids. This paper examines these six limitations, explains how each one weakens the case for real-world deployment, and outlines a targeted improvement for each, including multi-lot fog coordination, camera-assisted congestion estimation, learning-based occupancy forecasting, lightweight protection of radio links and payment data, calibrated pricing, and staged field validation. We also describe an evaluation plan with metrics, scenarios, and baselines against which the proposed improvements can be verified. The analysis is intended to guide future implementations toward designs that remain reliable outside the laboratory.

Index Terms—smart parking, fog computing, Internet of Things, LoRaWAN, occupancy forecasting, dynamic pricing, IoT security, limitation analysis

I. Introduction

Growth in vehicle ownership has pushed parking demand beyond what the supply in many cities can comfortably absorb. Drivers who cannot tell where free space exists tend to circle streets and car parks until they stumble on a vacant bay, and this cruising behavior has been shown to contribute noticeably to urban traffic [1]. The effects are wasted time and fuel, additional emissions, and heavier load on junctions that are already congested, especially near large gatherings such as religious events, sports fixtures, and conferences [2].

IoT technology offers a practical remedy. Sensors installed at individual bays, entrances, and road junctions can report occupancy as it changes, and the collected data can drive navigation, reservation, and pricing services [3]. Many early deployments forward every reading to a remote cloud, so the service depends on a single backbone link and every timecritical decision pays for a round trip across the network. Fog computing reduces this dependence by placing modest processing nodes close to the data sources, where filtering, aggregation, and first-level decisions can be made locally [4]. Paired with a long-range, low-power radio such as LoRa [5], the approach suits car parks that are large, outdoors, and short of mains power.

A representative design of this kind was reported by Balfaqih et al. [2]. Their system arranges sensing, fog, and cloud tiers; picks a bay by searching a junction graph for the least expected travel time; estimates near-future availability with a Markov-chain model; and prices short- and long-term parking through a hybrid scheme. The authors report that arrival time falls by 16–46% and revenue rises by 10–15% compared with conventional operation. These figures are encouraging, yet they rest on a hardware prototype of a very small area and on a simulation of a single lot with 150 spaces [2]. A careful reader is therefore left asking how far the reported gains would carry over to larger, busier, and less predictable settings.

This paper adopts a critical but constructive stance toward that work. Instead of proposing an unrelated system, we locate the points where the design or the evidence is weakest, explain why each matters for deployment, and outline how it could be strengthened. The main contributions are as follows:

- A structured analysis of six limitations in the reference system: limited prototype scale, missing camera-based congestion sensing, a simple fixed-rate Markov predictor, validation that is mostly simulated, shallow treatment of security and privacy, and pricing tests based on synthetic bids.
- For each limitation, a matching improvement drawn from the literature on fog-based parking, learning-based occupancy forecasting, and IoT security.
- An evaluation plan that lists metrics, scenarios, and baselines for verifying the proposed improvements in later work.

The rest of the paper is organized as follows. Section II gives background and reviews related work. Section III describes the reference system in more formal detail. Section IV analyzes the limitations, Section V presents the proposed improvements, Section VI sets out the evaluation plan, Section VII discusses deployment in practice, and Section VIII concludes the paper.

The appendices contain a glossary, tested calculation code, and reporting and risk checklists.

II. Background and Related Work

A. Fog-Assisted Parking Architectures

Fog-assisted parking systems are commonly described in three tiers. The lowest tier holds the sensing and dataacquisition devices, such as ultrasonic and magnetic sensors, RFID readers, cameras, and GPS units, that observe bays and vehicles. The middle tier consists of fog nodes that receive these observations over a wireless link, pre-process them, and run latency-sensitive services. The top tier is a cloud back end that stores history, performs heavy analytics, and serves the user-facing applications. LoRa is a popular choice for the sensor link because it trades data rate for range and energy economy [5], which suits sensors that report only when occupancy changes. The price of that choice is very limited bandwidth, so video-based sensors usually need a second technology such as Wi-Fi.

B. The Reference System

The system of Balfaqih et al. [2] combines four cooperating functions. For bay selection, the lot is modeled as a graph of junctions whose edges carry a congestion figure computed from lane occupancy, and Dijkstra's algorithm [6] finds the quickest route from the entrance to a free bay. For availability estimation, a lot is assigned to one of five occupancy states, and a transition matrix built from historical counts decides whether a reservation made at least three hours ahead can be accepted. The pricing module charges an hourly rate with stepwise discounts for short stays, while subscribers to longterm places bid for them. A fourth function, security alerting, appears in the workflow description but is not developed in the sections that follow.

The system was validated in two ways. A small prototype used an Adafruit Feather board with a LoRa radio, ultrasonic, magnetic, and RFID sensors, and a Raspberry Pi acting as the fog node. In addition, the Kudai parking area in Makkah was reproduced in the SUMO traffic simulator [7] to compare bay selection and pricing with conventional search and fixed-rate pricing.

C. Related Work

Several LoRa- and narrowband-based systems have been trialed. Barriga et al. [8] paired LoRaWAN sensors with a Kubernetes-managed messaging back end to improve availability and portability. Shi et al. [9] combined NB-IoT sensing with a third-party payment platform and tested it in two Chinese cities. Sotres et al. [10] focused on interoperability, running the same parking service over different IoT platforms in Santander and Busan. Sarker et al. [11] introduced dynamic pricing supported by edge–cloud processing.

On the prediction side, the literature has moved toward datadriven regressors. Zheng et al. [12] compared several machinelearning methods for occupancy forecasting using open data from San Francisco and Melbourne, and Vlahogianni et al. [13] used neural networks to forecast regional occupancy in Santander up to thirty minutes ahead from one-minute sensor data.

TABLE I
Summary of Representative Smart Parking Studies

Study	Focus	Notable feature
Balfaqih et al. [2]	Fog-based parking system	Dijkstra bay selection, Markov availability model, hybrid pricing
Barriga et al. [8]	LoRaWAN architecture	Kubernetes cluster with MQTT and MongoDB for availability and scalability
Shi et al. [9]	NB-IoT parking service	Third-party payment platform; tested in two Chinese cities
Sotres et al. [10]	Interoperability	One parking service reused across Santander and Busan
Sarker et al. [11]	Dynamic pricing	Edge–cloud processing with LoRa sensing
Zheng et al. [12]	Occupancy forecasting	Machine-learning comparison on San Francisco and Melbourne data
Vlahogianni et al. [13]	Occupancy forecasting	Neural networks, up to 30 min ahead, Santander sensors
You et al. [14]	Security	Enhanced LoRaWAN protocol for privacy
Jeon et al. [15]	Security	Secure element for key storage and protocol handling

Security has been studied largely at the protocol and device level. You et al. [14] proposed an enhanced LoRaWAN security protocol for privacy preservation, using a parking system as a case study, while Jeon et al. [15] designed a communication module that relies on a secure element to store keys and handle LoRaWAN security functions. Wider analyses of LoRaWAN point to threats such as replay attacks, rogue gateways, and physical capture of end devices [16].

Table I summarizes the studies discussed above.

Taken together, these studies show steady progress on individual components (sensing, prediction, pricing, and security) but, to our reading, they are seldom validated together within one deployable system. The next section examines where the reference design falls short of that goal.

D. The Parking Problem and Its Economics

The motivation for smart parking is easiest to see through the economics of curb and lot space. Shoup's review of cruising studies [1] and his broader treatment of parking policy [17] argue that when parking is cheap relative to its value to drivers, the shortage is paid for in time rather than money. Drivers circulate until a space appears, and the cost of that search is spread over everyone who shares the street. The mechanism can be illustrated with a deliberately stylized

model of our own. If each bay a driver inspects is vacant independently with probability p, the number of bays inspected before success is geometric with mean $1/p$. At 90% occupancy this is 10 bays, at 95% it is 20, and at 98% it is 50. Real search is not memoryless, since drivers see several bays at once and avoid streets they have just passed, but the shape of the relationship is the point: search effort rises slowly while occupancy is moderate and then climbs steeply as the last few spaces vanish. Information that tells a driver where those last spaces are therefore has its largest value exactly when a lot is nearly full.

TABLE II
COST COMPONENTS OF CRUISING FOR PARKING (QUALITATIVE)

Component	Mechanism	Typical proxy metric
Driver time	Extra minutes between reaching the destination area and stopping	Search time; arrival time at bay
Fuel and wear	Low-speed, stop-and-go driving over extra distance	Extra distance driven per parking event
Emissions	Fuel burned during circulation; cold-start and idling effects	Fuel-based estimate of extra emissions
Junction load	Circulating vehicles share lanes and signal phases with through traffic	Queue length and delay at nearby junctions
Reliability	Uncertainty forces drivers to leave early or abandon trips	Variance of arrival time; share of failed searches
Operator revenue	Drivers who give up leave bays unused, and underpriced bays turn over slowly	Utilization and revenue per bay

Arnott and Inci [18] place this behavior inside a model of downtown traffic. In their analysis, cruising vehicles occupy road space and so add to congestion, and the price of curb parking becomes a lever on both problems at once: a price that keeps a small share of spaces vacant shortens search and thereby removes cruising traffic. Pierce and Shoup [19] examined a practical version of this idea, in which meter rates in San Francisco were adjusted by block and time of day toward a target occupancy band rather than held fixed. Their evaluation is relevant to our review for two reasons. It shows that pricing can be driven by measured occupancy, which is exactly the data an IoT deployment produces, and it shows that the price signal must be calibrated against observed demand, a point we return to in Section IV-F.

The costs of cruising are borne by different parties and are measured differently, which matters when a smart parking system is evaluated. Table II lists the main components in qualitative terms. We deliberately give no magnitudes: they depend on city, street layout, and time of day, and the literature cited above should be consulted for measured values in specific settings.

Two consequences follow for the system studied in this paper. First, a guidance or reservation service is valuable only to the extent that it reduces some of these components, so a credible evaluation should report more than one of them. The reference system reports arrival time and revenue [2]; junction load and reliability are not reported, and they are the components most likely to differ between a 150-space simulation and a large lot. Second, guidance can shift cruising rather than remove it. If many drivers are routed to the same bay or the same entrance, the junction load moves rather than falls. This is one reason why the evaluation plan of Section VI includes surge scenarios.

E. Parking Sensing Technologies

Every smart parking service depends on an occupancy signal, and the choice of sensor shapes cost, accuracy, and the communication technology that can carry the data. Surveys of the field [20], [21] classify the options in several ways; we group them by what is installed and where. Table III compares the main families on the criteria that matter for a LoRaconnected lot. The entries describe tendencies reported in the surveys and general engineering experience, not measured figures, and individual products vary widely.

Three observations follow from the table. First, the perbay technologies (ultrasonic, magnetic, infrared) have the same communication profile, namely tiny, rare, event-driven messages, which is exactly what LoRa serves well. The reference prototype follows this pattern, combining ultrasonic and magnetic sensors with a LoRa radio [2]. A single sensor type, however, has recognizable failure modes, and combining two types is a common way to cross-check readings. We use that idea for plausibility checks in the security design of Section V.

Second, vision is the only family that can estimate presence for many bays, and also lane congestion, from one installation. Deep-learning detectors have been shown to work on parkinglot images with small models that can run on the camera side [22], and the surveys [20], [21] treat vision as the most flexible but also the most demanding option in computing and privacy terms. The consequence for a LoRa network is architectural: the camera cannot send its stream over the sensor radio, so the model has to run locally and send only counts or bay states, which is the arrangement assumed by the cameraassisted improvement of Section V; the corresponding gap in the reference system is discussed in Section IV-B.

Third, the crowd-sensing and drive-by approaches replace roadside hardware with data from vehicles and phones. Nawaz et al. [23] sense parking and departure events using smartphones carried by drivers, while Mathur et al. [24] mount range sensors and GPS on vehicles that drive past roadside bays and compile occupancy statistics. Both reduce capital cost and scale with participation, but they provide neither the

continuous per-bay state that a reservation service needs nor guaranteed coverage. They are best seen as complements for on-street parking, where fixed sensors for every bay would be expensive, whereas the lot-based setting of the reference system favors fixed sensors.

F. IoT, Edge, and Fog Computing Foundations

The IoT surveys of Atzori et al. [25] and Gubbi et al. [26] describe the layered architecture that smart parking inherits: things that sense and act, a communication layer that connects them, and services that turn data into decisions. Zanella et al. [27] apply this view to smart cities and list parking among the early use cases, and the SmartSantander testbed [28] is an example in which a city deployed a large number of heterogeneous devices, with parking sensors among them. These works establish the setting; the question that matters for this paper is where in the pipeline computation should happen.

a) Cloud, edge, and fog: In the cloud model, sensors send everything to centralized data centers that offer large and elastic computing. Edge computing moves some computation to the network edge, close to where data originate [29], [30]. Fog computing, introduced by Bonomi et al. [4] and developed in the surveys of Dastjerdi and Buyya [31] and Mouradian et al. [32], emphasizes a hierarchy of intermediate nodes between devices and cloud, which can filter, aggregate, and act on data and coordinate with each other. In practice the terms overlap, and the surveys differ on boundaries; mobile edge computing, which places servers in the cellular access network, is a related line of work [33]. For our purposes the distinction that matters is functional: a fog node is an inexpensive, locally managed computer, such as the Raspberry Pi in the reference prototype [2], that sits within one radio hop of the sensors and can make decisions without the wide-area network.

Fig. 1 shows the three tiers and the direction of data reduction. Volume is largest at the bottom in aggregate but is made of many very small messages for bay sensors, and it shrinks as it moves up because the fog node turns raw reports into counts, states, and exceptions. A camera changes this picture, since its raw volume is large at the sensing tier,

TABLE III
QUALITATIVE COMPARISON OF PARKING SENSING TECHNOLOGIES

Technology	Deployment effort	Accuracy tendencies	Power	Privacy	Suitability for LoRa
Ultrasonic (per bay)	Mounted overhead or in a wall or pavement; one unit per bay	Good at short range; sensitive to temperature, rain, and obstructions	Low to moderate; battery or small solar cell feasible	Low risk; no images	Good: small, infrequent state-change messages
Magnetic (in-ground)	Drilled or glued into pavement; one or two units per bay	Detects ferrous mass; neighboring bays, large vehicles, and drift can cause errors	Very low; long battery life is a design goal	Low risk	Very good: compact payload, event-driven
Inductive loop	Cut into pavement and wired to a controller	Reliable for passage and presence at gates and lanes; poor fit for open bays	Mains or wired supply	Low risk	Poor for bays; acceptable as an entry or exit counter feeding a gateway
Infrared (active or passive)	Mounted above or beside the bay	Affected by sunlight, dust, and occlusion; acceptable in covered lots	Low	Low risk	Good
RFID and tags	Readers at gates or bays; tags on vehicles	Identifies permit holders rather than detecting any vehicle	Low for tags; readers need supply	Higher: identifies vehicles and people	Mixed: readers are fixed, tag reads are local
Camera and vision	One camera can cover many bays; needs mounting, lighting, and a data link	High potential accuracy with good models; sensitive to light, weather, occlusion	High: computing is needed at or near the camera	Highest: images may show plates and faces	Poor as a payload: video exceeds LoRa capacity, so only extracted counts can use LoRa
Crowd-sensing (smartphones)	No roadside hardware; relies on an app and user participation	Coverage follows participation; detects events rather than static state	Phone battery; no infrastructure power	Higher: location traces	Not applicable: uses cellular links
Drive-by (vehiclemounted)	Sensors on vehicles that already drive the area	Coverage depends on routes and frequency; gives statistics rather than live state	Vehicle supply	Moderate: vehicle positions	Not applicable: batch upload

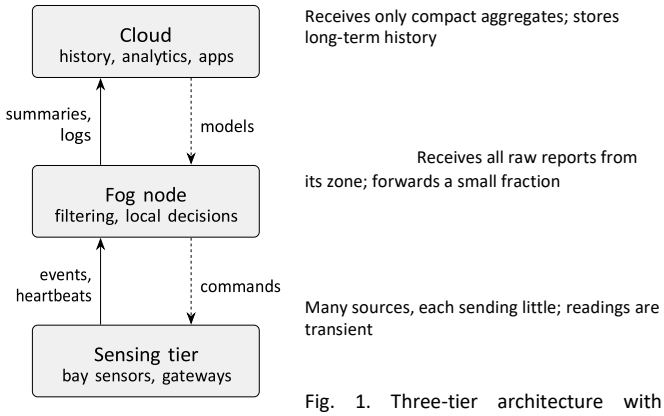


Fig. 1. Three-tier architecture with qualitative annotations of the data volume handled at each tier. Solid arrows show upward data flow and dashed arrows show downward control flow.

which is why it must be processed at the fog node rather than forwarded.

b) Latency decomposition: The practical argument for fog is about time. We write the end-to-end delay of a parking decision, from a change of bay state to the corresponding update reaching a driver, as

$$T_{\text{total}} = T_{\text{sense}} + T_{\text{uplink}} + T_{\text{queue}} + T_{\text{proc}} + T_{\text{downlink}}. \quad (1)$$

Here T_{sense} is the time for the sensor to detect and confirm a change, including any debouncing; T_{uplink} is the time to carry the report to the place where it is processed; T_{queue} is waiting in gateways, brokers, or servers; T_{proc} is computation; and T_{downlink} is the time to return the result to the driver or to an actuator such as a sign. The formula is an accounting identity rather than a model, and each term can itself be decomposed further.

Fog computing acts mostly on three of the five terms. Processing at a node one radio hop from the sensors removes the wide-area segment from T_{uplink} and T_{downlink}. A local node serving one zone also has a short queue, so T_{queue} is under the operator's control and does not depend on shared cloud load. For simple rules, such as updating a free-bay count, T_{proc} is trivial in both places, so fog yields no gain there; the gain appears when the alternative would be a remote service with variable response time. Fog does not remove T_{sense}, which is a property of the sensor, nor the radio segment of T_{uplink}, which for LoRa is dominated by the time on air computed in the next subsection and by any retransmission. In a LoRa deployment the radio terms can be much larger than the backbone terms, so the relevance of fog for latency should be established by measuring the terms separately, which the reference paper does not do, since it reports only a fog processing interval [2].

Latency is not the only argument. Bandwidth is the second: forwarding summaries instead of raw reports, or counts instead of video, keeps backbone use low and cost predictable. Availability is the third: if the link to the cloud fails, a fog node can continue to serve guidance and gate control from local state and synchronize afterward, whereas a cloud-only design stops. This autonomy has costs of its own, including more devices to maintain, patch, and secure, and the need to reconcile state when links return [32]. We take these trade-offs into account in the zoned design of Section V.

G. LoRa and LoRaWAN Primer

LoRa is a physical-layer modulation based on chirp spread spectrum, and LoRaWAN is the medium-access and network protocol defined above it by the LoRa Alliance [34]. Surveys of the technology and its limits are available in [5], [35], [36], and broader comparisons with other low-power widearea networks (LPWANs) in [37]. This subsection collects the facts that the rest of the paper uses.

a) Modulation and basic formulas: A LoRa symbol is a linear frequency sweep, or chirp, across the channel bandwidth BW, and the value carried by the symbol is encoded as a cyclic shift of a base chirp. The spreading factor SF, an integer typically between 7 and 12, determines that each symbol carries SF bits and spans 2^{SF} chips. Since the chip rate equals BW, the symbol duration is

$$T_s = \frac{2^{\mathrm{SF}}}{\mathrm{BW}}. \quad (2)$$

TABLE IV
TIME ON AIR IN MILLISECONDS, COMPUTED FROM EQS. (2)–(5)
(BW = 125 KHZ, CR = 4/5, 8-SYMBOL PREAMBLE, EXPLICIT HEADER, CRC ON)

PHY payload	SF7	SF9	SF12	DE at SF12
6 bytes	36.10	123.90	991.23	on
12 bytes	41.22	144.38	1155.07	on
24 bytes	61.70	205.82	1482.75	on
T_s (ms)	1.024	4.096	32.768	

With forward error correction at code rate $\mathrm{CR} = 4/(4+n)$ for $n \in \{1,2,3,4\}$, the raw modem bit rate is

$$R_b = \mathrm{SF} \cdot \frac{\mathrm{BW}}{2^{\mathrm{SF}}} \cdot \mathrm{CR}. \quad (3)$$

At $\mathrm{BW} = 125$ kHz and $\mathrm{CR} = 4/5$ this gives about 5.47 kbit/s at SF7, 1.76 kbit/s at SF9, and 0.29 kbit/s at SF12. These are modem rates, not application throughput, because headers, preambles, and protocol overhead are not included. A larger SF lengthens the symbol by a factor of two for each step and increases the processing gain of the receiver, which is what allows lower signal levels to be decoded and therefore longer range. The price is longer transmissions, which cost more energy per message and occupy the channel longer. Different spreading factors on the same channel are quasi-orthogonal,

so transmissions at different SFs interfere with each other much less than transmissions at the same SF, although they are not perfectly independent [36].

b) Time on air: The duration of a frame follows from the modem design guide of the chip vendor. A frame consists of a preamble of n_{pre} programmable symbols plus 4.25 symbols of synchronization, followed by a header and payload. With the notation PL for the payload size in bytes, $H = 0$ for explicit header mode and $H = 1$ for implicit mode, $\mathrm{CRC} \in \{0,1\}$ for the presence of the payload checksum, and $\mathrm{DE} \in \{0,1\}$ for the low-data-rate optimization, the number of payload symbols is

$$n_{\mathrm{pay}} = 8 + \max\left(\left\lceil \frac{8\mathrm{PL} - 4\,\mathrm{SF} + 28 + 16\,\mathrm{CRC} - 20\,H}{4\,(\mathrm{SF} - 2\,\mathrm{DE})} \right\rceil \times (n+4),\ 0\right), \tag{4}$$

and the time on air is

$$T_{\mathrm{air}} = (n_{\mathrm{pre}} + 4.25)T_s + n_{\mathrm{pay}}\,T_s. \tag{5}$$

The low-data-rate optimization is normally enabled when T_s exceeds 16 ms, which at 125 kHz applies to SF11 and SF12. We implemented Eqs. (2)–(5) in Python and give the results in Table IV.

The payload size in Eq. (4) is the physical-layer payload. In LoRaWAN it includes the MAC header, device address, frame control, frame counter, port, and message integrity code, which together add 13 bytes when no MAC options are carried [34]. A 6-byte application payload therefore becomes a 19-byte frame, which lasts about 51.5 ms at SF7, 185.3 ms at SF9, and 1318.9 ms at SF12. The ratio between SF7 and SF12 is about 26, which is the central design tension of a LoRa lot: devices near the gateway can send cheaply, while distant ones consume a large share of the channel and of their batteries.

c) Regulation and duty cycle: Unlicensed radio bands are regulated, and the rules differ by region. Some regulations restrict the fraction of time a device may transmit within a subband, others limit the dwell time on a channel or require listenbefore-talk, and some set only a power limit. These rules, and the frequency plan, must be checked against the regulations that apply at the deployment site; we do not state specific limits here. The design consequence is general: the time on air from Eq. (5) defines a message budget for every device. Purely as an illustration, if a regulation allowed 1% of the time on a channel, a device sending the 19-byte frame above could send at most 36 s of airtime per hour, which is about 699 frames per hour at SF7 but only about 27 at SF12. Whatever the actual rule, the budget grows tighter at higher SF, and event-driven reporting with a sparse heartbeat is the natural way to live within it.

d) Device classes and adaptive data rate: LoRaWAN defines three device classes [34]. Class A devices, the most energy-efficient, open two short receive windows only after each uplink, so downlink messages can reach them only after they have transmitted. Class B devices additionally open scheduled receive slots synchronized by beacons from the gateways, which gives bounded downlink latency at some extra energy cost. Class C devices listen almost continuously except when transmitting, and suit mains-powered equipment such as signs or barriers. A battery-powered bay sensor is naturally Class A, and the consequence for the latency budget of Eq. (1) is that a command to a Class A sensor waits for its next uplink. The adaptive data rate (ADR) mechanism lets the network server adjust the data rate, and hence spreading factor, and the transmit power of a device based on the link margin observed in its recent uplinks. ADR is well suited to stationary devices, such as parking sensors, because their channel changes slowly, and it moves nearby devices to fast settings to save airtime [34], [35].

e) Capacity limits: Channel access in LoRaWAN is essentially ALOHA: devices transmit when they have data, without sensing the channel. For a simple pure-ALOHA model with offered load G (frames per frame duration, on one channel and spreading factor), the probability that a frame suffers no collision is e^{-2G} and the throughput is $S = Ge^{-2G}$, with a maximum of $1/(2e) \approx 18.4\%$ at $G = 0.5$. For $G = 0.05$ the success probability is 0.905, and for $G = 0.1$ it is 0.819. This model is only a rough guide, since capture effect and quasi-orthogonality change real behavior, but it shows why density matters, as the scalability studies find [35], [38]. Those studies also point to limited gateway downlink capacity as a bottleneck, which argues against designs that acknowledge every uplink. Section IV-A applies these considerations to the reference system.

H. LPWAN and Short-Range Alternatives

LoRaWAN is one of several candidates for connecting bay sensors. Table V compares it qualitatively with two other LPWAN options and three short-range technologies. The comparison follows the surveys [36], [37] and assumes a parking lot of moderate size with sensors that report rarely. The entries are tendencies, not guarantees.

The comparison suggests a division of labor rather than a single winner. Long-range sensing with LoRa or NB-IoT avoids the dense relay infrastructure that short-range options require in large outdoor lots, and a private LoRaWAN network avoids recurring per-device fees. NB-IoT trades that independence for operator-managed coverage and higher data capacity. Wi-Fi remains the natural link for cameras and for fog-to-cloud traffic, which is why the architecture of Fig. 10 uses more than one radio technology.

I. Messaging and Data Layer

Behind the radio link, the system needs a way to move events between gateway, fog node, cloud services, and user applications. MQTT is the most common choice in this setting [8]. It is a lightweight publish/subscribe protocol standardized by OASIS [39]: clients publish messages to named topics on a broker, and clients subscribe to topics, using wildcards for one level or for all remaining levels, to receive them. Publishers and subscribers do not know about each other, so new consumers, such as a dashboard or a forecasting service, can be added without changing the sensor side. A topic hierarchy such as lot/zone/bay/state, which is our own example, lets a subscriber ask for a single bay, a zone, or the whole lot.

a) QoS and retained messages: MQTT offers three levels of delivery assurance per publication [39]. QoS 0 delivers a message at most once, with no acknowledgment. QoS 1 delivers at least once, using an acknowledgment, so duplicates are possible. QoS 2 delivers exactly once with a four-part handshake, at the highest cost. For bay-state updates, QoS 1 is a sensible default, provided that consumers are idempotent: processing the same state report twice must give the same result. A related feature is the retained message, which makes the broker store the last message on a topic and send it to every new subscriber on connection. Publishing bay state as retained messages means that a restarted application receives the current state immediately instead of waiting for the next change, which matters for sensors that report rarely. A last-will message, which the broker publishes when a client disconnects unexpectedly, is a simple way to signal that a gateway has failed. These mechanisms apply between the gateway and the services; the LoRa link itself has its own, different reliability mechanism.

b) Time-series storage: Occupancy data form a time series per bay, and the storage design should reflect how the data are used. Event logs with one row per state change are compact, and they allow the occupancy at any time to be reconstructed, while periodic snapshots make queries simple but waste space for bays that rarely change. A practical

TABLE V
QUALITATIVE COMPARISON OF CONNECTIVITY OPTIONS FOR BAY SENSORS

Technology	Range and coverage	Data capacity	Energy use	Infrastructure and cost model	Parking fit
LoRaWAN	Long range; a few gateways can cover a lot or a district	Low; messages limited to small payloads	Very low for Class A	Own gateways in unlicensed bands, or a shared network; no per-message operator fee if self-run	Strong for private lots with event-driven bays
NB-IoT	Long range through cellular coverage	Low to moderate; higher than LoRa	Low, but typically above LoRa in active use	Licensed spectrum; operator subscription per device	Strong where coverage and budget exist; no private network
Sigfox-style ultranarrowband	Long range	Very low; tiny payloads and limited daily messages	Very low	Proprietary network operated by a provider	Adequate for state-only reports; less flexible for downlink
Bluetooth Low Energy	Short; tens of meters	Moderate	Very low	Many gateways needed for a lot; phones can act as relays	Fits small garages and phone-based access, not outdoor lots
Wi-Fi	Short to medium	High	High for battery sensors	Existing access points can be reused; wired power needed	Suited to cameras and fog uplinks, not to battery sensors
Zigbee (IEEE 802.15.4 mesh)	Short per hop; mesh extends range	Low	Low	Mesh of mains-powered or battery routers; configuration effort	Workable in covered lots; multi-hop adds latency and failure points

design keeps raw events for a limited period, builds aggregates (occupancy per zone per interval, turnover per bay, dwelltime distribution) for the forecasting and pricing services, and applies retention rules that also serve privacy goals. Writes should be idempotent and keyed by bay and sensor timestamp, because duplicates and out-of-order arrival are normal at QoS 1 and when fog nodes synchronize after an outage. Where the gateway timestamp and the device clock disagree, the design should state which one defines the event time. Finally, forecasting experiments depend on repeatable splits of the data, so stored records should keep both event time and ingestion time.

c) An illustrative bay payload: Because airtime grows with payload size, the application payload of a bay sensor should be small. Table VI shows an illustrative 6-byte application layout that we constructed for discussion; it is not taken from the reference system or from any product, and the field choices are only examples. The fields that LoRaWAN already provides in its frame, namely the device address and the frame counter, need not be repeated in the payload, so they appear in the table only to show where they travel. Together with the 13 bytes of frame overhead described above, the layout gives the 19-byte frame used in the airtime example.

TABLE VI
ILLUSTRATIVE LAYOUT OF A BAY REPORT (NOT FROM THE REFERENCE SYSTEM OR FROM A PRODUCT)

Field	Bytes	Carried in	Purpose
Device address	4	MAC header	Identifies the sensor; mapped to a bay by the network server
Frame counter	2	MAC header	Ordering and replay protection
Status flags	1	Payload	Bit 0 occupied; bit 1 fault; bit 2 tamper; bits 3–7 reserved
Battery level	1	Payload	Coarse level for maintenance planning
Time since change	2	Payload	Seconds since the last state change; lets the fog node place the event in time
Temperature	2	Payload	Reading in tenths of a degree, to detect environmental drift in the sensor

J. Simulation Tools for Parking Research

Because field trials are costly, most parking research, including the reference study, relies on simulation. The tool used by the reference system is SUMO, an open-source microscopic traffic simulator [7], [40]. In a microscopic model each vehicle is simulated individually, with car-following and lanechanging rules, on a road network that can be imported

from map data. Routes and demand can be generated or specified, and a running simulation can be inspected and controlled from outside through an interface, which is how a parking-guidance algorithm can be attached to it. SUMO also provides parking areas with a capacity, to which vehicles can be sent and where they stop for a configured duration.

It is equally important to be clear about what SUMO does not model. It has no radio layer: packet loss, collisions, gateway coverage, and duty-cycle limits do not exist unless another tool supplies them, and a SUMO result therefore says nothing about whether the sensing network could deliver the data that a guidance algorithm assumes. It does not model human parking decisions beyond what the experimenter configures; the choice of destination, willingness to follow guidance, and reaction to full lots are set by rules and parameters, not learned from observed behavior. Its outputs therefore depend heavily on the demand and behavior inputs, which is the validity concern raised in Section IV-D. Studying the sensing network requires a separate network or radio simulator, or a co-simulation in which the traffic model supplies events and a network model decides which of them arrive. Using such a pair leaves the question of calibration open, since both models need inputs from measurement, and we regard the combination as the most appropriate direction for future work on this system rather than a settled method.

K. Survey Landscape and Scope of This Paper

Smart parking has been surveyed repeatedly. Lin et al. [20] organize the field by the functions a solution provides, such as collecting and distributing parking information, reservation, and pricing, and compare the sensing and communication options behind them. Biyik et al. [3] review the literature together with architectures and open directions, and Fahim et al. [21] take a comprehensive review across several aspects of the problem. These surveys are valuable for orientation, and we do not repeat their taxonomies. Their breadth, however, is also their limit: a survey that covers sensors, networks, algorithms, and business models in a single pass can say little about whether any one design was demonstrated at the scale, duration, and realism that a city operator would require.

This paper differs in scope in three ways. First, it is anchored on one system, the fog-based design of Balfaqih et al. [2], so each question about the literature is asked from the point of view of a concrete reference: does any other work supply the missing congestion input, a stronger forecaster, a security design, or field evidence? Second, it reads the literature through evidence rather than novelty. For every cited work we ask what kind of evidence it offers (a derivation, a simulation, a bench test, a pilot, or a deployment) and what that evidence can support. Third, it is explicitly an analysis: we do not claim to have surveyed all work, and the works discussed below were chosen as representatives of the themes in Section IV. Statements about individual works are restricted to what their titles and well-known contributions support; where we are unsure of a detail we say so and refer the reader to the cited text.

L. System Architectures

Table VII compares the system-level papers along four dimensions: the connectivity technology between sensors and the first processing point, where processing is placed, what the authors validated, and which services are provided. Cells that we cannot state with confidence from the descriptions in Section II or from well-known facts of the cited work read "see cited work"; we prefer an honest blank to an invented entry.

Three observations follow from the table. First, the set of connectivity choices is narrow. LoRa-class and NB-IoT links dominate because bay sensors send tiny payloads rarely and run on batteries, and both trade throughput for coverage and energy. Within the cited set, the placement of processing is more varied: a containerized cloud back end [8], an edge–cloud split [11], and a three-tier fog design [2]. The architectural question that matters for deployment is therefore not whether fog is used but which decisions must survive loss of the backhaul. A bay-selection decision made on a nearby node can continue during an outage; a payment or a city-wide view cannot.

Second, the service sets are uneven. Several works concentrate on infrastructure (portability, interoperability, payment integration) and offer limited guidance logic, whereas the reference design offers guidance, reservation, and pricing but on a thin infrastructure base. No single cited work, to our reading, joins a field-tested infrastructure to all three services, which is the gap that Section V tries to close on paper.

Third, the radio layer is treated as a solved component in most architecture papers, although the performance of LoRa at scale has been studied separately. Bor et al. [38] asked directly whether LoRa networks scale and showed, through simulation, that capacity is limited by collisions among uncoordinated transmissions as density grows. Adelantado et al. [35] discuss the limits of LoRaWAN in terms of duty-cycle regulation, the small downlink capacity, and the number of devices a gateway can serve. Sallum et al. [42] study how assigning radio parameters affects network performance. A parking lot concentrates thousands of devices in a small area under one gateway, which is close to the density regime these works warn about. Because an architecture paper rarely reports packet delivery at its own deployment density, its radio assumptions cannot be audited from the paper alone. This is the literature behind the scale limitation of Section IV-A.

TABLE VII COMPARISON OF CITED SYSTEM ARCHITECTURES

Work	Connectivity	Processing placement	Validation	Services
Barone et al. [41]	See cited work	See cited work	See cited work	Parking management in a smart-city setting
Barriga et al. [8]	LoRaWAN	Kubernetes-managed back end with MQTT messaging and MongoDB storage	See cited work	Availability information; stress on portability and scalability of the back end
Shi et al. [9]	NB-IoT	See cited work	Tested in two Chinese cities	Parking service with third-party payment
Sotres et al. [10]	See cited work	Existing IoT platforms in two cities	Same service run in Santander and Busan	Parking service reused across platforms
Sarker et al. [11]	LoRa	Edge and cloud	See cited work	Parking with dynamic pricing
Balfaqih et al. [2]	a/LoRaWAN	Sensing, fog, and cloud tiers	Small hardware prototype plus SUMO simulation of one lot	servation, hybrid pricing

TABLE VIII
QUALITATIVE COMPARISON OF FORECASTING MODEL FAMILIES FOR OCCUPANCY

Family	Data requirement	Interpretability	Uncertainty support	Edge suitability
Markov chains [45]	Low per matrix, but grows with the number of states and contexts	High: transition probabilities can be read directly	Natural: the model yields a distribution over future states, conditional on its assumptions	Excellent: a small table and a matrix product
Linear regression with lags	Low to moderate	High: coefficients have a direct meaning	Prediction intervals under distributional assumptions	Excellent
Support vector regression	Moderate	Low	No native intervals; add-on methods needed	Moderate: cost grows with the number of support vectors
Neural networks (including recurrent)	High	Low	Needs add-ons such as quantile outputs or ensembles	Moderate to good after compression
Tree ensembles	Moderate	Medium: feature importance, no simple closed form	Quantile variants or conformal wrappers	Good
ARIMA-type models [46]	Moderate series length per series	Medium to high	Analytical intervals under model assumptions	Good

M. Availability Prediction Methods

Prediction work in parking splits into two questions: how many bays will be free at a horizon, and how reliable that statement is. Caicedo et al. [43] treat the first in real time for lotbased supply, Rajabioun and Ioannou [44] build multivariate spatiotemporal models for on-street and off-street availability, Zheng et al. [12] compare several learners on open data, and Vlahogianni et al. [13] use neural networks for short-horizon regional forecasts. Whatever the method, the forecaster is only useful if it is evaluated against simple benchmarks such as persistence, and if its output carries a measure of uncertainty that a reservation rule can use.

Table VIII compares model families qualitatively. The ratings are relative and describe typical behavior; they are not measurements, and the best choice for a given lot depends on its data.

The Markov row deserves a closer reading because it is the reference system's choice. A chain over S states has $S(S-1)$ free transition probabilities, which is 20 for the five states of the reference design. If the chain is made timedependent by giving every hour of the day and two day types (weekday, weekend) its own matrix, the model has $24\times2\times20 = 960$ free parameters. These counts are illustrative, since the reference paper does not state its context structure here, but they show why a context-rich chain demands far more history than a single matrix. Regression, tree, and recurrent models absorb time-of-day and event effects as input features instead of multiplying matrices, which is the main reason the later literature moved away from pure chains.

ARIMA-type models occupy the middle ground. They are a classical tool for a single series with trend and seasonality [46], give analytical prediction intervals, and are cheap to run, but they treat each series separately and cope poorly with exogenous shocks unless regressors are added. For occupancy data the practical advice from forecasting texts applies: compare against naive and seasonal-naive benchmarks first, and report errors with time-ordered validation rather than random splits. Uncertainty deserves more attention than it receives in the parking literature. Quantile regression [47] and conformal prediction [48] are model-agnostic ways to attach intervals to any of the point forecasters above, and a reservation system that accepts bookings only when the interval's lower bound clears a margin turns that interval into a decision rule.

N. Reservation and Pricing

Pricing and reservation serve two goals that are easy to confuse: steering demand toward a desirable occupancy and raising revenue. The classic argument for the first goal is that underpriced curbs and lots create cruising for the few spaces that are free, and that a price tuned so that a few spaces remain vacant removes the incentive to circle [17]. Pierce and Shoup [19] evaluated one prominent implementation of this idea in San Francisco, where rates were adjusted periodically in response to measured occupancy. The lesson for sensorbased systems is that demand-responsive pricing needs an occupancy measurement that is trusted by the operator, which links pricing directly to the sensing and forecasting limitations discussed above. Lei and Ouyang [49] treat reservation and dynamic pricing jointly for urban parking management, which is the setting in which a forecast of future availability becomes an input to the price.

Conceptually, four mechanism families appear in the literature and in practice.

- *Fixed rates* are simple and predictable but cannot respond to scarcity, so they serve as the natural baseline.
- *Tiered or time-banded rates*, including discounts for short stays or higher rates for longer ones, shape the duration of stay but not the level of demand at a given hour.
- *Demand-responsive rates* adjust price to observed or forecast occupancy within bounds, aiming at a target range as in the San Francisco experience [19]. They need frequent, trustworthy occupancy data and a communication policy so drivers can anticipate changes.
- *Auctions* let users reveal willingness to pay, as in the longterm bidding of the reference design [2]. For a single item or for bidders who each want one unit, the second-price idea of Vickrey [50] removes the incentive to shade bids, and the auction literature [51] explains how the rules of payment shape strategic behavior. Auctions are attractive for scarce long-term places but add participation and trust costs.

Sarker et al. [11] place dynamic pricing in an edge–cloud pipeline, which suggests where such logic can run. What the cited works rarely provide is a comparison of mechanisms on a common demand model; it is difficult to say from published results alone which family is best for a lot of a given size. This is why Section IV-F asks for calibrated demand rather than a new mechanism.

O. Vision-Based Occupancy and Congestion Estimation

Cameras are attractive because one device observes many bays or lanes, and the output is directly interpretable. Amato et al. [22] proposed a compact convolutional network that classifies the occupancy of individual bays on smart cameras, so that only decisions, not images, need to travel to a server, and they released a dataset for others to test against. LuqueVega et al. [52] describe a bay-level presence sensor that uses visual aids as part of an IoT parking system. Both works address bay occupancy. Lane-level congestion, the input that the reference routing model needs, is a different measurement: it requires counting moving vehicles per lane rather than classifying parked ones.

General-purpose detectors are the usual starting point for that counting task. Single-pass detectors such as YOLO [53] trade a little accuracy for speed, and mobile-oriented network designs [54] and compression techniques such as quantization [55] are the standard route to fitting them on small devices. Counting also needs association over time, for which simple trackers [56] are widely used, so that one vehicle is not counted in every frame. The engineering questions are therefore well understood in outline; what is not documented in the parking literature we reviewed is the end-to-end effect of count error on routing. A count that is wrong by a few vehicles may barely change the lane density, while a systematic bias in one lane, for example from occlusion, can steer many drivers into the wrong aisle. The effect depends on lot geometry, so it has to be measured per site. The cost side is equally concrete: video needs far more bandwidth than the very small, infrequent messages of bay sensors, forcing a second radio technology and local processing, and it creates privacy exposure that a magnetic sensor does not.

P. Security of LPWAN-Based Parking

Security work relevant to parking sits at three levels. At the protocol level, Butun et al. [16] analyze the security risks of LoRaWAN, Yang et al. [57] study vulnerabilities of the protocol, and Eldefrawy et al. [58] provide a formal analysis. Together they show that the standard's protections depend on correct configuration and key handling, so a deployment cannot treat

"LoRaWAN is encrypted" as a complete answer. At the device level, Jeon et al. [15] propose a communication module with a secure element that stores keys and performs security functions, which addresses the physical exposure of unattended roadside nodes. At the application level, You et al. [14] use a parking system as a case study for a protocol that aims to preserve privacy.

What is missing from this body of work is the connection to the parking service itself. The protocol papers analyze message exchange but not the consequences of a forged occupancy report for the forecaster, the price, or a reservation; the parking system papers describe services but give security a paragraph at most. A threat model for parking therefore has to be written at the service level: which false inputs would change a decision, who can inject them, and what the cheapest detection is (for instance, comparing a bay report against entry and exit counters). Privacy deserves the same treatment, since reservation and payment records can reveal where a person lives and works even when each message is protected. These points support the layered design of Section V and the gaps recorded in Section IV-E.

Q. Evaluation Practice in the Smart-Parking Literature

The metrics reported across the cited works cluster into a few families. Guidance papers report search time or travel time to a bay, sometimes with distance or fuel as proxies; the reference design reports arrival time and searching time [2]. Pricing papers report revenue, occupancy, and sometimes the share of demand turned away. Prediction papers report forecast error, typically absolute or squared error against a horizon [12], [13]. Vision papers report detection or classification accuracy on labeled images [22]. Network and architecture papers report quantities such as packet delivery, delay, or service availability, although we found these less often than the system descriptions would justify. Rarely does one paper report all of these together, so the end-to-end chain from sensor reliability through forecast and price to driver outcome is seldom measured.

The balance between field and simulated evidence is skewed. Works that are validated in the field tend to be those that emphasize infrastructure and integration, as with the twocity tests of Shi et al. [9] and Sotres et al. [10], while works that evaluate guidance and pricing logic lean on simulation [2], [49]. This is a qualitative reading of the cited subset only; we do not count studies across the field, and no percentages are implied. The pattern is plausible for a structural reason: simulation can vary demand cheaply, whereas a field trial of pricing needs the cooperation of an operator and of the drivers it affects. Simulation methodology has its own standards for validity [59], and the reference paper's evidence, as Section IV-D showed, does not state whether those standards were met.

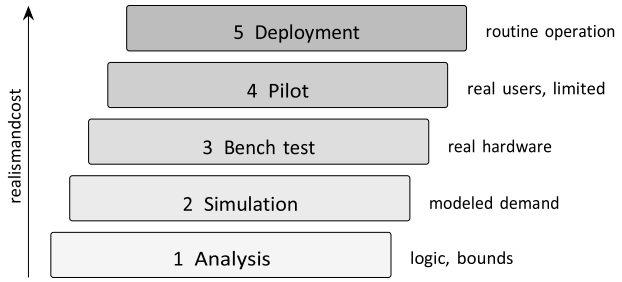


Fig. 2. Evidence ladder for smart-parking claims. Higher rungs add realism and cost; they do not replace the lower ones.

TABLE IX
WHAT EACH RUNG OF THE EVIDENCE LADDER CAN AND CANNOT ESTABLISH

Rung	Can establish	Cannot establish
Analysis	Correctness of a rule, bounds on capacity or revenue, parameter counts	Behavior under real arrivals, noise, or human response
Simulation	Relative effect of policies under stated demand, sensitivity to assumptions, scale effects	That the assumed inputs match reality; hardware and radio faults not modeled
Bench test	Device behavior, energy use, delivery under injected faults, software defects	Environmental drift over seasons; driver behavior; scale beyond the bench
Pilot	Sensor accuracy against audits, operator workflow, driver acceptance in one site	Generality across sites; rare events not seen during the trial
Deployment	Sustained reliability, maintenance cost, outcomes over seasons	Causal attribution, unless a control or before–after design is built in

To make the strength of evidence explicit, we propose an *evidence ladder* with five rungs, shown in Fig. 2. Each rung adds realism and cost, and each can establish some claims but not others, as Table IX sets out. The ladder is meant as a vocabulary for reading papers and planning studies, not as a quality score: a derivation can be the right evidence for a claim about capacity, and a deployment can be the wrong place to test a risky mechanism.

Placed on this ladder, the reference system combines rung 1 (its formulation), rung 2 (the SUMO study), and a small portion of rung 3 (the hardware prototype), with nothing yet at rungs 4 or 5. That is a legitimate position for a design paper, as long as the claims are phrased to match: statements about relative policy effects under stated demand are supported, whereas statements about expected real-world gains are not.

Section VI organizes the proposed evaluation so that each improvement climbs the ladder in order.

R. Synthesis and Open Gaps

Reading the themes together, five gaps stand out, and each one motivates part of the remaining paper.

1) *Scale evidence.* Radio-level studies [35], [38], [42] show that density matters, yet parking architecture papers seldom report delivery at their own density. This motivates Section IV-A and the zoned design of Section V.

TABLE X
GAP MATRIX: LIMITATION THEMES AGAINST REPRESENTATIVE WORKS

Theme	[2]	[8]	[9]	[10]	[11]	[43]	[44]	[12]	[13]	[49]	[19]	[52]	[22]	[14]	[15]	[16]	[57]	[58]	[38]	[35]	[42]
Scale			✓	◦																✓	✓
Congestion sensing	◦											◦	◦								
Forecasting	✓						✓	✓	✓												
Validation (field)		◦	✓	✓				◦		◦	✓										
Security	◦		✓	✓	✓	✓	Pricing		✓			✓			✓	✓					

2) *Measured congestion.* Vision work addresses bay occupancy [22], [52], and detectors for vehicles are mature [53], but we found no cited parking work that closes the loop from lane counts to routing outcomes. See Section IV-B.
3) *Forecasting with uncertainty.* Stronger forecasters exist [12], [13], [44], but reservation rules that consume forecast intervals are not part of the cited designs. See Section IV-C.
4) *Evidence strength.* Field-tested systems and logic-rich systems are largely different papers. See Section IV-D and the ladder above.
5) *Service-level security and pricing calibration.* Protocol security is studied in depth [16], [58], and pricing in the field has been evaluated [19], but neither is connected to sensor-based parking decisions end to end. See Sections IV-E and IV-F.

Table X makes the pattern explicit. A check mark means that the theme is a stated focus of the work; a circle means that the work touches the theme partially or briefly; a blank means that, to our reading, the theme is not a stated focus. The marks reflect the stated focus of each work and are not a ranking of quality: a work with few marks may be excellent at what it set out to do. We have been conservative, so a blank may hide a partial treatment that we did not recognize from the available description.

The matrix shows columns with isolated marks and rows with a few filled cells. Security and scale have deep but separate bodies of work; forecasting and pricing are well populated but rarely joined; congestion sensing and field validation are thinly covered in the cited set. The thin rows (congestion sensing and field validation), together with the missing links between rows, define the agenda that Sections IV and V take up.

III. The Reference System in Technical Detail

This section restates the four functions of the reference system, bay selection, availability estimation, pricing, and security alerting, in a more formal form, and then summarizes how the system was validated. The formalisation uses generic notation chosen to be consistent with the description of the reference system given in Section II; it is our reconstruction and not a quotation of the original equations. Wherever we need a number or a functional form that that description does not fix, we pick an illustrative instance and say so. All numerical examples below are illustrative, not data from the reference paper [2], and every figure quoted was recomputed in Python. The purpose is twofold: to make the mechanisms concrete enough to reason about their failure modes in Section IV, and to show precisely which details a reader would need in order to re-implement them.

A. System Model and Notation

The lot is modeled as an undirected graph $G = (V,E)$. Vertices are junctions, including a designated entrance s and the junctions that give access to bays; edges are lane segments between junctions. The description states that each edge carries a congestion figure derived from lane occupancy [2]. In generic form, an edge e of length L_e and capacity N_e vehicles, currently holding n_e vehicles, has

$$c_e = \frac{n_e}{N_e} \in [0,1]. \tag{6}$$

The description also says the figure is averaged along the route. Dijkstra's algorithm needs a cost that is additive over edges, so we work with per-edge costs and treat a routelevel average as a derived reporting quantity. The reference description only says that the edge carries a congestion figure; it does not give the cost function. One common choice in the traffic-assignment literature is a travel time that grows with congestion,

$$w_e = \frac{L_e}{v_{\text{free}}}\left(1 + \alpha c_e\right)^{\beta}, \tag{7}$$

with free-flow speed v_{free} and shape parameters $\alpha,\beta > 0$. We use Eq. (7) only as a stand-in; nothing in the argument depends on its particular form beyond $w_e > 0$ and monotonicity in c_e. Table XI collects the symbols used in this section.

B. Bay Selection by Shortest Path

1) Formulation: Given the current costs w_e, the entrance s, and the set $B \subseteq V$ of junctions that give access to a free bay, the selector returns the bay-access junction with the least total cost and the route to it:

$$t^{\star} = \operatorname{argmin}_{t \in B} d(t), \quad d(t) = \min_{\pi: s \to t} \sum_{e \in \pi} w_e .$$

One run of Dijkstra's algorithm [6] from s settles all vertices in order of increasing distance, so a single run answers the multitarget question; the search can stop at the first settled vertex in B. Correctness requires non-negative edge costs, which Eq. (7) guarantees. Algorithm 1 gives the procedure in the priorityqueue form.

With a binary heap each edge triggers at most one insertion and each insertion costs $O(\log V)$, so the running time is $O((V + E)\log V)$. For a junction graph of $V = 100$ and $E = 300$, the bound evaluates to about 2.7×10^3 elementary steps, a negligible load for a Raspberry Pi-class node even if the search is repeated for every arriving vehicle. The computational cost of routing is therefore not the constraint at this scale; the quality of the congestion input is.

TABLE XI
NOTATION USED IN THIS SECTION

Symbol	Meaning
$G = (V,E)$	Junction graph; s entrance, t target bay node
L_e, N_e, n_e	Length, capacity, and current vehicle count of edge e
c_e	Congestion of edge e, Eq. (6)
w_e	Edge cost (here seconds), Eq. (7)
$\alpha,\beta,v_{\text{free}}$	Illustrative cost parameters
$d(v)$	Tentative or final distance label of v from s
X_m	Occupancy state after m five-minute steps, in $\{1,\ldots,5\}$
$\mathbf{P}$, π	Transition matrix; stationary distribution
θ, τ	State threshold and probability threshold of the acceptance rule
r, $\delta(h)$	Hourly rate and discount at stay length h
k, $b_{(j)}$	Spaces offered; j-th highest bid
$R(j)$	Revenue if the j highest bidders win at price $b_{(j)}$

Algorithm 1 Bay selection by Dijkstra's algorithm

Require: Graph $G = (V,E)$, costs $w_e > 0$, source s, free-bay set B
Ensure: Target $t^{\star}$ and route
$d(v) \leftarrow \infty$ for all v; $d(s) \leftarrow 0$; pred $\leftarrow \emptyset$
$Q \leftarrow$ min-priority queue containing $(0,s)$
while $Q \neq \emptyset$ do
  $(d_u,u) \leftarrow$ extract-min of Q
  if u already settled then continue
  end if
  mark u settled
  if $u \in B$ then break
  end if
  for each edge $\{u,v\}$ with cost w do
    if $d_u + w < d(v)$ then
      $d(v) \leftarrow d_u + w$; pred$(v) \leftarrow u$; insert $(d(v),v)$
    end if
  end for
end while
$t^{\star} \leftarrow u$; return $t^{\star}$ and the path obtained by following pred back to s

2) Worked example: Consider the seven-junction graph of Fig. 3, with entrance S and the bay-access junction T. The lengths, congestion values, and the resulting costs from Eq. (7) with $v_{\text{free}} = 5$ m/s, $\alpha = 1.5$, and $\beta = 2$ are listed in Table XII. All values are hypothetical. For instance, edge S–B has $L = 140$ m, so $L/v_{\text{free}} = 28$ s, and $c = 0.1$ gives $28(1 + 1.5 \cdot 0.01) = 28.42$ s.

TABLE XII
ILLUSTRATIVE EDGE DATA FOR THE SEVEN-JUNCTION EXAMPLE

Edge	L_e (m)	L_e/v_{free} (s)	c_e	w_e (s)
S–A	100	20	0.2	21.20
S–B	140	28	0.1	28.42
A–B	60	12	0.3	13.62
A–C	120	24	0.2	25.44
B–D	100	20	0.2	21.20
C–D	70	14	0.4	17.36
C–T	80	16	0.6	24.64
D–T	90	18	0.3	20.43
D–E	50	10	0.1	10.15
E–T	60	12	0.3	13.62

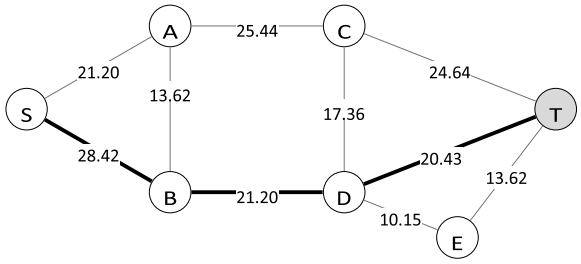


Fig. 3. Illustrative junction graph. Labels are edge costs w_e in seconds (Table XII); the thick path S–B–D–T (70.05 s) is the least-cost route.

TABLE XIII
DIJKSTRA TRACE ON THE EXAMPLE GRAPH (DISTANCE LABELS IN SECONDS)

Step	Settled	A	B	C	D	E	T
0	—	∞	∞	∞	∞	∞	∞
1	S (0)	21.20	28.42	∞	∞	∞	∞
2	A (21.20)	21.20	28.42	46.64	∞	∞	∞
3	B (28.42)	21.20	28.42	46.64	49.62	∞	∞
4	C (46.64)	21.20	28.42	46.64	49.62	∞	71.28
5	D (49.62)	21.20	28.42	46.64	49.62	59.77	70.05
6	E (59.77)	21.20	28.42	46.64	49.62	59.77	70.05
7	T (70.05)	21.20	28.42	46.64	49.62	59.77	70.05

Table XIII traces the algorithm. After each settling step the table shows the distance labels; the label of S is 0 throughout. Vertex C is settled before D because $46.64 < 49.62$, and the label of T first becomes 71.28 through C and is lowered to 70.05 when D is settled. Relaxing from E offers $59.77 + 13.62 = 73.39$, which does not improve T. Following the predecessors back from T gives $S \to B \to D \to T$ at 70.05 s.

If congestion were ignored ($c_e = 0$ everywhere), the shortest route would be $S \to A \to C \to T$ at 60.0 s, because C–T is short. The congestion-weighted costs reverse that choice, since C–T at $c = 0.6$ costs 24.64 rather than 16 s. This is the mechanism through which the reported arrival-time benefit would arise, and also the reason it depends so heavily on the congestion input.

3) Sensitivity to a wrong congestion input: Because the congestion values in the reference simulation were set by hand between 0.1 and 0.9 (Section IV-B), it is instructive to ask how the decision reacts to error in a single edge. Table XIV varies c on edge D–T with all else fixed. The chosen route flips between $c = 0.36$ and $c = 0.37$; solving $49.62 + 18(1 + 1.5c^2) = 71.28$ gives the exact switching point $c^{\dagger} \approx 0.368$. Above it the cost saturates at 71.28 s, because the selector has left the edge. The input error matters only when it crosses such a threshold, but the consequence can then be sizable. Suppose the system believes $c_{DT} = 0.3$ while the true value is 0.9. It sends the vehicle along $S \to B \to D \to T$, whose true cost is 89.49 s, whereas the true optimum is 71.28 s: a regret of 18.21 s, or 25.5%. A similar threshold exists for C–T at $c \approx 0.556$. In a real lot with many near-tied routes, thresholds are dense, so small estimation errors can change many routing decisions even when the average cost penalty remains modest.

TABLE XIV
EFFECT OF CONGESTION ON EDGE D–T ON THE CHOSEN ROUTE

c_{DT}	Chosen route	Cost (s)
0.10	$S \to B \to D \to T$	67.89
0.30	$S \to B \to D \to T$	70.05
0.36	$S \to B \to D \to T$	71.12
0.37	$S \to A \to C \to T$	71.28
0.50	$S \to A \to C \to T$	71.28
0.90	$S \to A \to C \to T$	71.28

Two further limits of the formulation deserve mention. The route is optimal for a snapshot of congestion; vehicles that take several minutes to cross a lot experience conditions that have changed by the time they reach later edges, and the description does not mention re-planning en route. And the cost is the same for every driver, so many vehicles sent simultaneously to the same cheapest corridor will themselves raise its congestion, an effect that a one-shot shortest-path selector does not anticipate.

C. Availability as a Five-State Markov Chain

1) States and transition matrix: In the description, a lot is assigned to one of five occupancy states, and a transition matrix built from historical counts in five-minute windows decides whether a reservation made at least three hours ahead can be accepted [2]. The band edges are not given in the description; as an illustration we use five equal bands of occupied share: state 1 for 0–20%, state 2 for 20–40%, state 3 for 40–60%, state 4 for 60–80%, and state 5 for 80– 100%. The chain $X_0, X_1, \ldots$ advances one step per five minutes. For a row-stochastic $\mathbf{P}$ with $p_{ij} = \Pr(X_{m+1} = j \mid X_m = i)$, the maximum-likelihood estimate from counts is $\hat{p}_{ij} = n_{ij} / \sum_{j'} n_{ij'}$, where n_{ij} counts observed transitions from i to j. An illustrative matrix, not estimated from any data, is

$$\mathbf{P} = \begin{bmatrix} 0.80 & 0.18 & 0.02 & 0 & 0 \\ 0.10 & 0.74 & 0.14 & 0.02 & 0 \\ 0.02 & 0.12 & 0.72 & 0.12 & 0.02 \\ 0 & 0.02 & 0.14 & 0.74 & 0.10 \\ 0 & 0 & 0.02 & 0.18 & 0.80 \end{bmatrix}. \tag{8}$$

Estimation error is a practical concern: if a row is estimated from $n = 20$ observed transitions and the true self-transition probability is 0.8, the standard error of $\hat{p}_{ii}$ is $\sqrt{0.8 \cdot 0.2/20} \approx 0.089$, and about 246 observations per row are needed to bring the 95% half-width down to 0.05. Rows for the extreme states are the ones with the fewest observations in most lots.

2) Multi-step transitions and the stationary distribution: By the Chapman–Kolmogorov equations [45], [60], the n-step transition matrix of a time-homogeneous chain is $\mathbf{P}^{(n)} = \mathbf{P}^n$, so $\Pr(X_{m+n} = j \mid X_m = i) = (\mathbf{P}^n)_{ij}$. For an irreducible, aperiodic chain, $\mathbf{P}^n$ converges to a matrix whose rows all equal the stationary distribution $\boldsymbol{\pi}$, the solution of $\boldsymbol{\pi}\mathbf{P} = \boldsymbol{\pi}$ with $\sum_i \pi_i = 1$. For the matrix in Eq. (8),

$$\boldsymbol{\pi} = \tfrac{1}{99}\,(14,\ 23,\ 25,\ 23,\ 14) \approx (0.141,\ 0.232,\ 0.253,\ 0.232,\ 0.141),$$

and the second-largest eigenvalue modulus is 0.9, which sets the geometric rate at which the rows approach $\boldsymbol{\pi}$.

3) Extending five minutes to three hours: A reservation accepted at least three hours ahead needs the state distribution 36 steps away. Under a time-homogeneous chain the only available route is $\mathbf{P}^{36}$. For the illustrative matrix, rounded to three decimals,

$$\mathbf{P}_{36} \approx \begin{bmatrix} 0.149 & 0.240 & 0.253 & 0.225 & 0.134 \\ 0.145 & 0.236 & 0.253 & 0.228 & 0.137 \\ 0.141 & 0.232 & 0.253 & 0.232 & 0.141 \\ 0.137 & 0.228 & 0.253 & 0.236 & 0.145 \\ 0.134 & 0.225 & 0.253 & 0.240 & 0.149 \end{bmatrix}.$$

The rows are almost identical. The largest total-variation distance between any row and π is 0.0145 at $n = 36$, and the largest distance between two rows is 0.029. After three hours the chain has nearly forgotten where it started: the forecast is the long-run average of the lot. Fig. 4 shows the decay, which follows 0.9^n up to a constant. For this matrix the distance to π drops below 0.01 after about 40 steps, or 3.3 hours.

This is a property of the model class and not a defect of one matrix. Any time-homogeneous ergodic chain mixes at a geometric rate, and a lot whose occupancy changes slowly relative to the step size mixes more slowly than ours but still converges. The consequence is that the information in the current state, the very thing a reservation decision would like to exploit, shrinks with the horizon, while the answer approaches an average that ignores time of day. Trends and time-of-day effects can enter such a chain only through separate per-period matrices (Section IV-C), and the description does not state how matrices for successive periods would be chained over a three-hour horizon. A natural option, which we flag as one possibility and not as the authors' method, is a product of period-specific matrices $\mathbf{P}_t\mathbf{P}_{t+1}\cdots\mathbf{P}_{t+35}$.

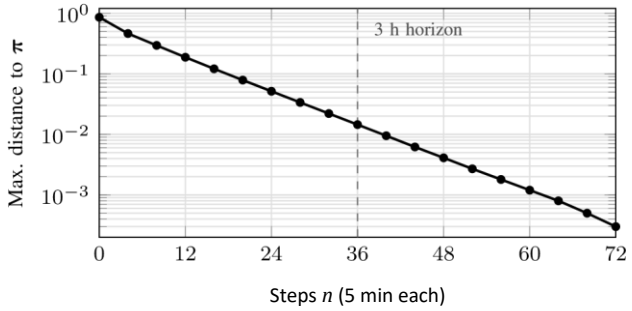


Fig. 4. Mixing of the illustrative chain of Eq. (8): largest total-variation distance between any row of $\mathbf{P}^n$ and π. At the three-hour horizon the rows differ from the stationary distribution by about 1.4%.

TABLE XV
ILLUSTRATIVE ACCEPTANCE PROBABILITY $\Pr(X_{t+n} \le 3 \mid X_t = i)$

Horizon	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
$n = 1$ (5 min)	1.000	0.980	0.860	0.160	0.020
$n = 6$ (30 min)	0.923	0.829	0.659	0.433	0.256
$n = 12$ (1 h)	0.801	0.729	0.631	0.526	0.439
$n = 36$ (3 h)	0.641	0.634	0.626	0.618	0.612

4) A generic acceptance rule: The description says that the matrix decides whether a reservation made at least three hours ahead can be accepted, without giving the rule. A generic form is: with current state $X_t = i$ and threshold θ,

$$\text{accept} \iff \Pr(X_{t+36} \le \theta \mid X_t = i) = \sum_{j \le \theta} (\mathbf{P}^{36})_{ij} \ge \tau. \tag{9}$$

Here θ names the highest occupancy band in which a space is still expected to be available, and τ is the required confidence. Table XV evaluates the left side for $\theta = 3$ (occupancy at most 60%) at several horizons.

With $\tau = 0.7$, a request made when the lot is in state 1 through 3 would be accepted at five minutes' notice, but at three hours' notice every starting state is rejected, since the largest value is 0.641. With $\tau = 0.6$ every starting state is accepted. The decision at the three-hour horizon is thus almost independent of the current state: the spread between best and worst start is 0.029, and the stationary value $\sum_{j \le 3} \pi_j = 0.626$ lies in between. Whatever τ is chosen, the rule at that horizon becomes close to an all-or-nothing policy, which is a poor basis for deciding individual bookings. Two other gaps are visible. The rule as stated treats a booking as an observation of occupancy, but bays already reserved for the interval reduce what remains, so acceptance needs a count of outstanding reservations that a state-only chain does not contain. And without a stated θ, τ, and band structure the rule cannot be reproduced.

D. Hybrid Pricing

1) Short stays: hourly rate with stepwise discounts: The description gives an hourly rate with stepwise discounts for short stays and does not state the breakpoints or discount

TABLE XVI
WHOLE-STAY VERSUS MARGINAL APPLICATION OF THE DISCOUNT (r = USD 2/H, ILLUSTRATIVE)

Stay h (h)	$F_w(h)$ (USD)	$F_m(h)$ (USD)
1.00	2.000	2.000
1.01	1.818	2.018
2.00	3.600	3.800
2.01	3.216	3.816
4.00	6.400	7.000
4.01	8.020	7.020
8.00	16.000	15.000

levels. A generic piecewise form with illustrative breakpoints at 1, 2, and 4 hours and discounts of 0%, 10%, and 20% is

$$\delta(h) = \begin{cases} 0, & 0 < h \le 1, \\ 0.10, & 1 < h \le 2, \\ 0.20, & 2 < h \le 4, \\ 0, & \text{otherwise.} \end{cases}$$

$$2 \quad < \quad h \le 4,\ h > 4,$$

where h is the stay length in hours. Two ways to apply it differ materially. If the discount applies to the whole stay, the fee is $F_w(h) = rh(1 - \delta(h))$. If it applies only to the hours inside each band, as in a marginal tax schedule, the fee is $F_m(h) = r\sum_i (1-\delta_i)|[0,h] \cap I_i|$ over the bands I_i. Table XVI compares them for an illustrative rate of r = USD 2 per hour.

The whole-stay variant produces non-monotone fares: staying 1.01 hours costs less than staying exactly 1 hour, and parking 4.01 hours costs USD 1.62 more than 4.00 hours. Such cliffs create incentives to time departures just after a breakpoint and are hard to explain to users. The marginal variant is continuous and monotone. The description does not say which is used, and it matters for any revenue comparison against a fixed rate, because in this example the two variants differ by USD 0.60 per stay at four hours and by USD 1.00 at eight hours.

2) Long-term places: a multi-unit uniform-price auction: For long-term places, subscribers bid, and the description says the price is chosen to maximize revenue. In generic terms there are k identical spaces and n bids, sorted so that $b_{(1)} \ge b_{(2)} \ge \cdots \ge b_{(n)}$. If the j highest bidders each pay a common price equal to the j-th highest bid, all of them accept, and the revenue is

$$R(j) = j\, b_{(j)}, \qquad j^{\square} = \arg \max_{1 \le j \le \min(k,n)} R(j). \tag{10}$$

This is a uniform-price rule of the kind studied in the multiunit auction literature [51]. It is revenue-maximizing only for the reported bids; it says nothing about whether those bids are truthful. When several bids are tied at the clearing price and more bidders tie than there are spaces, a tie-break is needed (random draw, arrival order); the description we have does not specify one.

Section IV-F reports that the worked example of the reference paper, with 20 bids of USD 200, 3 of USD 150, and 7 of USD 90, names USD 150 as optimal. We recomputed the

TABLE XVII
REVENUE (USD) OF THE WORKED BIDDING EXAMPLE BY NUMBER OF SPACES

Spaces k	Optimal price	Revenue at optimum	At 150	At 90
5	200	1000	750	450
10	200	2000	1500	900
20	200	4000	3000	1800
23	200	4000	3450	2070
30	200	4000	3450	2700

example. The revenue function is $R(j) = 200j$ for $j \le 20$, then $R(21) = 3150$, $R(22) = 3300$, $R(23) = 3450$, and $R(j) = 90j$ for $j \ge 24$, with $R(30) = 2700$. The maximum over $j \le \min(k,30)$ is at $j = \min(k,20)$, giving price USD 200. Table XVII compares the optimum with the outcomes at fixed uniform prices of USD 150 and USD 90, computed as $p \cdot \min\big(k, \#\{b_i \ge p\}\big)$.

For no k is USD 150 optimal. At $k = 23$, where it comes closest because all three USD 150 bidders can be served in addition to the 20 at USD 200, it still yields 3450 against 4000, a shortfall of 13.75%. USD 90 loses 32.5% at $k = 30$. A further observation is that USD 150 would become the best price only if the 20 top bids were below $3450/20 = 172.5$, for $k \ge 23$; with bids at USD 200 it cannot be. Note also that for $k < 20$ the optimal price leaves $20 - k$ bidders at the top price unserved, so a tie-break rule decides who receives the spaces.

E. Security Alerting

The fourth function, security alerting, appears in the workflow description but is not developed (Section II). We therefore give no formalisation. In particular, no alert condition, no data it consumes, and no response action is available to us, and for that reason a generic instance would be inventing the function rather than restating it. Section V proposes a concrete definition based on plausibility checks, and the evaluation plan of Section VI includes injected attacks.

F. Validation Set-Up

1) What was built and what was simulated: The system was validated through a hardware prototype and through a SUMO model of the Kudai parking area [2], [7]. Table XVIII separates what each exercised from what it assumed, using only the description in this paper.

The simulation parameters have a simple consequence. With 10 arrivals and 5 departures per minute, the net inflow is 5 vehicles per minute. If every arriving vehicle parks and departures are drawn from parked vehicles, a lot with the maximum initial availability of 90% (135 of 150 spaces free) fills in $135/5 = 27$ minutes, and a half-empty lot (75 free) in 15 minutes. In a five-minute window the expected net change is 25 vehicles, or 16.7% of capacity, close to the width of one of our illustrative bands. This is our reading of the stated rates under a deliberately simple assumption, not a statement about the reference results. It suggests that the simulated regime is

TABLE XVIII
PROTOTYPE AND SIMULATION: EXERCISED VERSUS ASSUMED

Element	Prototype	SUMO simulation
Sensing	Ultrasonic, magnetic, RFID over LoRa; camera omitted	Congestion set to chosen values in 0.1–0.9
Fog node	One Raspberry Pi; 10 ms processing asserted	Not applicable
Bay selection	Not described; demonstration covers a simplified fragment of the lot	Dijkstra vs. blind search

Availability	Not described	Rates 10 arrivals, 5 departures per min; initial availability up to 90%
Pricing	Not described	Random bids above default rate; tiered short-term fares
Scale	Part of one lot (about 2,500 spaces)	Single lot of 150 spaces

TABLE XIX
CLAIMS AND THE EVIDENCE BEHIND THEM

Claim	Evidence type	Status
Arrival time reduced by 16–46%	SUMO, congestion set by hand, blind-search baseline	Simulated; input assumed
Revenue up 10– 15%	SUMO, synthetic bids above default rate	Simulated; inputs synthetic
Fog node processes data every 10 ms	Stated	Asserted, not measured
Reservations accepted 3 h ahead	Markov chain, five-minute windows	No error reported; horizon extension unstated
Hardware chain	Prototype with one fog node	Small-scale demonstration only

a lot filling up and not one near equilibrium, a very different regime from the stationary behavior discussed in Section III-C.

2) Mapping claims to evidence: Table XIX lists the headline claims and the kind of evidence behind each, to be read with the analysis of Section IV.

G. What Can and Cannot Be Re-Implemented

A useful test of a technical description is whether a second group could rebuild the system and obtain comparable numbers. Table XX lists the items such a rebuild requires and whether the description in this paper fixes them. A "No" means that this paper's account does not fix the item; it does not show that the original is silent, so anyone re-implementing the system should check the original text for each such item. Entries marked with a dagger are ones that Section IV reports as missing from the reference paper itself.

Of eighteen items, four are fixed, seven are partly fixed, and seven are not. The pattern is informative. The structure of each function is clear, since the algorithm, the number of states, and the headline parameters are all given. The parameters that determine the numerical outcome are what is missing: cost shape, band edges, thresholds, discount levels, bid distributions. Section IV-D argued that the simulated gains describe idealized conditions; Table XX adds that, even in simulation, a faithful reproduction would have to supply many of these values by assumption. For that reason the evaluation plan of Section VI begins by reproducing the published configuration as a sanity check, and records every assumed value alongside the results.

IV. LIMITATION ANALYSIS OF THE REFERENCE SYSTEM

To keep the analysis systematic, we read the reference paper [2] against three questions: what was actually built, what was actually measured, and what was merely assumed. Six limitations emerge. For each one we describe what the paper offers, why the gap matters for deployment, and what evidence is missing.

A. Restricted Prototype Scale

The hardware demonstration covers a simplified fragment of the Kudai lot, whereas the lot itself has about 2,500 spaces [2]. The simulated lot is smaller still, with a capacity of 150 spaces. A single Raspberry Pi served as the only fog node, and the paper does not report how much load one node can carry, how its response time changes as sensors multiply, or how many packets are lost between sensors and gateway. The statement that fog nodes process incoming data every 10 ms is therefore asserted rather than measured. Scale also matters for a radio-level reason: LoRa end devices transmit without first sensing the channel, so collisions become more frequent as node density grows [5], and a lot with thousands of bay sensors is a very different environment from a demonstration board with a handful of them. Cooperation among several fog nodes, mentioned in the paper only as a possible collaborative task allocation, is likewise untested. Without such evidence it is unclear whether the reported gains would survive a full-size or multi-lot deployment.

B. Missing Camera-Based Congestion Sensing

The bay-selection algorithm relies on a congestion figure for each lane, computed as the ratio of vehicles present to lane capacity and averaged along the route [2]. In the architecture, this information is meant to come from cameras at junctions. The prototype, however, left out the visual sensor for simplicity, and in the simulation the congestion level was simply set to chosen values between 0.1 and 0.9. The step that turns raw video into a usable congestion estimate was therefore never exercised. That step is far from trivial: counting vehicles per lane under changing light, occlusion, and weather is a research problem in its own right, and vision-based presence sensing has been studied as a system component by itself [52]. Video streams also place demands on bandwidth and on the fog node's computing budget that low-rate LoRa sensor traffic does not, and cameras raise privacy questions because they can capture number plates and faces. Until the congestion input comes from real sensing, the reported arrival-time benefit rests on an assumed rather than an observed quantity.

TABLE XX
REQUIREMENTS FOR RE-IMPLEMENTATION AND THEIR COVERAGE IN THE DESCRIPTION

Item needed	Fixed?	What is available / what is missing
Junction graph and edge lengths	No	Kudai area reproduced in SUMO; graph structure and lengths not described
Congestion definition	Yes	Vehicles over lane capacity, averaged along the route
Cost as a function of congestion	Partly	Congestion figure used by Dijkstra; functional form not described
Congestion values in simulation	Partly	Set between 0.1 and 0.9; assignment per edge not described
Shortest-path algorithm	Yes	Dijkstra
Number of occupancy states	Yes	Five
State band edges	No	Not described
Transition matrix values	Partly	Estimated from historical counts in five-minute windows; counts not given
Extension to three-hour horizon†	No	Not stated
Acceptance rule parameters	Partly	Reservations at least three hours ahead; θ, τ not described
Forecast error†	No	Not reported
Discount breakpoints and levels	No	Stepwise discounts only; whole-stay or marginal not stated
Long-term pricing rule	Partly	Bidding, revenue objective; worked example inconsistent with it†
Bid generation	Partly	Random, above default subscription rate; distribution not described
Security alert generation†	No	Listed in workflow; not developed
Simulation parameters	Yes	spaces, 10 arrivals and 5 departures per min, 90% maximum initial availability
Baselines	Partly	Blind search; fixed-rate pricing; no simpler guidance policy
Repetitions and intervals†	No	No indication of repeated runs or confidence intervals

C. Simplistic Availability Forecasting

Availability is forecast with a classic Markov chain over five occupancy states, using a transition matrix estimated from five-minute windows in which arrival and departure rates are assumed constant [2]. Three weaknesses follow. First, a Markov chain is memoryless: the next state depends only on the present one, so trends, time-of-day effects, and external events can enter only through separate per-period matrices, which must then be estimated from few observations; the paper's sample data list a single averaged value per year for a given date and time. Second, the constant-rate assumption is exactly the one that fails during sudden surges, closures, or emergencies. Third, the paper reports no forecasting error, and the text does not state how a five-minute matrix is extended to the three-hour horizon that governs reservations. Other studies, in contrast, have compared regression trees, support vector regression, and neural networks for this task [12] and applied neural networks to multi-step forecasts [13]. The reference paper offers no comparison of that kind, so it cannot show that the Markov model is adequate, let alone the best choice.

D. Simulation-Dominated Validation

The performance claims come almost entirely from SUMO experiments [7]. The baseline is a driver who searches blindly, which is a relatively weak competitor; the paper does not compare against simpler guidance policies, such as sending each driver to the nearest known free bay. Key inputs, namely initial availability, congestion, and the arrival and departure rates, were chosen by the authors, and the reported curves give no indication of repeated runs or confidence intervals. Real conditions such as heat, dust, sensor drift, radio interference, and lost packets were not represented, and the authors themselves name the construction of a test field as future work [2]. The headline improvements therefore describe what the algorithm can achieve under idealized conditions and should not be read as expected field performance.

E. Shallow Treatment of Security and Privacy

The system handles reservations, payments, vehicle positions, and RFID or QR credentials, all of which are attractive targets. The reference paper acknowledges security only briefly: it notes that encryption could be used in the cloud layer and lists security alerting among its algorithms, yet it gives no threat model, no key-management or device-authentication design, and no account of how alerts are generated [2]. Several attacks are plausible. Forged occupancy reports could mislead the availability estimator; replayed or injected LoRaWAN frames and rogue gateways are documented threats [16]; and floods of reservation requests could block legitimate users. Location traces from GPS, combined with payment data, would also make drivers linkable across visits. Earlier work shows that protocol-level protection [14] and hardware-backed key storage [15] are feasible for parking systems, which highlights how much of this layer the reference design leaves unspecified.

F. Synthetic Bidding Data in Pricing

The hybrid pricing scheme was assessed with simulated demand. In the long-term plan, bids were generated randomly above the default subscription rate, so a revenue increase over the fixed rate is close to guaranteed by construction. The

evaluation also states that the tiered discount made shortterm revenue lower than under fixed pricing, which means the overall gain of 10–15% comes entirely from the long-term component, whose inputs were synthetic [2]. In addition, we could not reproduce the paper’s worked example from its own objective. With 20 bids of USD 200, three of USD 150, and seven of USD 90, a single uniform rate of USD 200 would earn more than USD 150 for any number of spaces on offer, yet

TABLE XXI
SUMMARY OF LIMITATIONS AND THEIR DEPLOYMENT RISK

Limitation	Evidence gap	Deployment risk
Prototype scale	Part of one lot; 150-space simulation versus a 2,500-space lot	Unknown node load, radio collisions, multi-lot behavior
Congestion sensing	Camera omitted; congestion set manually	Core routing input unverified
Forecasting	Fixed-rate Markov model; no error reported	Wrong reservation decisions under irregular demand
Validation	SUMO only, with a blind-search baseline	Gains may not hold in the field
Security and privacy	No threat model or key management	Spoofing, replay, data leakage
Pricing	Random bids above the default rate	Revenue gain not established

the example names USD 150 as optimal. The mechanism also ignores strategic behavior, since bidders who expect the price to be lowered until demand clears may shade their bids. Real willingness-to-pay data, or at least a demand model calibrated on observed occupancy, would be needed before the revenue claim can be trusted.

G. Summary of Findings

Table XXI condenses the six limitations. They share a pattern: each component was demonstrated in isolation, at small scale, and under assumed inputs. The next section proposes a matching improvement for every row.

H. Deep Dive: Scale and Radio Capacity

Section IV-A argued that a 150-space simulation and a handful of prototype sensors say little about a lot of about 2,500 spaces. This subsection turns that qualitative argument into numbers. All results are analytical: they follow from a stated traffic model, a standard time-on-air formula, and an ALOHA collision model. No figure below is a measurement of the reference system or of any deployment, and every parameter that the reference paper does not report is marked as illustrative.

1) Traffic model and assumptions: Consider N bay sensors served by one gateway. Each sensor sends an *event report* whenever its bay changes state, and a *heartbeat* every T_h if nothing else has been sent. If a bay changes state at rate λ_e events per hour (one arrival plus one departure per stay, so $\lambda_e = 2\tau$ for a turnover of τ stays per bay per hour), the per-sensor packet rate is

$$r = \frac{\lambda_e}{3600} + \frac{1}{T_h} \quad \text{packets/s}, \qquad \Lambda = N\,r. \tag{11}$$

We adopt $\lambda_e = 1$ h^{-1} as an illustrative busy-hour value, and $T_h \in \{60, 15\}$ min. With $T_h = 60$ min every sensor then sends two packets per hour ($r = 2$ h^{-1}), and with $T_h = 15$ min five. We make six simplifying assumptions explicit, because the conclusions depend on them.

A1 Packet starts form a Poisson process (independent, memoryless). Event reports are in fact correlated, since a car park empties in bursts after an event, so A1 is optimistic at peak times.

A2 All sensors use the same spreading factor (SF), bandwidth 125 kHz, and coding rate 4/5, and share one channel. This is a bounding case; Section IV-H8 relaxes it.

A3 Any overlap in time of two frames on the same channel and SF destroys both (no capture effect). This is pessimistic wherever a stronger frame would survive a collision.

A4 Frames with different SFs are treated as perfectly orthogonal. Real SFs are only quasi-orthogonal and interfere when the interferer is strong [38]; the idealization therefore overstates the benefit of mixing SFs.

A5 The link budget is sufficient, so loss is caused by collisions only. Fading, obstruction by vehicles, and external interference are ignored.

A6 The 12-byte frame length is used as the full physicallayer payload. LoRaWAN adds at least 13 bytes of MAC overhead [34], which we quantify below.

2) Time on air: The time on air (ToA) follows the standard LoRa formula of Eqs. (2)–(5), with $n_{\text{pre}} = 8$, explicit header, CRC on, coding rate 4/5, and low-data-rate optimization at $SF \geq 11$ for $BW = 125$ kHz. For $PL = 12$ bytes the payload has $n_{\text{pay}} = 28$, 23, and 23 symbols at SF7, SF9, and SF12, and the ToA is 41.2, 144.4, and 1,155.1 ms (Table IV). The nominal bit rate falls from about 5.5 kb/s at SF7 to 0.29 kb/s at SF12, while the airtime grows by a factor of 28. Under A6, adding a 13-byte MAC overhead (a 25-byte frame) raises the ToA to 61.7, 205.8, and 1,482.8 ms, that is by 50%, 43%, and 28%; the offered loads below scale by the same factors and so are optimistic.

3) Offered load and ALOHA success probability: The offered load is the mean number of frames in the air, $G = \Lambda T_{\text{air}}$. A frame of duration T_{air} overlaps another frame if that frame starts anywhere in a window of length $2T_{\text{air}}$ around its own start (the previous T_{air} and the following T_{air}). Under A1 the number of starts in this vulnerable window is Poisson with mean $2G$, so the probability of no overlap, i.e., the packet success probability, is

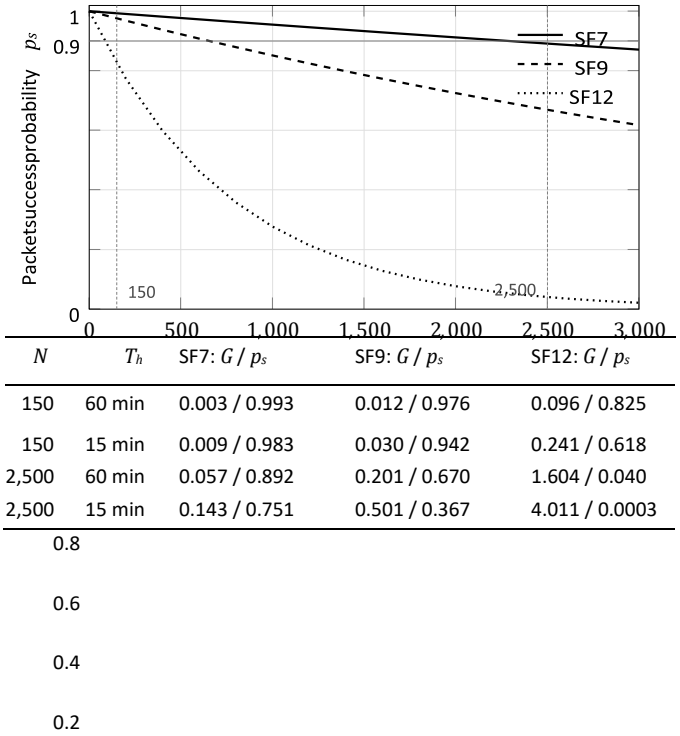
$$p_s = e^{-2G}. \tag{12}$$

Pure ALOHA throughput Ge^{-2G} peaks at $G = 0.5$ with value $1/(2e) \approx 0.184$. LoRa end devices do not sense the channel before sending, which is why this model is the standard first approximation for LoRaWAN scaling studies [35], [38]. Table XXII gives G and p_s for the two lot sizes. The 150space lot is almost insensitive to the choice of SF; the 2,500space lot is not. At SF12 with hourly heartbeats the model predicts $p_s = 0.04$,

TABLE XXII

and at SF9 with 15-minute heartbeats $p_s = 0.37$. Inverting Eq. (12), the number of sensors a single channel can carry at a target success probability p^* is $N_{\max} = -\ln(p^*)/(2rT_{\text{air}})$. For $p^* = 0.9$ and $r = 2$ h^{-1} this gives about 2,300 sensors at SF7, 656 at SF9, and 82 at SF12; with 15-minute heartbeats ($r = 5$ h^{-1}) it gives 920,

OFFERED LOAD G AND SUCCESS PROBABILITY p_s (ILLUSTRATIVE, SINGLE CHANNEL, ALL SENSORS AT THE SAME SF)



N	T_h	SF7: G / p_s	SF9: G / p_s	SF12: G / p_s
150	60 min	0.003 / 0.993	0.012 / 0.976	0.096 / 0.825
150	15 min	0.009 / 0.983	0.030 / 0.942	0.241 / 0.618
2,500	60 min	0.057 / 0.892	0.201 / 0.670	1.604 / 0.040
2,500	15 min	0.143 / 0.751	0.501 / 0.367	4.011 / 0.0003

Fig. 5. Pure-ALOHA packet success probability versus number of sensors for 12-byte frames, $r = 2$ h^{-1} per sensor, one channel, one SF per curve (computed from Eqs. (5) and (12)).

262, and 32. The 150-bay lot thus fits comfortably under every combination except SF12, and its simulation could not have revealed that the full lot will not.

Fig. 5 plots Eq. (12) against N for $r = 2$ h^{-1}. The two vertical markers show the simulated lot (150) and the real lot (about 2,500); the horizontal line marks $p_s = 0.9$. The curves separate sharply with SF, which is why a design that lets every sensor fall back to the most robust SF is not a scalable default.

4) Regulatory and gateway limits: Three further constraints sit on top of collisions. First, many regions limit the fraction of time a transmitter may occupy a band (a duty cycle, commonly of the order of 1% in some unlicensed sub-bands). The limits are regional, differ between sub-bands, and may take other forms such as dwell-time rules, so they must be checked against the rules of the deployment country and not assumed. If a 1% limit applies, a sensor may transmit for 36 s per hour, i.e., about 873 frames per hour at SF7 but only about 31 at SF12 (Table IV). The nominal 2–5 frames per hour are well inside this limit, but a retransmission storm or a bug that floods reports is not.

Second, a gateway has a finite number of demodulation paths, and a frame that arrives when all are busy is lost even if it does not collide. Treating the paths as a loss system with $M = 8$ servers (a common figure for multi-channel gateways; this is generic and device-dependent) and Poisson arrivals with mean load $a = \Lambda T_{\text{air}}$ across all channels, the Erlang-B blocking probability for $N = 2{,}500$ and $r = 5$ h^{-1} is below 10^{-7} at SF9 and about 0.03 at SF12 ($a = 4.0$). Under these assumptions demodulator exhaustion is a secondary effect, and same-channel collisions dominate.

Third, downlink is much scarcer than uplink. Where a duty-cycle rule applies, a gateway is itself subject to it on its downlink sub-band, and every acknowledgement costs airtime comparable to an uplink frame. If the limit is 1% and an acknowledgement is a minimal 12-byte frame at the same SF as the uplink, the gateway can send about 0.24, 0.069, and 0.010 acknowledgements per second at SF7, SF9, and SF12. Against an uplink rate of $\Lambda = 1.39$ frames per second ($N = 2{,}500$, $r = 2$ h^{-1}), a gateway at SF9 could acknowledge only about 5% of uplinks and one at SF7 about 17% (acknowledgement airtimes computed with CRC off). Hence confirmed uplinks cannot be the default for every bay; they must be reserved for the few messages that need them, such as reservation confirmations.

5) Cost of confirmed uplinks: If every report is a confirmed uplink that is repeated until acknowledged (or until R retries have been used), the retransmissions add to the load. In our simplified model each attempt succeeds with probability $p = e^{-2G_{\text{eff}}}$, where G_{eff} is the *effective* load including repeats. Assuming that retries are independent and that acknowledgements are never lost, the expected number of transmissions per message is $(1 - (1 - p)^{R+1})/p$, so

$$G_{\text{eff}} = G\,\frac{1-(1-p)^{R+1}}{p}, \qquad p = e^{-2G_{\text{eff}}}, \tag{13}$$

a fixed-point equation solved by iteration. With unlimited retries ($R \to \infty$) it reduces to $G = G_{\text{eff}}e^{-2G_{\text{eff}}}$. The righthand side has maximum $1/(2e) \approx 0.184$ at $G_{\text{eff}} = 0.5$, so *no* solution exists for $G > 0.184$: the retry queue grows without bound and the

TABLE XXIII

channel collapses. Two cases illustrate the effect for $N = 2{,}500$, $r = 2$ h^{-1}.

- *SF7* ($G = 0.057$): $G_{\text{eff}} = 0.065$, a per-attempt success of 0.878, and 1.14 transmissions per message. With three retries the delivery ratio is 0.9998, against 0.892 for unconfirmed uplinks.
- *SF9* ($G = 0.201 > 0.184$): unlimited retries are unstable. With $R = 3$ the iteration converges to $G_{\text{eff}} = 0.41$, a per-attempt success of 0.44, and 2.05 transmissions per message; delivery improves from 0.67 to 0.90, but only by doubling the channel load, which in turn degrades every other sensor's first attempt.

Real LoRaWAN implementations add random back-off between retries and ack-timeout windows, and acknowledgement loss further increases repetitions, so these figures describe the first-order trend and not a protocol simulation.

6) How often must a bay report?: Freshness requirements fix a lower bound on reporting, which then competes with the capacity bound above. Let a bay change state as a Poisson process of rate μ (we set $\mu = \lambda_e$). The probability that the true state differs from a view that is Δ old, counting at least one change, is $1 - e^{-\mu\Delta}$. Requiring this to stay below x gives the maximum report gap

$$\Delta_x = -\frac{\ln(1-x)}{\mu}. \tag{14}$$

MAXIMUM REPORT GAP Δ_x (MINUTES) SO THAT THE PROBABILITY OF A CHANGE SINCE THE LAST REPORT STAYS BELOW x (EQ. (14))

μ (h^{-1})	$x = 1\%$	$x = 5\%$	$x = 10\%$
0.5	1.21	6.16	12.64
1	0.60	3.08	6.32
2	0.30	1.54	3.16

TABLE XXIV

ESTIMATED STALE-STATE FRACTION P_{stale} FROM LOST REPORTS (EQ. (15), $\mu = 1$ H^{-1}, ILLUSTRATIVE)

N	T_h	SF7	SF9	SF12
150	60 min	0.003	0.012	0.088
150	15 min	0.002	0.007	0.048
2,500	60 min	0.054	0.165	0.480
2,500	15 min	0.031	0.079	0.125

Table XXIII shows that a periodic scheme would need reports every few minutes: $x = 5\%$ at $\mu = 1$ h^{-1} requires $\Delta = 3.1$ min. For $N = 2{,}500$ this means 13.5 frames per second, a load of $G = 0.56$ and $p_s = 0.33$ even at SF7. Heartbeatonly reporting is therefore not viable at lot scale, and eventdriven reports are the right design. (The "at least one change" criterion is conservative, since two changes can cancel; the exact probability that the observed state is wrong, i.e., that an *odd* number of changes occurred, is $\frac{1}{2}(1 - e^{-2\mu\Delta})$.)

With event-driven reporting the stale-state probability comes from *lost* reports. If a change report is lost with probability $1 - p_s$ and the view is repaired by the next heartbeat, whose residual time is uniform on $[0, T_h]$ with mean $T_h/2$, then the long-run fraction of time a bay is stale is approximately

$$P_{\text{stale}} \approx \mu\,(1 - p_s)\,\frac{T_h}{2}. \tag{15}$$

valid when the product is small (we ignore repair by later events, which makes the estimate conservative). Table XXIV applies Eq. (15) with $\mu = 1$ h^{-1}. The trade-off is non-obvious: shorter heartbeats raise the load but, because they shorten the repair time, can lower the stale fraction (SF9, 2,500 bays: 0.165 at $T_h = 60$ min versus 0.079 at 15 min) until collisions dominate. At the 150-bay scale all entries are below 0.09, which again shows why a small lot cannot expose the problem.

7) Fog node load: service time is not latency: The reference paper states that fog nodes process incoming data every 10 ms [2]. We read this as a mean service time $s = 0.01$ s per message, an assumption, since the text could also describe a polling period. Model the node as an M/M/1 queue with arrival rate $\lambda = N\,r$ messages per second (taking offered load, i.e., ignoring losses, which is conservative) and service rate $1/s$. The utilization is $\rho = \lambda s$, the mean wait in queue is $W_q = \rho s/(1-\rho)$, the mean time in system is $W = s/(1-\rho)$, and, because the sojourn time is exponential with rate $1/s - \lambda$, its 99th percentile is $W_{99} = \ln(100)s/(1-\rho)$. Table XXV gives results for several arrival rates and service times.

TABLE XXV

M/M/1 MODEL OF THE FOG NODE (SERVICE TIME s ASSUMED; TIMES IN MS)

N	r (h^{-1})	λ (s^{-1})	s	ρ	W	W_{99}
150	2	0.083	10	0.001	10.0	46
2,500	2	1.39	10	0.014	10.1	47
2,500	5	3.47	10	0.035	10.4	48
2,500	5	3.47	100	0.347	153	706
2,500	5	3.47	250	0.868	1,895	8,726

Three lessons follow. First, with $s = 10$ ms the node is nearly idle: saturation ($\rho = 1$) would need 100 messages per second, which equals 180,000 sensors at 2 reports per hour. Under the assumed model the radio channel (Table XXII) saturates long before the fog CPU does. Second, the 10 ms figure is a property of the processing step; if the real per-

message path includes a database write, a TLS session, or a routing recomputation, s is a different number, and it must be

TABLE XXIV

measured as a distribution. At s = 250 ms and r = 5 h^{-1} the same node runs at ρ = 0.87 with a 99th-percentile delay of 8.7 s. Third, mean-based models hide bursts. If a power cut or an event end makes B sensors report within one second, the last message waits about $B\,s$: 5 s for B = 500 at s = 10 ms, and 50 s at s = 100 ms. A 10 ms service time is therefore a throughput statement; a latency guarantee needs a stated percentile, a stated arrival process, and a measured servicetime distribution. Real service times are not exponential, and an M/D/1 model with the same mean roughly halves W_q; an M/M/c model applies if the node processes messages on several cores.

8) Zoning reduces per-gateway load: Split the lot into Z zones, each with its own gateway and sensors that reach only that gateway. Under a uniform split each zone carries N/Z sensors, so

$$G_z = \frac{G}{Z}, \qquad p_s^{(Z)} = e^{-2G/Z} = \left(p_s^{(1)}\right)^{1/Z}. \tag{16}$$

For N = 2,500, SF9, r = 2 h^{-1} ($p^{(1)}{}_s$ = 0.670), Table XXVI shows Z = 4 already exceeds 0.90. The zones needed for $p_s \geq 0.9$ are $\lceil G/0.0527 \rceil$ with the single-zone G of Table XXII, i.e., 2 zones at SF7, 4 at SF9, and 31 at SF12. Zoning is thus most effective when combined with a lower SF, which requires gateways close enough to sensors; it is not a substitute for SF assignment. The same arithmetic holds for adding channels, which divide G by the number of channels under uniform channel selection. Two caveats apply. Zoning divides the load only if zones are radio-isolated; where coverage overlaps, a frame occupies the air at every gateway that can hear it, and sensors at a zone boundary load both neighbors. Uneven bay density (a block near the entrance) breaks the uniform split, so the busiest zone defines p_s.

9) What the prototype and the 150-space simulation could not reveal: A small prototype can establish that the sensing chain works: sensors read bays, frames reach the Raspberry Pi, and the Pi calls the routing logic. It can also give a rough

TABLE XXVI

(Z) $^{-1}$ Z ZONES

SUCCESS PROBABILITY p_s FOR $N = 2{,}500$, $r = 2$ H WITH (EQ. (16))

Z	SF7	SF9	SF12
1	0.892	0.670	0.040
2	0.944	0.818	0.201
4	0.972	0.905	0.448
8	0.986	0.951	0.670

TABLE XXVII

MEASUREMENTS MISSING FROM THE REFERENCE PAPER

Quantity	How to obtain it	Claim it would test
Packet delivery ratio vs. N, SF, T_h	Gateway logs; frame-counter gaps	p_s model (Eq. (12))
Time on air actually used	Gateway metadata per frame	Frame size, SF assignment
Concurrent receptions and drops	Gateway statistics under load	Demodulator limit
Downlink ACK success and duty use	Downlink counters	Feasibility of confirmed uplinks
Service-time distribution (P50, P95, P99)	Per-message timestamps at the fog node	"10 ms" claim; ρ, W_{99}
Fog CPU and memory vs. message rate	Replay of recorded or synthetic traffic	Capacity of one node
End-to-end latency percentiles	Synchronized clocks, sensor to decision	Latency, not just throughput
Stale-state fraction	Compare server view with manual audit	Eq. (15)
Zone boundary behavior and handover time	Move sensors across zone edges; fail one node	Zoning and failover
Battery life vs. reporting interval	Current logging of a sensor	Heartbeat choice

single-link range and a first estimate of the per-message processing time. It cannot reveal the quantities that govern scale. With the simulated lot of 150 sensors or fewer, Table XXII predicts success above 0.94 at SF9 for either heartbeat setting; collisions would be present but almost invisible, queues would be empty ($\rho < 0.01$), and the stale fraction would be negligible. The prototype would give no information on how p_s and P_{stale} degrade with density, how the fog node behaves in bursts, or how downlink limits constrain acknowledgements. Table XXVII lists the measurements that would close this gap; they feed the evaluation plan of Section VI.

I. Deep Dive: From Pixels to Congestion

Section IV-B observed that the reference routing consumes a lane congestion figure that its simulation set by hand between 0.1 and 0.9. This subsection asks what a camera pipeline must deliver for that figure to be usable, what it costs on a Raspberry-Pi-class fog node, and how to evaluate it. As before,

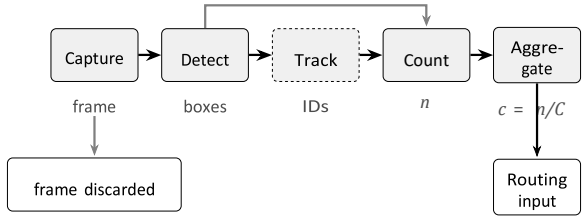


Fig. 6. Camera-to-congestion pipeline on the fog node. The dashed tracking stage is optional for density counting. Only the aggregated count leaves the node.

the analysis is analytical, and all compute and bitrate values are stated assumptions and not measurements.

1) What the routing needs, and how counting errors propagate: The routing needs the lane density $c = n/C$, where n is the number of vehicles present in a lane and C the lane capacity in vehicles [2]. If the vision system reports $\hat{n} = n+\varepsilon$, then

$$\hat{c} - c = \frac{\varepsilon}{C}, \qquad \frac{\hat{c} - c}{c} = \frac{\varepsilon}{n}. \tag{17}$$

The absolute error shrinks with larger capacity, the relative error grows when the lane is nearly empty. Let the edge cost be $w(c)$; the argument does not depend on the exact function used by the reference routing, so we use an illustrative linear form $w = w_0(1+\alpha c)$ with $w_0 = 30$ s and $\alpha = 2$. A first-order expansion gives $\delta w \approx w'(c)\varepsilon/C = w_0\alpha\varepsilon/C$. For $C = 20$, a true count $n = 8$ ($c = 0.40$, $w = 54$ s), and an error of $\varepsilon = +2$ vehicles, c becomes 0.50 and $w = 60$ s: a 25% relative error in c becomes a 6 s (11%) error in edge cost. For a convex cost function the error matters more at high density, exactly when routing decisions are most sensitive.

Errors on a route partly average out. For a route of m edges whose count errors are independent with standard deviation σ_n, the route-average density has error standard deviation $\sqrt{}$

$\sigma_n/(C \quad m)$: with $\sigma_n = 1.5$, $C = 20$, and $m = 5$ this is 0.034, against 0.075 for a single edge. Systematic errors such as a night-time undercount do not average out. The decisionlevel consequence is the probability of choosing the wrong one of two routes. If their true average densities differ by $\Delta c = 0.05$ and each is estimated with standard deviation $0\sqrt{.034}$ (independent), the difference has standard deviation

$2 \cdot 0.034 = 0.047$ and the probability that the order is reversed is $\Phi(-0.05/0.047) = 0.146$, where Φ is the standard normal distribution function. About one in seven decisions between such near-equal routes would be wrong, though the cost of a wrong choice is small by construction, because the routes are close.

2) The vision pipeline: Fig. 6 shows the stages from camera to routing input. Capture grabs a frame at rate f; detection finds vehicle bounding boxes; tracking (optional) links boxes across frames, which is needed to count flow through a line but not to count the vehicles present in a lane at one instant; counting assigns boxes to lane polygons; and aggregation smooths counts over time and converts them to c.

3) Detector families: Two-stage detectors such as Faster R-CNN [61] first propose regions and then classify them;

TABLE XXVIII
FAILURE MODES OF CAMERA-BASED COUNTING

Condition	Effect on detection	Mitigation
Night, low light	Missed dark vehicles; headlights counted as vehicles	Infrared or low-light camera; night-time training data
Rain	Reduced contrast, drops on lens, splashes	Weather-augmented training; lens hood
Glare, low sun	Saturated regions hide vehicles	Camera placement; HDR; polarizing filter
Occlusion	Undercount behind buses or in dense queues	Higher mounting; occupancy-ratio fallback
Camera shake	Lane polygons misalign, counts shift between lanes	Rigid mount; automatic polygon re-registration
Dirty lens, fog	Gradual loss of contrast and recall	Scheduled cleaning; image-quality monitor with alarm

they are typically the more accurate family, in particular for small objects, but are heavy for a Pi-class device. Singleshot detectors predict boxes and classes in one pass, as in SSD [62] and YOLO [53]; YOLOv3 [63] adds multiscale prediction that helps with small and distant vehicles. Lightweight backbones built on depthwise-separable convolutions, such as MobileNets [54], cut the multiply–accumulate count of a standard 3×3 convolution layer by roughly an order of magnitude, which is the usual route to running on an embedded CPU. Tracking by detection with Kalman prediction and assignment, as in SORT [56], adds little compute on top of the detector. Pretrained detectors are typically trained on general datasets such as COCO [64], which contains car, bus, truck, and motorcycle classes but not every locally common vehicle (for example, three-wheelers), so site-specific finetuning and a held-out local test set are needed.

Two forms of edge compression reduce the cost. Integer quantization [55] stores weights and activations in 8 bits, shrinking the model by $4\times$ relative to 32-bit floats and enabling integer arithmetic, and pruning with weight sharing and entropy coding [65] can compress further. The cost is a possible accuracy loss, which depends on the model and must be measured on the local test set, and some accuracy-sensitive classes such as small distant vehicles are affected first.

4) Failure modes: Table XXVIII lists conditions under which counting degrades. Each effect is qualitative; magnitudes have to be measured per site.

5) Counting versus occupancy ratio: Object counting gives integer vehicle numbers and supports $c = n/C$ directly, but degrades when vehicles cannot be separated in a dense queue. An occupancy-ratio alternative estimates the fraction of the lane region covered by vehicles (from segmentation or foreground masks) and maps it to c through a calibrated monotone function. It is cheaper, it degrades gracefully under occlusion, and it needs no object identity; its weaknesses are sensitivity to shadows and to the vehicle mix, since a bus covers more road than a motorcycle. A practical design runs counting as the primary estimator and uses the occupancy ratio as the fallback and as a consistency check, flagging disagreement for review.

TABLE XXIX
PRIVACY EXPOSURE OF VIDEO VERSUS COUNTS-ONLY OUTPUT

Aspect	Video retained or streamed	Counts only
Identifiers	Plates, faces, vehicle make visible	None in the output
Linkability	Vehicles trackable across cameras and time	Not linkable
Breach impact	High: sensitive, reusable footage	Low: aggregate numbers
Legal exposure	Retention, purpose, and access rules apply	Reduced; still verify locally
Auditability	Footage can resolve disputes	Cannot reconstruct events

6) Privacy: video versus counts-only: Table XXIX contrasts the two data products. Number plates and faces in video can be personal data under data-protection regimes such as the GDPR [66] and India's Digital Personal Data Protection Act, 2023 [67]; the precise legal status depends on jurisdiction and should be confirmed with the site operator. A counts-only design collects only an aggregate, and it keeps the identifying material off the network.

The last row is a real cost: without footage, a wrong count cannot be audited after the fact. A compromise is to keep short, access-controlled clips only during the labeled evaluation periods described below.

7) Compute and bandwidth budget: Let a fog node serve J cameras, each sampled at f frames per second, with a perframe inference time t_{inf}. The inference load is $Jf\,t_{\text{inf}}$ and must stay below a utilization cap $u_{\max}$ that leaves room for LoRa ingestion, routing, and logging. Hence

$$J_{\max} = \left\lfloor \frac{u_{\max}}{f\, t_{\inf}} \right\rfloor. \tag{18}$$

We assume, for illustration only, $t_{\inf} = 0.2$ s per frame for a compact quantized detector on a Raspberry-Pi-class CPU and $u_{\max} = 0.7$; a real value must be measured. Table XXX shows that 30 fps video cannot be processed even for one camera ($u = 6$), and neither can 5 fps ($u = 1.0$ for one camera). Frames must be sub-sampled. For density counting this is acceptable: c is a state, not a flow, and changes over seconds, so one frame every 1–5 s per camera is a defensible first design, to be validated against the staleness tolerance of routing. Equivalently, serving J cameras round-robin refreshes each camera every $J\, t_{\inf}/u_{\max}$ seconds: 2.3 s for $J = 8$ and 1.1 s for $J = 4$. Flow counting through a virtual line needs tracking and a higher frame rate, so it multiplies the budget.

Bandwidth shows an even larger gap. Assume a 720p H.264 stream of 2 Mb/s, i.e., 250 kB/s per camera; eight cameras produce 16 Mb/s. A counts-only message of 8 bytes per camera every 2 s needs 4 B/s per camera, or 32 B/s for eight cameras, a ratio of $62{,}500{:}1$. The nominal SF7 bit rate of Table IV is 5.5 kb/s, so even one such stream exceeds the LoRa rate by $366\times$ (before any duty-cycle limit, which at 1% would leave about 55 b/s). Video therefore needs a separate link (Wi-Fi or wired) to the fog node, whereas counts could, in principle, ride on LoRa. In the proposed architecture, however, the fog node sits next to the cameras, so only counts travel onward.

TABLE XXX
MAXIMUM CAMERAS PER FOG NODE (EQ. (18); ASSUMED $t_{\inf} = 0.2$ S, $u_{\max} = 0.7$)

f (frames/s)	1 camera needs u	$J_{\max}$	Refresh
30	6.0	0	—
10	2.0	0	—
5	1.0	0	—
1	0.2	3	1 s
0.5	0.1	7	2 s
0.2	0.04	17	5 s

TABLE XXXI
CONGESTION-STATE ACCURACY UNDER GAUSSIAN COUNT NOISE (ILLUSTRATIVE; $C = 20$, THRESHOLDS 0.3 AND 0.6)

σ_n (vehicles)	MAE (vehicles)	State accuracy
0.5	0.40	0.970
1.0	0.80	0.927
1.5	1.20	0.888
2.0	1.60	0.850
3.0	2.39	0.777

8) Evaluating counting accuracy: Let M labeled samples have ground-truth counts n_i and estimates $\hat{n}_i$. We define three metrics.

$$\text{MAE} = \frac{1}{M}\sum_{i=1}^{M} |\hat{n}_i - n_i|, \qquad \text{Bias} = \frac{1}{M}\sum_{i=1}^{M} (\hat{n}_i - n_i),$$

$$\text{RE} = \frac{\sum_i |\hat{n}_i - n_i|}{\sum_i n_i}. \tag{19}$$

The relative error RE is normalized by the total count to avoid division by zero on empty lanes. Because routing uses a congestion *level* and not the integer count, we add the *congestion-state accuracy*: map c to states (for illustration, low $c < 0.3$, medium $0.3 \le c < 0.6$, high $c \ge 0.6$) and report the share of samples whose estimated state equals the true state, together with the confusion matrix and the share that is off by two states. Table XXXI shows how count noise translates into state accuracy for $C = 20$ and true counts uniform on $0,\ldots,20$, assuming Gaussian count error of standard deviation σ_n before rounding (MAE $\approx 0.8\sigma_n$). The state boundaries make accuracy fall faster than MAE suggests, and the loss concentrates near thresholds.

A ground-truth protocol for the follow-up study is as follows.

1) *Sampling:* choose frames stratified by condition (day, night, rain, glare) and by true-density bin, so rare conditions are not drowned by easy daytime frames. To estimate MAE within $\pm h$ at 95% confidence from a sample with error standard deviation s_e, about $(1.96 s_e/h)^2$ frames are needed per stratum; for an assumed $s_e = 1.5$ and $h = 0.2$ this is 217 frames.
2) *Counting rule:* written in advance: a vehicle is counted if more than half of its body lies inside the lane polygon; parked or stopped vehicles are counted; partially visible vehicles at the frame edge follow a fixed rule.
3) *Double annotation:* two annotators label independently; disagreements are adjudicated by a third. The MAE between the two annotators is reported as the noise floor of the ground truth, and Cohen's kappa is reported for the states.
4) *Splits:* train, validation, and test sets are separated by camera and by day, not by random frame, because adjacent frames are nearly identical and random splits leak.
5) *Reporting:* MAE, bias, RE, and state accuracy per condition stratum with bootstrap confidence intervals, and the downstream effect on arrival time when measured c replaces the assumed values.
6) *Privacy:* clips used for labeling are stored accesscontrolled, deleted after annotation, and visible notices are posted at the site.

J. Deep Dive: Why a Fixed-Rate Markov Chain Falls Short

The summary in Section IV-C is qualitative. This subsection makes it quantitative with a small queueing model. Everything below is our own analytical construction: the numbers illustrate how a fixed-rate chain behaves under stated assumptions and are not evidence about the accuracy of the reference system, which reports none [2].

1) Occupancy as a queue: Let cars arrive as a Poisson process of rate λ and stay for independent exponential times with rate μ per car (mean stay $1/\mu$). If the lot is uncapped, the number of occupied bays $N(t)$ is an M/M/∞ system, and in steady state N is Poisson with mean $\rho = \lambda/\mu$, the offered load in Erlangs [60]. With K bays and arrivals that find the lot full being lost, the system is M/M/K/K, a birth–death process with birth rate λ and death rate $n\mu$ in state n. Its stationary distribution is a truncated Poisson,

$$\pi_n = \frac{\rho^n/n!}{\sum_{j=0}^{K} \rho^j/j!}, \qquad n = 0, \ldots, K, \tag{20}$$

and the blocking probability is the Erlang B formula $B(\rho,K) = \pi_K$. Evaluating the factorials directly overflows for $K = 150$, so we use the stable recursion $B(\rho,0) = 1$, $B(\rho,k) = \rho B(\rho,k-1)/\big(k + \rho B(\rho,k-1)\big)$. The mean occupancy is the carried load $\rho(1 - B)$.

Table XXXII evaluates these quantities for $K = 150$ at four offered loads. Blocking is negligible at 100 Erlangs, rises to about 0.8% at 130, to 6.2% at 150, and to 18.8% at 180, while mean occupancy saturates near 146 bays. The last column shows why the uncapped model is a poor guide near capacity: at $\rho = 150$ the Poisson model gives $P(N > K) \approx 0.48$, an overflow that a real lot converts into rejected arrivals.

The reference experiments use 150 spaces with 10 arrivals and 5 departures per minute [2]. These are rates of the whole

TABLE XXXII

ERLANG LOSS EXAMPLES FOR $K = 150$ BAYS (COMPUTED FROM THE STATED FORMULAS)

ρ	$B(\rho,K)$	Mean occ.	Util.	M/M/∞ $P(N>K)$
100	6.5×10^{-7}	100.0	0.667	1.2×10^{-6}
130	0.0078	129.0	0.860	0.039
150	0.0624	140.6	0.938	0.478
180	0.1881	146.1	0.974	0.988

stream, not stay-time parameters, and a per-car μ cannot be recovered from them alone. One reading, which is ours and is not stated in the reference, treats 10 per minute as λ and 5 per minute as the aggregate departure rate observed when about half the lot, $n = 75$ cars, is occupied. Then $\mu = 5/75 = 1/15$ per minute (mean stay 15 min) and $\rho = \lambda/\mu = 150$, which equals K and gives the 6.2% blocking of Table XXXII. More generally, under this reading $\rho = 2n$ for whatever occupancy n the 5 per minute is attributed to, so the ratio of arrivals to departures fixes ρ only once that occupancy is chosen. A second reading treats the 5 per minute as a constant departure stream independent of occupancy. There is then no stationary regime: the lot gains a net 5 cars per minute and fills in 30 min from empty, or 27 min from 135 free bays. Because the two readings imply qualitatively different dynamics, the reported curves cannot be interpreted without knowing which one the simulation implements.

2) From a birth–death process to a five-state chain: A binned chain can be derived rigorously from this model. Sampling the process every $\Delta = 5$ min with constant (λ,μ) gives a time-homogeneous transition matrix $P = e^{Q\Delta}$, where Q is the generator, and the chain [45] has stationary distribution π from (20). Grouping the $K+1$ states into five bins $\mathrm{B}_1,\ldots,\mathrm{B}_5$ yields a five-state matrix

$$\tilde{P}_{ij} = \frac{\sum_{n\in\mathcal{B}_i} \pi_n \sum_{m\in\mathcal{B}_j} P_{nm}}{\sum_{n\in\mathcal{B}_i} \pi_n}, \qquad (21)$$

whose stationary vector is the bin mass $\tilde{\pi}_i = \sum_{n\in B_i} \pi_n$. The lumped process is only approximately Markov, but an estimator that counts transitions between bins in a long stationary record would converge to exactly this matrix. Since the reference does not give bin edges, we take equal bins 0–29, 30–59, 60–89, 90–119, 120–150 bays. For $K = 150$, $\mu = 1/60$ per minute, and $\lambda = 1.278$ per minute (the dayaverage rate of the profile defined below, $\rho = 76.7$), we obtain

$$\tilde{P} \approx \begin{bmatrix} 0.173 & 0.827 & 0 & 0 & 0 \\ 0 & 0.623 & 0.377 & 0 & 0 \\ & 0.009 & 0.967 & 0.024 & 0 \\ & 0 & 0.293 & 0.707 & 0 \\ & 0 & 0 & 0.644 & 0.356 \end{bmatrix},$$

TABLE XXXIII

THOUGHT EXPERIMENT: 3 H-AHEAD ERRORS IN BAYS ($K = 150$, 60 MIN STAY; SURGE = TARGETS 16:30–19:30). OUR SIMULATION, NOT REFERENCE DATA

Forecaster	MAE	RMSE	MAE surge
Lumped chain	26.1	30.2	50.5
Unbinned chain	25.3	29.3	49.1
Persistence	22.5	26.0	22.4
Climatology (day mean)	26.1	30.2	51.3

with $\tilde{\pi} \approx (0,\ 0.021,\ 0.904,\ 0.075,\ 0)$. The matrix is a faithful summary of a lot whose demand never changes, and it is useless for a lot whose demand does.

Suppose now that λ depends on time, so the process is a non-homogeneous birth–death process. The exact state distribution still follows the Kolmogorov forward equation $\dot{p}(t) = p(t)Q(\lambda(t))$, which we integrate numerically with oneminute matrix exponentials. The homogeneous chain, however, can only apply its single matrix $\tilde{P}$ repeatedly. After h steps its forecast is $\tilde{P}^{\,h}$, which converges geometrically to $\tilde{\pi}$ at a rate set by the mean stay. For a 3 h horizon ($h = 36$ steps) and a 60 min mean stay, the horizon is three relaxation times, so the forecast has all but forgotten the current state: it is the long-run average occupancy, whatever the hour.

Thought experiment: We specify the arrival profile $\lambda(t) = 1.2[1+0.5\sin(2\pi(h-9)/24)]+1.5e^{-(h-17.5)^2/(2\cdot 0.5^2)}$ cars per minute, with h the hour of day: a sinusoid peaking at 15:00 plus a Gaussian evening surge centered at 17:30 with a standard deviation of 30 min. With $K = 150$ and a mean stay of 60 min, the offered load $\lambda(t)/\mu$ ranges from 36 to 191 Erlangs. We compute the exact periodic-regime expected occupancy $\mathrm{E}[N(t)]$ (it peaks at 144.9 bays at 17:50, shortly after the surge peak), and at each of the 288 fiveminute origins of a day we set the observed state to the true expected occupancy rounded to a whole car. We then compare, against the exact conditional expectation three hours later: the lumped chain, the unbinned homogeneous chain, persistence (repeat the current value), and climatology (the day-average occupancy). The computation is deterministic, so no random seed enters.

Table XXXIII gives the outcome. The lumped chain has a mean absolute error (MAE) of 26.1 bays (17% of K) and a root mean square error (RMSE) of 30.2; for targets between 16:30 and 19:30 its MAE is 50.5. Its worst error is 68 bays: at 14:50 it forecasts 76.7 bays for 17:50 when the expected occupancy is 144.9. The chain is statistically indistinguishable from climatology (skill -0.001), and it is worse than persistence (skill -0.35 by the formula given below). A sensitivity run over mean stays of 30, 60, 120, and 240 min, holding the load profile $\lambda(t)/\mu$ fixed, leaves the ordering unchanged: the lumped-chain MAE falls from 26.4 to 18.9 as memory lengthens, but persistence is better in every case except for the unbinned chain at 240 min. The chain is not wrong about the average; it is silent about the hour.

Fig. 7 shows the same experiment as a curve. Targets are plotted at the time they refer to; the chain and persistence forecasts were issued three hours earlier. The chain is a flat line at 76.7 bays. Persistence simply copies the curve three hours late, so it also misses the surge by three hours, though it avoids the large systematic bias of the chain during the afternoon build-up. A forecaster that knows $\lambda(t)$, or learns

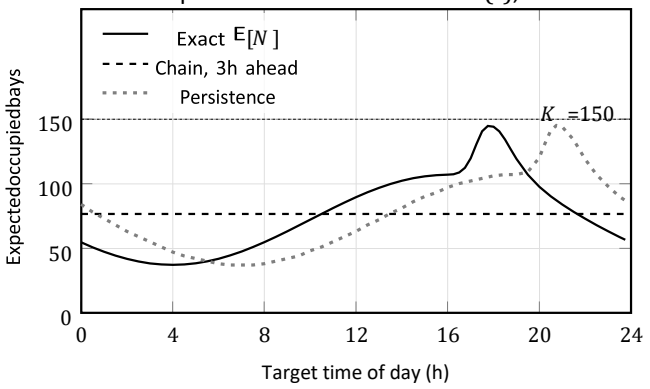


Fig. 7. Thought experiment: exact expected occupancy under a sinusoidalplus-surge arrival profile, and 3h-ahead forecasts from a time-homogeneous chain and from persistence. Our illustrative simulation; not data from the reference system.

the hour-of-day pattern from data, can follow the true curve.

The lesson is not that the reference model must perform this badly; its time-period-specific matrices, if estimated well, could soften the problem, and the paper does not say how the five-minute matrix is extended to three hours. The lesson is that a single fixed-rate matrix cannot carry information about the hour, and that its multi-step forecasts converge to a constant on the scale of a few mean stays. Any claim that the model supports three-hour-ahead reservation decisions must be tested against exactly these two baselines.

3) Memorylessness, seasonality, and autocorrelation: Occupancy has two kinds of structure that a first-order chain misses. The first is short-range memory. The sample autocorrelation function (ACF) of a series x_t at lag k is $r_k = \sum_t (x_t - \bar{x})(x_{t+k} - \bar{x}) / \sum_t (x_t - \bar{x})^2$, and the partial autocorrelation function (PACF) at lag k is the correlation between x_t and x_{t+k} after removing the linear effect of the intermediate values. For an M/M/∞ occupancy process, a standard result is that the autocorrelation at lag τ is $e^{-\mu\tau}$: a smooth exponential decay whose time constant is the mean stay. The second structure is seasonality, which shows up as ACF peaks at 24 h and 168 h. Daily and weekly cycles, public holidays, and (in the paper's own case study) the Hijri calendar are properties of $\lambda(t)$, not of the current state, and can enter a Markov chain only by multiplying the number of matrices.

The persistence baseline predicts $\hat{x}_{t+h} = x_t$. For a stationary series with variance σ^2 and autocorrelation $r(h)$, its mean squared error is $2\sigma^2(1 - r(h))$, whereas the long-run mean has error σ^2. Persistence therefore beats the mean only when $r(h) > 1/2$. With $r(h) = e^{-h/60}$ for a 60 min stay, the breakeven horizon is $60\ln 2 \approx 41.6$ min. The lag-h autocorrelations at 15 min, 1 h, and 3 h are 0.779, 0.368, and 0.050. A serious forecaster must beat persistence at short horizons and seasonal persistence (repeat the value one week earlier) at long ones.

TABLE XXXIV

The *skill score*

$$\mathrm{SS} = 1 - \frac{\mathrm{MSE}_{\mathrm{model}}}{\mathrm{MSE}_{\mathrm{baseline}}} \qquad (22)$$

is positive when the model improves on the baseline and negative when it is worse. For persistence against climatology under the stationary assumption, $\mathrm{SS} = 2r(h) - 1$, which equals 0.558, −0.264, and −0.900 at the same three horizons.

FORECAST METRICS AND THEIR USE FOR A RESERVATION SYSTEM

Metric	Appropriate use	Caution
MAE	Typical error in bays; easy for operators	Treats a 20-bay miss near capacity like one at mid-load
RMSE	Penalizes rare large misses	Dominated by surges; not robust to outliers
MAPE	Avoid for occupancy	Undefined at zero; asymmetric
MASE	Comparison with a naive method across lots	Needs the naive method to be stated
Pinball loss	Scoring a chosen quantile, e.g. the acceptance threshold	One loss per quantile level
Coverage and width	Checking intervals used for acceptance	Coverage alone can be met by wide intervals
CRPS, interval score	Comparing probabilistic forecasters	Needs a full distribution or interval
Falseaccept rate	Share of accepted bookings that find no space	Depends on the policy, not only the forecast

4) Forecast error metrics and their cautions: Let y_t be the realized value and $\hat{y}_t$ the forecast with error $e_t = y_t - \hat{y}_t$ over T evaluation points. MAE is $T^{-1} \sum |e_t|$, in bays, and RMSE is $\sqrt{T^{-1} \sum e_t^2}$, which penalizes large misses more. The mean absolute percentage error (MAPE) divides each error by y_t and is undefined for an empty lot and unstable near it: an error of 2 bays on an actual of 2 counts as 100%, while an error of 10 on an actual of 100 counts as 10%. The scaled errors proposed by Hyndman and Koehler [68] avoid this; the mean absolute scaled error divides MAE by the in-sample MAE of a naive forecast, so values below 1 beat the naive method. Point errors say nothing about uncertainty, which is what a reservation decision needs. The pinball (quantile) loss of the τ-quantile forecast q is $\ell_\tau(y - q)$ with $\ell_\tau(u) = u(\tau - \mathbf{1}\{u < 0\})$ [47]. A $(1 - \alpha)$ prediction interval is judged jointly by its empirical coverage, which should be near $1 - \alpha$, and its sharpness, the average width, since the interval $[0, K]$ covers everything and

informs nothing [69]. Strictly proper scoring rules, such as the continuous ranked probability score and the interval score, reward a forecaster only for stating its true belief, so they cannot be gamed by hedging [70]. Table XXXIV summarizes when each is appropriate.

5) A decision-theoretic view of reservations: A reservation is a decision, so forecast quality should be judged by decision cost. Let c_{FP} be the cost of accepting a booking that finds no space (a false positive: refund, diversion, lost goodwill) and c_{FN} the cost of rejecting a booking that would have found space (a false negative: lost margin and an unhappy customer). If p is the forecast probability that a bay is free at arrival, accepting costs $(1 - p)c_{FP}$ in expectation and rejecting costs pc_{FN}. The expected cost is smaller for acceptance exactly when

$$p \geq p^{\star} = \frac{c_{FP}}{c_{FP} + c_{FN}}. \tag{23}$$

The threshold depends on costs, not on the forecaster. For example, with illustrative costs $c_{FP} = 5$ and $c_{FN} = 1$ (in arbitrary units), $p^{\star} = 5/6 \approx 0.833$. If the three-hour forecast for a 150-bay lot is, under an uncapped Poisson approximation, Poisson with mean $\rho\hat{}$, then $p = P(N \leq 149)$. For $\rho\hat{} = 130$, $p = 0.954$: acceptance costs $0.046 \times 5 = 0.23$ and rejection costs 0.954, so we accept. For $\rho\hat{} = 140$, $p = 0.790$: acceptance costs $0.210 \times 5 = 1.05$ and rejection 0.790, so we reject. The cut-off is $\rho\hat{} \approx 138.1$, even though the expected occupancy is still 12 bays below capacity; with equal costs the cut-off moves to $\rho\hat{} \approx 149.7$. This is a quantile statement: accepting is optimal when the $p^{\star}$-quantile of forecast occupancy is at most $K - 1$, so the pinball loss at $\tau = p^{\star}$ is the matching training objective. Conformal prediction offers a distributionfree way to calibrate such intervals from recent residuals [48]; its coverage guarantee assumes exchangeability, which time series violate, so in practice calibration windows must roll and coverage must be monitored.

6) Validation of forecasters and data needs: Forecasts must be evaluated in time order. A random k-fold split lets the model train on the future of the series it is tested on, and consecutive five-minute targets are so strongly correlated that a random test point has near-duplicates in the training set. Chronological splits, with a gap at least as long as the forecast horizon between the end of training and the start of testing, and rolling-origin evaluation, in which the model is refit or updated at successive origins and scored on what follows each [71], are the minimum standard. Leakage also arises when normalization statistics are fitted on the full series, when features use information that is only known later (for example the final event attendance), and when hyperparameters are tuned on the test period. Eight weeks of five-minute data supply 16,128 origins, but the effective sample size is far smaller because of the autocorrelation above.

TABLE XXXV

Gradient-boosted trees [72], [73] and long short-term memory networks [74] are natural candidates. Both need explicit time features: lagged occupancy, rolling means, entry and exit counts over the last 15 min, sine and cosine of the hour and the weekday, calendar and event flags, and the lead time of the forecast. Table XXXV lists the data such a study requires; the history lengths are our recommendations, not established requirements.

K. Deep Dive: Validity of Simulation-Based Evidence

Section IV-D identifies simulation as the main source of the reference system's performance claims. Simulation can be strong evidence, but only when the model, its inputs, and its output analysis meet standards that the reference paper, as described in Section IV, does not report meeting. All numbers below that look like results are made up to illustrate a calculation.

1) Verification, validation, and credibility: Sargent [59] distinguishes *verification* (is the model implemented as intended?), *validation* (is the model an adequate representation of the real system for its intended purpose?), and, in Law's treatment [75], *credibility* (will decision makers accept the

DATA REQUIREMENTS FOR A FORECASTING STUDY (OUR RECOMMENDATIONS)

Data	Resolution / history	Purpose
Occupied count per lot or zone	1–5 min; ≥8 weeks, ideally a year	Target series; weekly cycle seen at least eight times
Entry and exit events	Per vehicle, timestamped	Arrival and departure rates; stay-time distribution
Reservation log	Per request	Acceptance, no-shows, overstays, outcome at arrival
Calendar and events	Daily, with start/end times	Holidays, Hijri dates, large gatherings
Sensor health flags	Per report	Separating missing data from zeros
Manual audits	Several hours across days	Ground truth for sensor error

TABLE XXXVI

SIMULATION VALIDITY THREATS MAPPED TO THE REFERENCE SETUP AS DESCRIBED IN THIS PAPER

Threat	Reference setup	Evidence needed
Input validity	Initial availability, congestion (0.1–0.9), and rates chosen by the authors	Inputs fitted to logged data, or sweeps over stated ranges
Structural validity	One lot, 150 spaces, reproduced from a larger real lot	Comparison of simulated and observed occupancy and delay
Baseline strength	Blind search only	Stronger guidance policies (below)

TABLE XXXVI

Statistical validity	No repeated runs or intervals shown	Replications, seeds, confidence intervals
Initial-state bias	Runs begin from stated availability	Warm-up analysis or a defensible start state
Reporting	A 16–46% range	Conditions under which each end occurs

results?). A simulation can be perfectly verified and still invalid, as when it correctly implements an unrealistic driver. Table XXXVI maps common threats to the reference setup as described in this paper; the third column states what a reader would need to see.

2) Statistics of simulation output: A single run of a stochastic simulation is one sample path, not an estimate. The standard remedy is n independent replications with different random-number streams. If $X_1,\ldots,X_n$ are the replication means of a metric with sample mean $\bar{X}$ and sample standard deviation $\sqrt{s}$, a $100(1-\alpha)\%$ confidence interval is

$$\bar{X} \pm t_{n-1,1-\alpha/2}\, s/\sqrt{n}$$

[75]. To reach a target half-width h, we need $n \geq (t_{n-1,1-\alpha/2}\, s/h)^2$; because t depends on n, the bound is solved iteratively from a pilot estimate of s.

Table XXXVII lists ten made-up replication means of the mean arrival time under blind search. They give $\bar{X} = 9.74$ min, $s = 0.698$ min, standard error 0.221, $t_{9,0.975} = 2.262$, and a 95% interval of 9.74 ± 0.50, that is $[9.24, 10.24]$ min, about $\pm 5.1\%$ of the mean. A half-width of

TABLE XXXVII
ILLUSTRATIVE (MADE-UP) REPLICATION MEANS OF ARRIVAL TIME UNDER BLIND SEARCH, IN MINUTES

1	2	3	4	5	6	7	8	9	10
9.4	10.1	8.7	10.8	9.9	9.2	10.5	8.9	9.6	10.3

h = 0.25 min (15 s) requires $n \geq (2.037 \times 0.698/0.25)^2 \approx 32.3$ (using $t_{32,0.975} = 2.037$), so n = 33 runs; the normal approximation with 1.96 gives 30, a slight underestimate; h = 0.1 min needs 190 runs. At the 30 seeds planned in Section VI, the half-width would be 0.261 min with the same s. A pilot s is itself uncertain, so a safety margin is wise.

When two policies are compared, *common random numbers* drive both with the same arrival stream, so that the paired difference $D_i = X_i - Y_i$ has variance $\sigma_X^2 + \sigma_Y^2 - 2\rho_{XY}\sigma_X\sigma_Y$ [75]. With $\sigma_X = \sigma_Y = 1.2$ and correlation $\rho_{XY} = 0.7$ the variance falls from $2.88\sqrt{}$ to 0.864, so the

standard deviation of the difference is $\overline{0.3} \approx 0.55$ of its independent value and the replications needed for a given halfwidth fall by a factor of 0.3. A study that does not say whether it used common random numbers leaves this unclear.

Terminating simulations have a natural end, such as a threehour event surge, and are analyzed by independent replications of the whole horizon. *Non-terminating* (steady-state) questions require discarding the initial transient, for instance with a graphical procedure such as Welch's, described by Law [75]. Initial conditions matter here. Under our interpretation of the 150-space parameters, with $\mu = 1/15$ per minute, an uncapped lot started empty reaches 95% of its limiting mean after $\ln 20/\mu \approx 45$ min; with a 60 min stay the figure is three hours. A run shorter than this is mostly measuring its starting state, and a note that availability starts at up to 90% is a statement about the answer as much as about the setup.

A range is not a guarantee: A claimed improvement of 16–46% is a pair of numbers, and conditions can generate any such pair. We use a toy model of ours, not of the reference. A blind driver probes bays in random order until one is free, so with F free bays among K the expected number of probes is $(K + 1)/(F + 1)$. Take K = 150, a 0.15 min cost per probe, 2.0 min of driving, and a guided driver who needs 1.5 probeequivalents (stale or incomplete information). Table XXXVIII shows the relative improvement $1 - T_{\text{guided}}/T_{\text{blind}}$ as the number of free bays at search time varies: 46% corresponds to about 10 free bays (6.7% of the lot) and 16% to about 34 (23%). A single algorithm yields both ends of the range depending on the regime. Without the regime the interval says only that gains exist somewhere; it cannot be used to plan a lot where occupancy is outside the tested range, and it is not a bound.

3) The baseline ladder: Guidance policies form a ladder. At the bottom is blind search. Next is *last-known-free-bay* guidance, which sends the driver to a bay that was free when last reported and suffers from stale data. Then comes *nearest-free-bay* guidance using current availability, followed by *central assignment or reservation*, which allocates a specific bay to each arrival and avoids two drivers heading for the same bay. The reference compares only with the bottom rung. Relative improvement against a baseline b is $1 - T_{\text{new}}/T_b$, so a weaker baseline raises it mechanically. Table XXXIX uses made-up mean arrival times: a proposed method at 5.8 min appears 42.0% better than blind search but only 17.1% better than nearest-free-bay guidance and 9.4% better than central assignment. In this example, simple guidance policies already deliver most of the gain over blind search (30% from nearestfree-bay of the 42 points), so the credit for the method's own contribution is far smaller than the headline suggests.

TABLE XXXVIII
TOY MODEL: RELATIVE IMPROVEMENT VERSUS FREE BAYS AT SEARCH TIME (K = 150)

Free bays F	T_{blind} (min)	T_{guided} (min)	Improvement
10	4.06	2.23	45.2%
20	3.08	2.23	27.7%
34	2.65	2.23	15.9%
60	2.37	2.23	6.2%
90	2.25	2.23	1.1%

TABLE XXXIX
ILLUSTRATIVE (MADE-UP) BASELINE LADDER: GAIN OF A PROPOSED METHOD AT 5.8 MIN

Baseline	Arrival (min)	Gain vs. blind	Proposed gain
Blind search	10.0	0%	42.0%
Last-known free bay	8.2	18.0%	29.3%
Nearest free bay	7.0	30.0%	17.1%
Central assignment	6.4	36.0%	9.4%

4) Sensitivity analysis: Inputs chosen by the authors need a sensitivity analysis. A one-at-a-time (OAT) design moves one factor from a base case and holds the others fixed; it is cheap but blind to interactions. A full factorial design with k factors at two levels requires 2^k runs, and fractional designs reduce the cost at the price of confounding high-order interactions [76]. For four factors, a full 2^4 design with 30 seeds per point needs $16 \times 30 = 480$ runs; a half fraction 2^{4-1} needs 240.

To show how OAT can mislead, we extend the toy model above with a drain term: the mean free bays seen by drivers over the first W = 5 min of a surge is $\bar{F} = \max\{aK - (\lambda - d)W/2, 1\}$, with initial free fraction a, arrivals λ, departures d = 5 per minute, and driving time $2.0(1 + c)$ min for congestion c. Table XL lists the improvement at low and high levels with the others at base (a = 0.3, λ = 10, c = 0.5, K = 150; base improvement 12.3%). The ordering of the swings (initial availability, lot size, arrival rate, congestion) is what a tornado diagram would

display. Interaction is large: at a = 0.15, raising λ from 6 to 14 per minute raises the improvement from 20.9% to 77.5%, whereas at a = 0.6 the same change moves it only from 1.0% to 3.2%. The 77.5% arises because the lot is essentially full and the clip at F^{-} = 1 applies, which is itself a warning about extrapolating a toy. These numbers belong to this toy model only. They show that

TABLE XL
TOY TORNADO TABLE: IMPROVEMENT (%) AT LOW AND HIGH INPUT LEVELS

Input	Low / high	At low	At high	Swing
Free fraction a	0.15 / 0.60	36.3	1.9	34.3
Lot size K (bays)	75 / 300	20.1	9.8	10.3
Arrivals λ (/min)	6 / 14	8.4	18.6	10.2
Congestion c	0.1 / 0.9	15.7	10.1	5.6

TABLE XLI REPRODUCIBILITY CHECKLIST FOR SIMULATION STUDIES

Item	What to record
Random seeds	Seed per replication; separate streams for arrivals, stay times, and sensors
Software versions	Simulator, libraries, language runtime, operating system
Configuration files	All parameters, including defaults; network and demand files
Scripts	Code for runs, post-processing, tables, and figures; one command to rerun
Raw outputs	Per-replication results, not only means
Input provenance	Source and fitting method of every input; which were assumed
Run length and warm-up	Horizon, discarded transient, initial state
Statistics	Number of replications, interval method, use of common random numbers

factor effects are not additive.

5) Reproducibility: A simulation result can be checked only if the experiment can be rerun. Table XLI lists what a report should supply; the evaluation plan of Section VI and the reporting checklist in the appendices apply it.

L. Deep Dive: Threat Model and Privacy Analysis

The brief treatment of security in the reference paper (Section IV-E) leaves open what exactly is to be protected, from whom, and at what cost. This deep dive supplies the missing pieces in analytical form. It defines assets, actors, and trust boundaries; derives simple quantitative statements about forgery, replay, and data poisoning; examines privacy against a legal and design background; and prices the protection in airtime and battery life. All numbers in this subsection follow from stated assumptions and are illustrative of method; none was measured on the reference system.

1) Assets, actors, and trust boundaries: We consider a generic three-tier LoRa parking system with the structure described in Section III: bay sensors reach a gateway and fog node over LoRa, the fog node forwards summaries to a cloud back end, and drivers interact through a mobile application and a third-party payment service. The assets worth protecting fall into five groups.

- *Integrity of occupancy information.* Every downstream decision (bay guidance, reservation acceptance, dynamic price) is a function of reported occupancy.
- *Availability of the service.* Radio coverage, fog processing, and the reservation interface must keep working at peak load.
- *Booking and financial records.* Reservations, subscriptions, bids, and payment tokens carry direct monetary value.
- *Personal data.* Number plates, RFID or QR credentials, account identifiers, and the arrival and departure times attached to them.
- *Secrets and code.* Device root keys, session keys, firmware images, and cloud credentials, because their loss undermines every other asset.

Five kinds of actors matter. An *outside radio attacker* is anyone within radio range of a sensor or gateway, equipped with inexpensive software-defined radio gear. A *network attacker* sits on the backhaul or on the public Internet. A *physical attacker* can touch a deployed sensor: bays are in public view, and a unit mounted in or on the pavement is available to anyone with a screwdriver. An *insider* is an operator employee or contractor holding legitimate credentials. A *malicious user* is a registered driver or a bot that abuses the booking interface. We exclude nation-state attackers and attacks on the cryptographic primitives themselves.

Fig. 8 is a data-flow diagram of the system with four trust boundaries. A trust boundary marks a place where data crosses between parties that do not share the same level of control, and it is where authentication, validation, and logging are required [77]. TB1 encloses the field devices, which are physically exposed and resource-constrained. TB2 is the operator's edge: the gateway, the fog node, and any junction camera. TB3 is the cloud. TB4 holds the driver's phone and the external payment provider, which the operator does not control. The three radio and network links that cross these boundaries (D1–D3) and the local camera feed (D4) are the data flows that the threat analysis below must cover.

2) Attacker model for LPWAN radio: Dolev and Yao [78] formalized the idea of an attacker who controls the network: it can read, drop, delay, replay, and inject messages, and can apply any operation a legitimate party could apply, but it cannot break the cryptography. For a LoRa deployment the same abstraction applies to the radio channel, with four

adaptations. First, the attacker must be physically near the target, so reach is bounded by link budget rather than by Internet connectivity. Second, the physical layer is itself attackable: a transmitter on the same channel can jam reception, which has no counterpart in the symbolic model. Third, devices are cheap and unattended, so the attacker can also be given a captured unit and with it that unit's keys. Fourth, the attacker's rate is bounded by airtime, a point that Section IV-L5 turns into numbers. We therefore grant the attacker the following capabilities.

C1 *Eavesdrop* any frame in range, including its length and timing.

C2 *Replay* recorded frames, immediately or later, on any gateway in range.

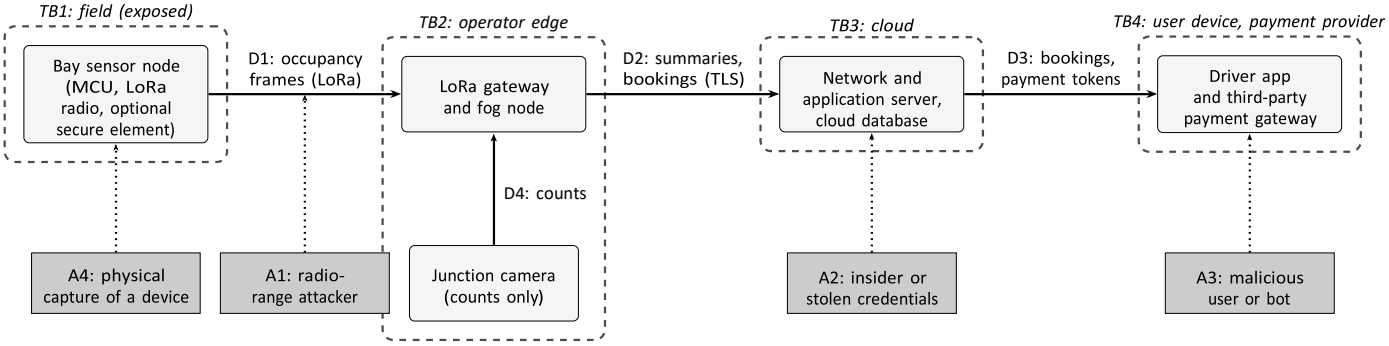


Fig. 8. Data-flow diagram of a three-tier LoRa parking system with four trust boundaries (dashed). Solid arrows are data flows D1–D4; dotted arrows show where the attacker classes A1–A4 act. The diagram is our generic model, not a reproduction of the reference design.

C3 *Inject* frames it has constructed, without possessing the key of the claimed sender.

C4 *Delay or selectively drop* frames, for example by acting as a relay or a rogue gateway.

C5 *Jam* a channel or spreading factor.

C6 *Extract keys* from one captured device, but not from the others.

C7 *Act as a registered user*, with the ability to submit requests at network speed.

3) STRIDE-style threat enumeration and risk ranking: STRIDE classifies threats as spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege, and is applied by walking the elements and flows of a data-flow diagram [77]. Table XLII lists eight threats that follow from Fig. 8, with the component affected and the concrete consequence for a parking operator or driver.

To rank these threats we follow the semi-quantitative style of NIST SP 800-30 [79], in which risk is judged from the likelihood that a threat event occurs and succeeds and from the magnitude of its impact. We use a three-level scale for each factor (1 low, 2 medium, 3 high), multiply them into a score from 1 to 9, and band the score as low (1–2), medium (3–4), or high (6–9). Table XLIII applies this to T1–T8. The scores are our illustrative assessment, made for a plain LoRaWAN deployment without the application-layer measures of Section V; they are meant to demonstrate the ranking procedure, and an operator should redo them with site-specific knowledge.

Two observations follow. First, the three highest-ranked threats (jamming, flooding, and key extraction) are exactly the ones that cryptography on the radio link does not address, which is why the application layer cannot be left to the radio standard alone. Second, the product score hides differences in kind: T6 has a modest likelihood but an impact that depends on a design decision, namely whether keys are unique per device, so a single design change (per-device keys) moves it from a high-impact to a contained event.

4) What LoRaWAN provides and what it leaves to the application: LoRaWAN offers a layered set of mechanisms, described in the specification [34] and analyzed in several surveys and formal studies [16], [57], [58]. We summarize them at a conceptual level and refer to the specification for details.

A device joins either by *over-the-air activation* (OTAA) or by *activation by personalization* (ABP). In OTAA the device sends a join request carrying identifiers and a nonce, authenticated with a root key shared with the join server; the join answer, protected under the same root key, carries the material from which both sides derive fresh *session keys*. In ABP, the device address and session keys are provisioned in advance and do not change at each start. Session keys serve two purposes: one authenticates the frame and the networklevel fields, the other encrypts the application payload so that it is readable only by the application server and not by the network operator. Every frame carries a *frame counter* that increases with each uplink, and the receiver rejects a frame whose counter is not larger than the last one accepted. A 4-byte *message integrity code* (MIC), computed with AESCMAC over the frame including the counter, protects against modification and, together with the counter check, against replay. Payload encryption uses AES-

128 in a counter-style mode keyed by the application session key. Version 1.1 refines this with separate root and session keys for network and application purposes and requires 32-bit frame counters [34]; in both versions only the 16 least significant bits of the counter travel in the frame, and the receiver reconstructs the upper bits.

The weaknesses reported in the literature are, in our reading, of five kinds. (i) ABP devices that restart must reset their counters, which forces the server either to accept counter resets, and with them replay, or to reject the device until it is re-provisioned. (ii) In the earlier specification versions, join requests could be replayed or the join process disrupted because of the way nonces were handled. (iii) In the 1.0.x versions a device may keep only a 16-bit counter, which then wraps around and creates a window in which old frames look new unless the device rejoins. (iv) The MIC is only 32 bits, so a forgery has a small but not negligible chance per attempt; Section IV-L5 quantifies this. (v) Metadata (frame length, timing, and frequency) is visible even when payloads are encrypted, and the standard does not protect against jamming or selective delay by a relaying gateway [16], [57]. Eldefrawy et al. analyze the join and key-derivation procedures with formal methods [58]; we refer to their paper for the properties they verify and the weaknesses they identify.

The application layer therefore has to add at least five things: (a) a unique root key per device, ideally held in a secure element [15]; (b) OTAA instead of ABP, or an ABP variant that preserves counter monotonicity across restarts; (c) a freshness check tied to the content (for example a sequence number or time bucket inside the encrypted payload) so that a latedelivered frame is detected even if the network layer accepts it; (d) plausibility checks on the content (Section IV-L6); and (e) TLS [80] or an equivalent on every hop beyond the gateway, because LoRaWAN stops at the network server.

5) Quantitative sketches: replay, forgery, and brute force:

TABLE XLII
STRIDE-STYLE THREATS MAPPED TO COMPONENTS AND PARKING CONSEQUENCES (OUR ANALYSIS)

ID	Threat	STRIDE	Component	Mechanism	Parking consequence
T1	Spoofed occupancy reports	S, T	Sensor, D1	Forged frames, or reports from a compromised device, state the wrong bay status	Phantom "full" lots lose revenue; phantom "free" bays send drivers to occupied spaces; bias enters the forecaster
T2	Replay of old frames	S, T	D1, gateway	A recorded "free" frame is retransmitted while the bay is occupied	Stale state is accepted as current if counters or timestamps are not checked
T3	Rogue or relaying gateway	S, T, I	D1, TB2	Frames are captured, delayed, or selectively dropped before they reach the real gateway	Late or missing updates; targeted starvation of a zone
T4	Radio jamming	D	D1	Noise or colliding transmissions on the uplink channels	Zone goes blind; stale occupancy drives guidance and pricing
T5	Reservation flooding	D	D2, D3, cloud	Many bookings made and left unpaid, or bids submitted in bulk	Capacity held without revenue; honest users refused service
T6	Key extraction from a captured device	I, E	Sensor, TB1	Keys are read from flash or memory of one unit	Forgery as that sensor; fleet-wide compromise if keys are shared
T7	Payment fraud	S, R	D3, TB4	Stolen tokens, charge-backs, or fake QR credentials	Revenue loss, disputes, liability for card data
T8	Location tracking and linkage	I	Cloud, D3, D1 meta-data	Plates, tokens, and timestamps are linked across visits and lots	Profiling of individuals; legal exposure of the operator

TABLE XLIII
ILLUSTRATIVE QUALITATIVE RISK SCORING (OUR ASSESSMENT; LIKELIHOOD × IMPACT)

ID	L	I	Score	Band	Rationale
T4	3	2	6	High	Jammers are cheap and need no keys
T5	3	2	6	High	Only an account is needed
T6	2	3	6	High	Needs physical access; impact grows with shared keys
T1	2	2	4	Medium	Needs a key or a weak join/session setup
T2	2	2	4	Medium	Easy to record; effective only if freshness is unchecked
T8	2	2	4	Medium	Impact rises sharply if plates are stored
T7	1	3	3	Medium	Lower if a third party handles cards
T3	1	2	2	Low	Requires sustained local presence

a) Replay acceptance.: Let a device have sent N frames with counters $0,\ldots,N-1$, all captured by an attacker, and let f be the highest counter the server has accepted. If the server enforces the rule "accept only a counter larger than f", then every captured frame has counter at most f and the probability that a replayed frame is accepted is exactly zero (apart from the wrap-around case below). If the server instead checks the MIC but not the counter, the probability is one for every captured frame. The intermediate, realistic fault is the counter reset of an ABP device: after a restart the server resets f to zero, the device then sends m new frames, and a captured frame with counter c is accepted if $c > m$. Taking a captured frame uniformly at random, the acceptance probability is

$(N-1-m)/N$; for example, $N = 1000$ captured frames and $m = 100$ new ones give $899/1000 = 0.899$.

The conclusion is conceptual rather than empirical: correct counter handling makes replay impossible by construction, whereas a single faulty reset rule makes most of the attacker's archive usable. The wrap-around case adds a time scale. A 16-bit counter wraps after $2^{16} = 65{,}536$ uplinks, which is $65{,}536/144 \approx 455$ days at 144 uplinks per day (one every ten minutes, our assumption) and only about 45.5 days at one uplink per minute.

b) Forgery against a 32-bit MIC.: An attacker who does not know the session key can only guess the MIC. Each guess succeeds with probability $p = 2^{-32} \approx 2.33 \times 10^{-10}$, independently of earlier guesses if the content is fixed. The number of attempts until the first success is geometric, with expectation $1/p = 2^{32} = 4{,}294{,}967{,}296$ and median $\ln 2 \cdot 2^{32} \approx 2.98 \times 10^{9}$. After 10^6 attempts the success probability is $1 - (1 - p)^{10^6} \approx 2.33 \times 10^{-4}$.

What matters in practice is how fast attempts can be made. Let T be the time on air of a forged frame. We compute T with the standard LoRa formula (Section IV-L8) for a 17byte physical payload, that is, 13 bytes of LoRaWAN framing and MIC plus a 4-byte application payload, on a 125 kHz channel. The result is $T = 51.5$ ms at SF7 and $T = 1.319$ s at SF12. An attacker transmitting continuously can then make at most $1/T$ attempts per second: 19.4 per second at SF7 and 0.758 per second at SF12. The expected time to 2^{32} attempts is 2558 days (7.0 years) at SF7 and $65{,}563$ days (179.5 years) at SF12. Under a 1% duty-cycle limit, which we assume for illustration as in EU868-style rules (regional rules differ, and an attacker need not obey them), the same figures grow by a factor of 100, to about 700 years and about 17,950 years. The probability that a single always-on SF7 transmitter forges *one* frame within a year is $1 - (1 - p)^{19.43 \times 3.156 \times 10^7} \approx 0.133$; at SF12 it is about 0.0056. Two conclusions follow. Online MIC forgery is slow but not impossible against a single frame at the fastest data rate, which is a reason to monitor the rate of MIC failures at the network server rather than to rely on the 32-bit tag alone. And parallel attackers scale the rate linearly, so the figures are per transmitter.

c) Key search.: The AES-128 session key has $2^{128} \approx 3.40 \times 10^{38}$ values, and an exhaustive search needs $2^{127} \approx 1.70 \times 10^{38}$ trials on average. Even if an adversary could test 10^{12} keys per second (a hypothetical figure chosen only to be generous), the expected time would be 1.7×10^{26} s, or about 5.4×10^{18} years. Brute force on the key is therefore not the realistic path: the realistic paths are key extraction from a device (T6), poor key management, and implementation faults such as the counter handling above.

6) Data-integrity attacks on forecasting: Spoofed or poisoned reports do not have to take a lot offline to be harmful; a small, persistent bias can tilt a decision rule. We formalize this in generic notation. Let the lot have N bays with true occupied fraction ρ, and let an attacker control a fraction φ of the bays so that their reports are shifted by $\delta \in [-1,1]$ in occupancy units (δ = +1 means every controlled bay is reported as occupied whatever its state). The fraction estimated from reports, truncated to $[0,1]$, is

$$\hat{\rho} = \min\big(1, \max(0,\ \rho + \varphi\delta)\big). \tag{24}$$

Suppose the operator accepts a reservation only if the estimated number of free bays is at least a safety margin m, so accept iff $N(1 - \hat{\rho}) \geq m$, or equivalently $\hat{\rho} \leq \rho^* = 1 - m/N$. Then a positive bias causes wrongful *rejection* when $\varphi\delta > \rho^* - \rho$, and a negative bias causes wrongful *acceptance* (overbooking) when $\varphi|\delta| > \rho - \rho^*$. The smallest controlled fraction that flips the decision for a given δ is

$$\varphi_{\min} = \frac{|\rho^* - \rho|}{|\delta|}, \tag{25}$$

which exists as a feasible fraction (≤ 1) only if $|\delta| \geq |\rho^* - \rho|$.

Table XLIV takes the simulated lot size of the reference paper, $N = 150$ bays [2], with an illustrative true occupancy of 0.80 and a margin of 18 bays (12% of the lot), so $\rho^* = 0.88$. A full-strength attack ($\delta = 1$) needs only $\varphi_{\min} = 0.08$, that is 12 bays; a half-strength shift needs 0.16 (24 bays); a mild shift of $\delta = 0.10$ would need 0.80 of the lot, or 120 bays, and is therefore of little concern. The reverse case is equally simple:

TABLE XLIV
ESTIMATED OCCUPANCY $\hat{\rho}$ UNDER BIAS (ILLUSTRATIVE: $N = 150$, $\rho = 0.80$, $m = 18$, HENCE $\rho^* = 0.88$). STARRED ENTRIES CAUSE REJECTION

	Shift δ			
φ	0.10	0.25	0.50	1.00
0.05	0.805	0.813	0.825	0.850
0.10	0.810	0.825	0.850	0.900*
0.20	0.820	0.850	0.900*	1.000*
0.40	0.840	0.900*	1.000*	1.000*

if 141 bays are truly occupied (9 free) and 18 free bays are needed, the attacker has to make only $18 - 9 = 9$ occupied bays report "free" to obtain a wrongful acceptance. The same bias fed into a learned forecaster is worse than this single-step effect, because it also corrupts the training data and leaves a persistent error in future forecasts.

a) Plausibility checks as detectors.: Two cheap checks use information that the attacker would have to falsify consistently. The first is cross-sensor agreement between independent sensor types on the same bay, such as ultrasonic and magnetic. The second is a conservation law at the entrances: with E entries and X exits counted since a reference time at which occupancy was $o(0)$,

$$o(t) = o(0) + E(t) - X(t), \quad (26)$$

so the sum of bay reports $o\hat{}(t)$ should match the right-hand side. An attacker who flips s bay reports without touching the gate counters creates a discrepancy $D = o\hat{}-o(0)-E+X$ of size s.

Counters are not perfect, so the test needs a threshold τ, and the choice of τ trades false alarms against missed detections. Model the counting noise, as an assumption, as independent misses with probability ε per event, so that missed entries and missed exits within a window are Poisson with means εE and εX. Their difference D then follows a Skellam distribution with mean $\varepsilon(E-X)$ and variance $\varepsilon(E+X)$. Take $E = X =$ 100 events per hourly window and $\varepsilon = 0.01$ (illustrative): the mean is zero and the variance is 2. We raise an alarm when $|D| > \tau$. Table XLV gives the exact false-alarm probability per window, the expected number of false alarms per day for 24 windows, and the detection probability when $s = 5$ and $s = 10$ bays are spoofed.

A threshold of $\tau = 4$ gives about one false alarm every two weeks (0.076 per day) while catching a 10-bay spoof almost surely and a 5-bay spoof about 65% of the time within one window; repeated windows raise the detection chance of a persistent attack. The check is blind to attacks that shift gate counters as well, which is why entry and exit counters should themselves be authenticated and sourced from a different sensor class. The numbers also show a design rule: the usable threshold scales with $\sqrt{\varepsilon(E+X)}$, so busier lots tolerate a larger absolute discrepancy but a smaller relative one.

7) Privacy: traces, linkability, and legal context:

TABLE XLV
THRESHOLD τ FOR THE CONSERVATION CHECK (ILLUSTRATIVE POISSON COUNTING NOISE; $E = X = 100$ PER HOUR, $\varepsilon = 0.01$)

	False alarms		Detection		
	per window	per day			
2	0.0745	1.79	0.963	1.000	
3	0.0169	0.41	0.870	1.000	
4	0.0032	0.076	0.654	1.000	
5	0.0005	0.012	0.346	0.998	
τ6	0.00007	0.002	0.131	0.992	$s = 5$
		$s = 10$			

a) Linkability.: A parking record, taken alone, says that some vehicle used a lot at some time. Linked with other records it becomes a movement trace. The identifiers that make linkage easy are the number plate, a fixed RFID or QR credential, a payment token reused across visits, and an account identifier. Even without any of these, the *pattern* of visits can single out a person. As a back-of-envelope bound, suppose five lots, 24 hourly arrival bins, and 7 weekdays, giving $5 \cdot 24 \cdot 7 = 840$ distinguishable (lot, hour, day) cells. Three observed visits correspond to $840^3 = 592{,}704{,}000$ ordered combinations. If a city's one million regular users distributed their visits uniformly over the cells, the expected number of other users sharing the same three-visit pattern would be about $10^6/592{,}704{,}000 \approx 0.0017$, so the pattern would almost surely be unique. Real behavior is far from uniform (commuters cluster in a few cells), which makes patterns less unique than this calculation; the point is that a handful of timestamps can serve as a quasi-identifier, so removing names does not make a trace anonymous. Encryption does not remove the risk either, because an eavesdropper who sees only the timing of frames from a sensor that reports on change of state still learns when vehicles arrive and leave that bay.

b) Privacy by design.: Cavoukian's seven foundational principles [81] are proactive prevention, privacy as the default, privacy embedded into the design, full functionality without false trade-offs, end-to-end security through the data lifecycle, visibility and transparency, and respect for user privacy. For this system they translate into default settings that collect the minimum, a data-flow diagram reviewed for personal data before deployment, and user-facing controls.

c) Legal context (not legal advice).: Two regimes are relevant to a system studied in an Indian setting that may also process data of foreign visitors: the EU General Data Protection Regulation [66] and India's Digital Personal Data Protection Act, 2023 [67]. Both regulate the processing of personal data, which in this context is likely to include number plates and any identifier linked to a person. The following mapping is high-level and for design orientation only; the exact obligations, exemptions, and rules depend on jurisdiction and on the operator's role, and should be confirmed with qualified counsel.

TABLE XLVI
PRIVACY PRINCIPLES MAPPED TO DESIGN REQUIREMENTS (OUR HIGH-LEVEL READING)

Principle	Design requirement
Data minimisation	Sensors send occupancy bits only; cameras output counts; no plate or face is stored; coarse timestamps where possible
Purpose limitation	Parking records are used for guidance, billing, and capacity planning; any other use (such as analytics for third parties) needs a separate decision and notice
Retention limit	Raw traces are deleted or aggregated after a fixed short period; billing records are kept only as long as accounting requires
Notice and consent	Visible signs at lots; an in-app notice that states what is collected and why; consent where it is the basis of processing
Security safeguards	Per-device keys, TLS on backhaul, access control and audit logs on stored data, breach-response procedure

User rights	Account holders can see, correct, and delete their data; deletion propagates to backups within a stated period	
Accountability	A written data-flow inventory and an impact assessment before deployment of cameras or large-scale tracking	
Physical attack surface	Few sites	More devices in the field to harden
Operation during outage	Stops with the link	Local service continues
Patching and keys	Centralized	Many nodes to update and audit
Data minimisation	Done after upload	Applied before upload

d) Cameras.: Camera sensing is the most privacysensitive element, because a frame can contain plates, faces, and the surroundings. The GDPR in particular expects an impact assessment when a public area is systematically monitored on a large scale [66]. Design measures that follow from Table XLVI are on-device inference with only counts leaving the node, no frame storage beyond the processing buffer, a field of view and mounting angle that exclude footpaths and neighboring property, masking of regions that are not parking lanes, and visible signage. These are consistent with the count-only camera path in Section V and with the limitation discussed in Section IV-B.

e) Cloud-only versus fog processing.: Fog placement can limit exposure, but it moves rather than removes risk. Table XLVII compares the two options on data exposure, following the general discussions of IoT security and privacy challenges in the literature [82], [83].

8) The cost of security: airtime and battery: Every byte of security overhead is paid for in airtime, and airtime is paid for in energy and, on regulated bands, in duty-cycle budget. We quantify this for the application-layer additions suggested above, such as a sequence number or time bucket, a truncated message authentication code, or an authenticated-encryption tag, whose sizes we take as $+4$, $+8$, and $+16$ bytes.

a) Time on air.: The time on air T_{air} follows the standard formula of Eqs. (2)–(5), with CRC enabled and explicit header. We use $BW = 125$ kHz, coding rate 4/5, 8 preamble symbols, and low-data-rate optimization for SF11 and SF12 only. The physical payload is $PL = 13 + L$ bytes, where 13 is the LoRaWAN overhead (header with device address, control, counter, and port, plus the 4-byte MIC) and L the application payload; we take $L = 4$ bytes as the base case. Table XLVIII gives the results, computed by the code of Listing 1.

Two details stand out. The relative cost is highest at the faster data rates, where a few extra bytes occupy a larger share of a short packet: 16 added bytes lengthen an SF7 frame by 39.8% and an SF9 frame by 49.7%. Second, airtime grows in steps because the payload is carried in blocks of symbols: at SF12, $+4$ and $+8$ bytes both round up to the same number of symbols, so an 8-byte tag costs no more than a 4-byte tag, and a designer should choose the largest tag that fits into the current block. Under a 1% duty cycle (an assumption, as above) the hourly budget is 36 s of airtime, which allows at most $36/T_{\text{air}}$ messages per hour: 700 at SF7 and 27 at SF12 for the base frame, and 500 and 19.9 with 16 added bytes.

TABLE XLVII
DATA EXPOSURE: CLOUD-ONLY VERSUS FOG-ASSISTED PROCESSING (OUR QUALITATIVE COMPARISON)

Aspect	Cloud only	Fog-assisted
Raw data leaving site	All readings and video	Only summaries and counts, if designed so
Backhaul exposure	Every item crosses the Internet	Fewer, aggregated items
Breach blast radius	One central store of everything	Cloud holds less detail; but each fog node holds local raw data

TABLE XLVIII
TIME ON AIR (MS) WITH ADDED SECURITY BYTES; PERCENT CHANGE IN PARENTHESES (L = 4 B BASE PAYLOAD, 125 KHZ)

SF	+0 B	+4 B	+8 B	+16 B
7	51.5	56.6 (+10.0%)	61.7 (+19.9%)	71.9 (+39.8%)
9	164.9	185.3 (+12.4%)	205.8 (+24.8%)	246.8 (+49.7%)
12	1318.9	1482.8 (+12.4%)	1482.8 (+12.4%)	1810.4 (+37.3%)

Listing 1. Time-on-air and battery-life model used for Tables XLVIII and XLIX.

```
import math

def toa(pl, sf, bw=125e3, cr=1, npre=8, crc=1, ih=0):
    de = 1 if sf >= 11 else 0 ts = 2**sf /
    bw n = 8 + max(math.ceil(
        (8*pl - 4*sf + 28 + 16*crc - 20*ih)
        / (4*(sf - 2*de))) * (cr + 4), 0) return (npre + 4.25) * ts + n
    * ts

def life_years(sf, extra, per_day=144): p_tx = 3.3 * 0.040      #
    W p_sl = 3.3 * 5e-6              # W e_fix = 0.020          #
    J per message e_bat = 2.4 * 3.3 * 3600 # J t = toa(13 + 4 +
    extra, sf) e_msg = p_tx * t + e_fix e_day = per_day * e_msg
    + p_sl * (
        86400 - per_day * t) return e_bat /
    e_day / 365.25
```

b) Energy and battery life.: We use a deliberately simple energy model whose parameters are assumed for illustration and must be replaced by measured values for any real sensor. Per message, $E_{\text{msg}} = P_{\text{tx}} T_{\text{air}} + E_{\text{fix}}$, where $P_{\text{tx}} = V\, I_{\text{tx}} = 3.3$ V × 40 mA = 0.132 W is the transmit power draw, and $E_{\text{fix}} = 20$ mJ

TABLE XLIX
ILLUSTRATIVE BATTERY LIFE (YEARS) AND CHANGE WITH ADDED SECURITY BYTES (n_d = 144, ASSUMED ENERGY MODEL)

SF	+0 B	+4 B	+8 B	+16 B
7	14.77	14.51 (−1.8%)	14.25 (−3.6%)	13.76 (−6.9%)
9	10.49	9.97 (−5.0%)	9.50 (−9.5%)	8.68 (−17.3%)
12	2.66	2.40 (−9.6%)	2.40 (−9.6%)	2.02 (−24.1%)

lumps together wake-up, sensing, and the receive windows, which we hold constant. With n_d messages per day, sleep power P_{sl} = 3.3 V×5 μA = 16.5 μW, and a battery of capacity C = 2.4 Ah at V = 3.3 V (energy $E_b = CV \cdot 3600$ = 28,512 J), the daily energy and the battery life in days are

$$E_{day} = n_d E_{msg} + P_{sl}\,(86400 - n_d T_{air}), \tag{27}$$

$$L_{bat} = E_b / E_{day} \quad \text{(days, with } E_{day} \text{ in J/day)}. \tag{28}$$

Table XLIX uses n_d = 144. The model ignores battery selfdischarge, temperature, and voltage sag, so the absolute lifetimes are optimistic; the relative changes are the informative part.

The per-message energy at SF7 rises from 26.79 mJ to 29.50 mJ with 16 added bytes, a 10% increase, but because the fixed cost dominates at low spreading factors the battery life falls by only 6.9%. At SF12 the radio dominates (E_{msg} is 194.1 mJ before and 259.0 mJ after), and the same overhead removes almost a quarter of the lifetime. The practical implication for the improvements of Section V is twofold: security overhead is cheap for sensors with good links and costly for those at the cell edge, so spreading factors should be assigned per device, and an operator should include the cost of authentication in the link budget rather than adding it afterward.

M. Deep Dive: Pricing Mechanisms and Evidence

The pricing limitation (Section IV-F) combines three issues: what the mechanism optimizes, whether its incentives hold, and what the evidence behind it can show. This deep dive treats each with definitions, worked numbers, and an illustration of sensitivity. The Monte Carlo results below are computed on synthetic data of our own and are not evidence about any real parking system.

1) A primer on uniform-price multi-unit auctions: Let k identical spaces be offered to n bidders, each of whom wants one space and has a value v_i (the most she would pay) and submits a bid b_i. Write the bids in decreasing order as $b_{(1)} \geq b_{(2)} \geq \cdots \geq b_{(n)}$. In a *uniform-price* auction all winners pay the same price p. Given p, the demand is $D(p) = |\{i : b_i \geq p\}|$, the number of units sold is $\min(k, D(p))$, and the revenue is

$$R(p) = p \cdot \min\big(k, D(p)\big). \tag{29}$$

Revenue is piecewise linear in p and can jump only at bid values, so an optimal price can always be taken from the set of bids. Setting $p = b_{(j)}$ with $j \leq k$ sells (at least) j units, which gives the revenue-maximizing rule

$$j_* = \arg\max_{1 \leq j \leq \min(k,n)} j\, b_{(j)}, \qquad p_* = b_{(j_*)}, \qquad R_* = j_* b_{(j_*)}. \tag{30}$$

This is the rule that Section V states for the reference mechanism. It treats the realized bids as a demand curve and picks the point on it with the largest rectangle under the curve.

Two other prices serve as reference points. The *marketclearing price* is $p_c = b_{(k+1)}$, the highest rejected bid, which makes demand equal supply: exactly k bidders have bids strictly above it when there are no ties. It yields $R_c = k\, b_{(k+1)}$ and is the price at which the highest rejected bidder would be indifferent. A *posted price* p_0 is announced in advance and bids do not influence it; revenue is $p_0\, \mathrm{E}[\min(k, D(p_0))]$ over the random demand. Under the standard model of independent private values drawn from a distribution F with density f, Myerson's analysis [84] shows that a revenue-maximizing mechanism for identical units is a $(k+1)$-st-price auction with a reserve price r^* that solves $r^* - \big(1 - F(r^*)\big)/f(r^*) = 0$ (for regular distributions), and Krishna's textbook treatment covers the multi-unit case [51]. Two points deserve emphasis. The rule (30) maximizes revenue *given the submitted bids*, which is a different matter from maximizing expected revenue when bidders respond to the rule. And it is a rule applied after the bids are known, so the posted price it implies is the best price in hindsight, an upper bound that no price fixed in advance can exceed on the same bids.

2) The reference paper's worked example: The main text (Section IV-F) describes the example of the reference paper as 20 bids of USD 200, three of USD 150, and seven of USD 90, so n = 30, and states that the paper names USD 150 as optimal. Section III-D recomputed the example from rule (30) and tabulated the revenue of each price (Table XVII): USD 200 is revenue-maximizing for every k, and USD 150 never is. We add only the market-clearing price $b_{(k+1)}$, whose revenue $k\, b_{(k+1)}$ is 1000, 2000, 3000, and 2070 for k = 5, 10, 20, and 23.

The clearing price $b_{(k+1)}$ equals 200 for $k \leq 19$, 150 for $k \in \{20,21,22\}$, and 90 for $k \in \{23,\ldots,29\}$; for k = 30 no bid is rejected and the clearing price is undefined (it would be set by a reserve). At k = 20 the clearing price of USD 150 yields USD 3000, 25% below the USD 4000 of the revenuemaximizing price. We cannot tell from the main text whether the paper intended a clearing-price reading; we only note that the revenue objective and the clearing objective give different answers on this example.

a) When could 150 be optimal?: USD 150 is optimal only under a different objective. Suppose the operator first maximizes the number of spaces *allocated*, for example to maximize occupancy, and uses revenue only to break ties. At p = 200 only 20 bidders buy; at p = 150 up to 23 do; at p = 90 up to 30 do. For k = 21,22,23 the allocation is $\min(k,23)$ at p = 150 and at p = 90 but only 20 at p = 200, and the tie between 150 and 90 is broken by revenue in favor of 150 (for k = 23: 3450 against 2070). For $k \geq 24$, price 90 fills more spaces (24 to 30 against 23), and for $k \leq 20$, 200 allocates as

many as any other price. So 150 is the allocation-first optimum exactly for $k \in \{21,22,23\}$. A weighted objective gives the same qualitative result: if the operator values each occupied space at an additional λ USD, then at $k = 23$ price 150 beats 200 iff $3450+23\lambda > 4000+20\lambda$, that is $\lambda > 550/3 \approx 183.3$. Such a λ is large relative to the bids and so is implausible as a stated rationale. The comparison also shows why the objective needs to be stated before the mechanism is judged: "the best price" is not defined until the objective is.

3) Bid shading and incentive compatibility: A mechanism is incentive compatible if bidding one's true value is optimal. For a single unit, the second-price (Vickrey) auction has this property in a dominant-strategy sense [50]. For k identical units and bidders who each want *one* unit, with private values and quasi-linear utility (value minus payment), the corresponding rule awards the units to the k highest bids and charges every winner the highest losing bid $b_{(k+1)}$. A winner's price does not depend on her own bid, and she wins exactly when her value exceeds the price she would face, so truthful bidding is weakly dominant. The statement fails if the price is the lowest *winning* bid, $b_{(k)}$, or the pay-yourbid price, and it fails for the revenue-maximizing rule (30), because there the price is chosen with the help of the bids of the winners. It also fails for bidders with demand for several units, such as a company renting several bays: such a bidder can lower the price on all her units by understating demand for the marginal ones, which is called demand reduction in the literature [51]. The Vickrey–Clarke–Groves construction [50], [85], [86] restores truthfulness for general multi-unit demand but charges each bidder the externality she imposes on others, so prices differ across winners, which is a hard feature to explain to customers.

*a) Worked example: pay-your-bid versus second price.:*Let $k = 2$ and three bidders have unit-demand values 100, 80, and 60; bidders B and C bid their values and we vary A's bid. Under *pay-your-bid*, A bidding 100 wins and pays 100 for a surplus of $100 - 100 = 0$. If A shades to 61, she still ranks among the top two bids (61 exceeds C's 60), wins, and pays 61 for a surplus of $100 - 61 = 39$. Under the $(k+1)$-*st-price* rule the price is 60, set by C. A bidding 100 wins and pays 60 for a surplus of 40; shading to 61 changes nothing (still wins, price still 60, surplus 40); shading to 59 loses the unit and gives surplus 0. No deviation beats truth-telling, whereas under pay-your-bid shading raised A's surplus from 0 to 39. Any price rule that lets bids pull the price down invites exactly this behavior.

b) Shading against the revenue-maximizing rule.: The reference rule can also be gamed collectively, which matters if bidders are a stable group of regular subscribers. In the worked example with $k \geq 23$, suppose m of the 20 bidders whose value is 200 lower their bid to 150. The sorted bids are then $(20-m)$ of 200, $(3+m)$ of 150, and 7 of 90. Rule (30) compares $200(20 - m)$ with $150 \cdot 23 = 3450$. For $m \leq 2$ the first is at least 3600 and the price stays at 200, so one or two shaders gain nothing (they either lose the space or pay 200). For $m = 3$ the first is $3400 < 3450$ and the price drops to 150, so each of the three shaders obtains a space that was worth 200 to her for a payment of 150, a surplus of 50 against 0 under truthful bidding, and the operator's revenue falls from 4000 to 3450 (a 13.75% loss). The example shows that the rule needs only a small coalition to be profitable to manipulate, and that its revenue gain is therefore fragile when bidders learn the rule.

4) The synthetic-bid problem: The main text notes that the reference evaluation generated long-term bids randomly above the default subscription rate. We show why this makes a revenue gain over the fixed rate close to automatic. Let r_0 be the default rate and suppose every bid satisfies $b_i \geq r_0$ and $n \geq k$. A fixed-rate scheme sells all k spaces at r_0 and earns kr_0. The revenue-maximizing uniform price satisfies

$$R^* \geq k\, b_{(k)} \geq k\, r_0, \tag{31}$$

since $j = k$ is one of the candidates in (30). The same bound holds for $n < k$ with n in place of k. The auction therefore cannot earn less than the fixed rate for the same allocation, and equals it only if every bid sits exactly at r_0. A numerical illustration: let $r_0 = 100$, $k = 5$, and six bids $180,150,130,120,110,105$. Then $j\ b_{(j)}$ for $j = 1,\ldots,5$ is $180,300,390,480,550$, so $R^* = 550$ against 500 at the fixed rate, a gain of 10.0%. The gain is produced by the datagenerating assumption and says nothing about whether real subscribers would bid above r_0.

a) When bids may fall below the default rate.: If bids are drawn from a distribution that extends below r_0, the comparison depends on the parameters, and on whether bids equal values. We illustrate this by simulation. Values are lognormal, $v_i = \mu e^{0.4Z_i}$ with $Z_i \sim \mathrm{N}(0,1)$ and median μ; there are $n = 30$ bidders, $k = 20$ spaces, and $r_0 = 100$. The fixed-rate scheme sells to bidders with $v_i \geq r_0$ up to k; the auction applies rule (30) to bids $b_i = sv_i$, where $s \leq 1$ is a common shading factor ($s = 1$ is truthful bidding). We report the mean over 20,000 replications per cell with a fixed random seed (42). Table L gives the percent change in mean revenue of the auction relative to the fixed rate and, in parentheses, the fraction of replications in which the auction earned less.

Three points emerge, none of which is evidence about a real lot. With truthful bids, the auction never loses in any

TABLE L
SIMULATED REVENUE CHANGE OF THE AUCTION VERSUS THE FIXED RATE $r_0 = 100$, IN PERCENT (FRACTION OF RUNS WITH A LOSS). SYNTHETIC ILLUSTRATION ONLY

	Shading factor s			
Median μ	1.0	0.9	0.8	0.7
70	+119.3 (0.00)	+97.4 (0.00)	+75.5 (0.02)	+53.5 (0.06)
100	+17.0 (0.00)	+5.3 (0.38)	−6.4 (0.72)	−18.1 (0.91)
130	+14.9 (0.00)	+3.4 (0.39)	−8.1 (0.87)	−19.6 (>0.99)

Posted price p (USD)

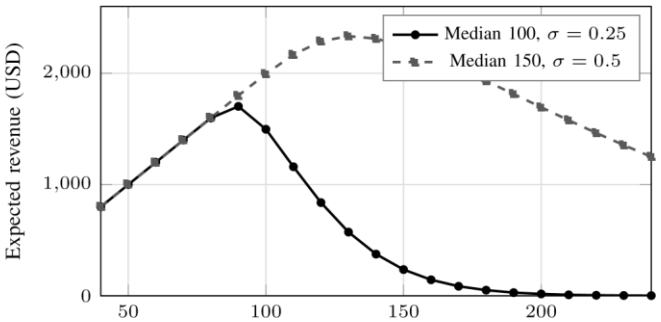
Fig. 9. Expected revenue of a posted price for 30 bidders and 20 spaces under two assumed lognormal demand distributions (σ is the log-standarddeviation). Illustrative; both curves are computed, not measured.

replication, which is again inequality (31) at work (the rule is the best price in hindsight). The size of the gain depends strongly on the demand distribution: it is large when the fixed rate is far from the typical value and the fixed-rate scheme sells few spaces (median 70), and modest when the default rate is near the median. And the sign of the gain depends on bidder behavior: once bidders shade by 20% or more, the auction loses to the fixed rate on average when the default rate is near the median of values. A pricing claim therefore needs a statement about bidder behavior as well as about demand.

Fig. 9 shows the other side of the same issue: the expected revenue of a *posted* price for two assumed demand distributions, computed exactly as $p\mathrm{E}[\min(k,D)]$ with $D \sim \mathrm{Binomial}(30,\ 1 - F(p))$ and $k = 20$. In the first, values are lognormal with median 100 and log-standard-deviation 0.25; in the second, median 150 and 0.5. Both curves rise linearly while capacity binds, since all 20 spaces are sold and revenue is $20p$, and then fall once demand drops below capacity. The best price is about 89 (expected revenue 1706) for the first and about 131 (2334) for the second. A single default rate of 100 is 12% below the best for the first (1497 against 1706) and 15% below for the second (1995 against 2334). The optimal fixed or reserve price is a feature of the demand distribution, which is why calibration, discussed below, comes before any revenue claim.

5) Short-term tiered discounts and occupancy-aware multipliers: The reference paper reports that the tiered shortterm discount lowered short-term revenue relative to fixed pricing and that the overall gain came from long-term bids (Section IV-F). Revenue accounting makes the implication explicit. Write the total revenue as $R = R_L + R_S$ for long-term and short-term parts. With baseline shares $s_L = R_L^0/R^0$ and $s_S = 1 - s_L$ and growth rates g_L and g_S relative to fixed-rate pricing,

$$\frac{\Delta R}{R^0} = s_L\, g_L + s_S\, g_S. \tag{32}$$



If $g_S < 0$, the total gain is positive only if $g_L > -(s_S/s_L)g_S$. As a hypothetical illustration, with $g_S = -3\%$ and a total gain of $+10\%$, the long-term component must grow by $g_L = (0.10 + 0.03s_S)/s_L$: 18.7% if long-term revenue is 60% of baseline, 29.5% if it is 40%, and 62% if it is 20%. These numbers are not claims about the reference system, whose revenue shares we do not know. They show that a headline gain can hide a large effect on the smaller segment and that the segment shares should be reported.

TABLE LI
REVENUE CHANGE $m^{1+\epsilon} - 1$ UNDER A CONSTANT-ELASTICITY DEMAND MODEL (ILLUSTRATIVE)

Elasticity ϵ	$m = 0.8$ (discount)	$m = 1.2$ (surcharge)
−0.3	−14.5%	+13.6%
−0.6	−8.5%	+7.6%
−1.0	0.0%	0.0%
−1.5	+11.8%	−8.7%

a) Elasticity reasoning.: Let demand respond to a price multiplier m with constant elasticity $\epsilon < 0$, so that quantity scales as m^{ϵ} and revenue as $m^{1+\epsilon}$. Then a discount ($m < 1$) raises revenue only if demand is elastic, $|\epsilon| > 1$, and a surcharge ($m > 1$) raises it only if demand is inelastic, $|\epsilon| < 1$. Table LI shows the revenue change $m^{1+\epsilon} - 1$ for a 20% discount and a 20% surcharge. A discount ladder that lowers short-term revenue, as reported, is therefore consistent with short-term demand that is not strongly elastic; the data we have do not identify the elasticity, and the model ignores capacity limits, substitution between periods, and the possibility that discounts pull in additional drivers.

An occupancy-aware multiplier has a different purpose from a revenue-maximizing price: it steers occupancy toward a target band so that drivers can find a space. When capacity binds, a higher price does not reduce the number of spaces sold as long as enough drivers remain willing to pay it, so the surcharge raises revenue while it frees a few spaces for later arrivals; this is the logic behind demand-responsive pricing of curb parking. Pierce and Shoup evaluated San Francisco's program of adjusting on-street meter rates by observed occupancy and discuss whether the adjustments moved occupancy toward the target [19]; the setting is on-street parking, not reservationbased lots, so the transfer of its conclusions is uncertain. At the model level, Arnott and Inci

show in a stylized equilibrium model that curb parking priced below the level that avoids cruising contributes to traffic congestion [18], and Lei and Ouyang study joint pricing and reservation for urban parking in a dynamic model [49]. We read these as supporting the *direction* of an occupancy-linked price but not as providing parameters for any particular lot.

6) Data needed to calibrate a demand model: A credible revenue claim needs a demand model whose parameters are estimated from data about the lot in question. Table LII lists four sources with their strengths, limits, and ethical points.

The sources complement each other: occupancy and payment logs are plentiful but passive, surveys are cheap but hypothetical, and experiments are causal but costly. A staged plan would start with logs to fit a baseline demand curve,

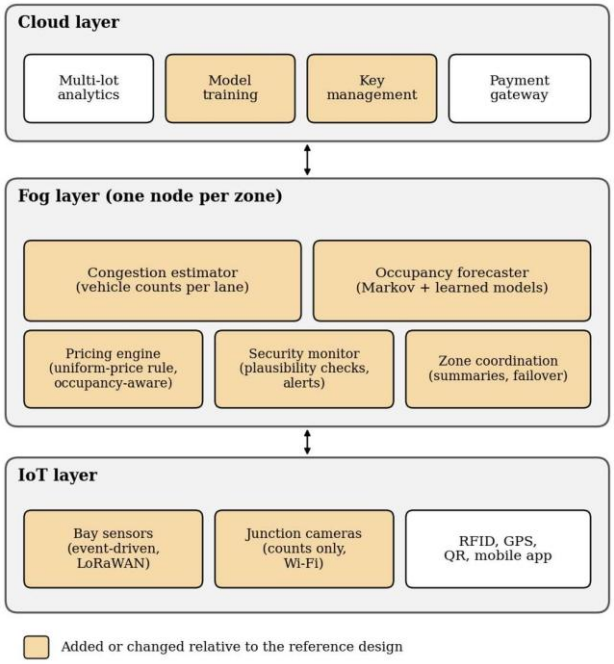


Fig. 10. Proposed architecture. Shaded blocks are added or changed relative to the reference design.

use a small survey to bound the range of price response, and then run a bounded experiment to confirm the elasticity before dynamic multipliers are enabled. Fairness matters throughout. A price rule that raises prices when a lot is predicted to fill burdens the drivers with the least flexibility, such as shift workers and people with mobility needs. Common safeguards are a published price cap and floor, a rule that spaces reserved for accessibility are never subject to surge multipliers, notice of price changes before purchase, and periodic review of who pays what. These safeguards also limit the damage if a demand model is wrong. Shading by regular subscribers, which Table L showed to be capable of reversing the sign of a revenue gain, is another reason to prefer calibrated posted prices and bounded experiments over reliance on bids alone until the behavior of real bidders has been observed.

V. Proposed Improvements

The proposals below keep the three-tier structure of the reference design and change only the components that Section IV found weak. Fig. 10 shows the resulting architecture, and each subsection maps one limitation to one improvement. All proposals are meant to be testable, and the measures used to test them are set out in Section VI.

A. Zoned Fog Coordination and Radio Planning

To address scale, the lot is divided into zones, each served by a gateway and a fog node that handles only its own sensors. Zones exchange compact summaries, namely free-bay counts and lane congestion, instead of raw readings, and a neighboring node adopts the sensors of a failed one, which extends the handover idea that the reference paper mentions [2]. The cloud

TABLE LII
DATA SOURCES FOR CALIBRATING A PARKING DEMAND MODEL (OUR ASSESSMENT)

Source	What it provides	Limits	Ethical and fairness notes
Occupancy history from sensors	Arrival and departure patterns by time, day, and event; baseline utilization	Shows only what happened at the price that was charged, so price response cannot be separated from time effects; censored when the lot is full	Aggregate before storage; no personal identifiers needed
Payment and booking logs	Realized prices, stay durations, cancellations, no-shows, repeat use	Reflects only those who parked; excludes those deterred by price; linked records are personal data	Pseudonymize accounts; keep retention short; avoid per-person price profiling
Stated-preference surveys	Willingness to pay for attributes (price, distance, reservation) through hypothetical choices	Hypothetical bias (stated differs from actual); sample may not represent all users	Obtain consent; avoid sampling only app users, who skew toward higher income
Controlled price experiments (A/B tests)	Causal estimate of price response when prices are randomized across periods or zones	Needs enough volume and time; spillover between zones; cost of testing a bad price	Disclose that prices vary; limit the range by a cap; do not test on essential services or accessible bays

keeps only cross-lot views. On the radio side, sensors report on a change of state, with a periodic heartbeat and a small random offset that spreads transmissions in time and reduces collisions, and spreading factors are assigned per device according to link quality, following the radio-parameter optimization studied by Sallum et al. [42]. Node load, end-to-end delay, and packet delivery ratio are logged from the outset, because these are the quantities the reference paper did not report.

B. Camera-Assisted Congestion Estimation

Junction cameras feed a compact object detector running on the fog node, of the single-pass kind exemplified by YOLO [53], which counts vehicles per lane. The count divided by lane capacity gives the density used by the reference routing model, so the routing algorithm stays unchanged and only its input becomes measured. Only counts leave the node; frames are discarded after processing, which limits both bandwidth use and privacy exposure. When a camera fails or is obstructed, the system falls back to a proxy computed from entry and exit counters and from the occupancy of nearby bays, and it marks the degraded mode in its logs. Estimates are checked against manual counts collected during pilot hours.

C. Learning-Based Occupancy Forecasting

We propose to keep the Markov chain as an interpretable baseline and to add two data-driven forecasters: a long shortterm memory network [74], suited to sequential occupancy data, and a gradient-boosted tree model [73], suited to tabular features. Inputs would include recent occupancy lags, entry and exit rates, hour of day, day of week, calendar season (including the Hijri calendar used in the reference case study), and an event flag. Models are trained in the cloud on historical data and deployed in compact form to the fog node, and forecasts are produced for horizons from fifteen minutes up to the three-hour reservation lead time. To make reservation decisions safer, the forecaster also outputs a prediction interval, and a booking is accepted only when the lower bound of forecast availability exceeds a safety margin. A check on recent error triggers retraining when demand patterns shift, for example around unusual events. Evaluation against the Markov model and against simple persistence forecasts follows the practice of earlier studies [12], [13].

D. Staged Field Validation

Validation is organized in three stages of increasing realism. First, the SUMO model [7] is extended to a lot of the real size of about 2,500 spaces, with additional baselines, repeated runs so that confidence intervals can be reported, and stress scenarios such as event surges and sensor outages. Second, a bench test with several sensor nodes and injected packet loss exercises the LoRa link and the fog software under controlled faults. Third, a supervised pilot in one section of a real lot runs for several weeks, and its readings are compared with manual occupancy audits. Each stage has to be passed before the next begins, which keeps the cost of failure low.

E. Layered Security and Privacy Design

Security begins with an explicit threat model covering spoofed or replayed sensor frames, rogue gateways, reservation flooding, and leakage of location and payment data. Against these threats we propose four measures.

- *Device and link protection:* unique per-device keys, the frame counters and session keys built into LoRaWAN for replay protection, secure-element storage of keys [15], TLS between fog nodes and the cloud, and the enhanced protocol of You et al. [14] evaluated as an option.
- *Plausibility checks:* a bay report is cross-checked against a second sensor type and against entry and exit counters, and inconsistent reports raise an alert. This gives the security-alert function of the reference workflow a concrete definition.
- *Reservation safeguards:* rate limits per account, short holding times for unpaid bookings, and small cancellation penalties to deter flooding.

- *Privacy by design:* pseudonymous identifiers, short retention of location traces, payment through a thirdparty gateway so that card details never reach parking servers [9], and no storage of video.

The delay and energy overhead of these measures is recorded together with performance, so that the trade-off stays visible.

F. Demand-Calibrated Pricing

For long-term places, the mechanism is stated precisely. Bids are sorted from highest to lowest, and the price is set to the *j*-th highest bid, where *j* is chosen to maximize *j* times that bid, subject to *j* not exceeding the number of spaces k. Applied to the worked example in the reference paper [2], this rule selects USD 200 for any k, which is consistent with the revenue objective that the paper itself states. Simulated bids are then drawn from distributions fitted to observed data or to a small stated-preference survey, and results are reported over several distributions to show how sensitive the outcome is to that assumption. For short-term parking, the fixed discount ladder is complemented by an occupancy-aware multiplier driven by the forecast state, so that prices rise when a lot is predicted to fill and ease when it is predicted to stay empty, with an operator-set cap for fairness. Finally, because uniformprice bidding can invite bid shading, sealed-bid rules with better incentive properties, in the tradition of Vickrey [50], are left as a topic for study.

G. Design Detail: Zoned Fog Coordination

The subsections above state each improvement in a few sentences. The subsections that follow give the design detail needed to implement and test them. Everything here is a *design proposal*: nothing has been built or run, every number is illustrative, and every threshold is a proposal to be fixed before a study (Section VI). Radio quantities such as time on air and the per-channel sensor limits are taken from the analysis in Section IV.

1) Zone partitioning: Let the lot be the junction graph $G = (V,E)$ of Section III, with a weight w_v equal to the number of bay sensors attached to junction v, and an edge weight ω_{uv} equal to the expected vehicle flow over the lane segment (u,v). A partition into Z zones is a map $z : V \to \{1,\ldots,Z\}$. We want zones that carry equal sensor load, so that no gateway is overloaded, and few cut roads, so that vehicles seldom cross a zone boundary and the summary exchange stays small:

$$\min \sum_{(u,v)\in E:\, z(u)\neq z(v)} \omega_{uv}$$
$$\text{s.t. } (1-\varepsilon)\frac{W}{Z} \le W_k \le (1+\varepsilon)\frac{W}{Z}, \quad W_k \le N_{\max}, \qquad (33)$$

where $W_k = \sum_{v:z(v)=k} w_v$ and $W = \sum_v w_v$. The cap $N_{\max}$ is the per-channel sensor limit from the radio analysis. For the 2,500-space lot, a 15-minute heartbeat at SF9 gave $N_{\max} = 262$, so $Z \ge \lceil 2500/262 \rceil = 10$, whereas hourly heartbeats ($N_{\max} = 656$) would permit $Z = 4$. We therefore take $Z = 10$ and a tolerance $\varepsilon = 0.10$, i.e., 225 to 275 sensors per zone around a target of 250. Balanced graph partitioning is NP-hard, so we propose a seed-and-grow heuristic with a local refinement pass (Algorithm 2); any standard multilevel partitioner could replace it.

Line 4 grows the lightest zone first, which keeps loads balanced; line 5 prefers vertices strongly connected to the zone, which keeps the cut small. The refinement pass is a onevertex version of the Kernighan–Lin idea and stops because Algorithm 2 Greedy Zone Partitioning with Boundary Refinement

Require: $G = (V,E)$, weights w_v, ω_{uv}, Z, ε
1: pick Z seeds by farthest-point selection on road distance
2: $z(s_k) \leftarrow k$; $W_k \leftarrow w_{sk}$ for each seed
3: while unassigned vertices remain do 4: $k \leftarrow \operatorname{argmin}_k W_k$ among zones with a free neighbor
5: $v \leftarrow$ free neighbor of zone k with the largest $\sum_{u:z(u)=k} \omega_{uv}$
6: $z(v) \leftarrow k$; $W_k \leftarrow W_k + w_v$
7: end while
8: repeat refinement
9: for each boundary vertex v, zone $k = z(v)$, neighbor zone j do
10: $g \leftarrow \sum_{u:z(u)=j} \omega_{uv} - \sum_{u:z(u)=k} \omega_{uv}$
11: if $g > 0$, $W_j + w_v \le (1+\varepsilon)W/Z$, $W_k - w_v \ge (1-\varepsilon)W/Z$ then
12: move v to zone j
13: end if
14: end for
15: until no move improves the cut
16: return z

TABLE LIII
ILLUSTRATIVE BYTE LAYOUT OF A ZONE SUMMARY (L LANES)

Field	Content	Bytes
ver	Format version	1
zone	Zone identifier	1
epoch, seq	Version (e,q), 2 B each	4
ts	Unix time of the snapshot (s)	4
free	Free bays (excluding held)	2
held	Bays held for reservations	2
flags	Degraded-mode bits	1
nl	Number of lane records L	1
lane×L	id (1), density in 1/255 units (1), flags (1)	$3L$
mac	Truncated message authentication code	8
	Total for $L = 8$	48

every move strictly lowers the cut. After partitioning, gateway positions are chosen so that every bay has a link margin at its assigned SF, and adjacent zones are planned with overlapping coverage, because a neighbor can adopt a failed zone's sensors only if it can hear them.

2) Summary messages and versioning: Fog nodes of neighboring zones exchange a fixed-layout summary every T_s = 10 s over IP. The format is illustrative; its purpose is to show that a zone can be described in a few dozen bytes.

For $L = 8$ the summary is $16 + 24 + 8 = 48$ bytes. Raw reports of 250 sensors, at 12 bytes each, would be 3,000 bytes, so the summary is about $62\times$ smaller, and it does not grow with the number of sensors.

Each summary carries a version (e,q): the epoch e is incremented whenever a different node starts serving the zone, and the sequence number q is incremented at every summary. A receiver keeps for each zone the summary with the lexicographically largest (e,q) and discards older ones, so out-of-order or duplicated delivery is harmless and a node that restarts cannot overwrite newer state with stale state. A summary is also ignored once its age exceeds $3T_s$ (three missed periods), at which point the zone's entry is marked *stale* for routing and booking.

TABLE LIV
SUCCESS PROBABILITY OF A HEARTBEAT AFTER A SYNCHRONIZED RESTART, $N = 250$, SF9 ($T_{\text{air}} = 144.4$ MS), EQ. (34)

Window J (s)	G	p_s (exact)	e^{-2G}
0 (no jitter)	–	≈ 0	–
10	3.61	0.0007	0.0007
60	0.60	0.301	0.300
300	0.12	0.787	0.786
600	0.060	0.887	0.887
900	0.040	0.923	0.923

3) Heartbeat jitter: If many sensors send heartbeats on a common schedule, for example after a power restoration, a gateway reboot, or a time-synchronizing command, their transmissions coincide. Let each sensor delay its next heartbeat by an independent offset $U \sim \text{Uniform}[0,J]$. For two frames of duration T_{air}, the probability that they overlap is $1 - (1 - T_{\text{air}}/J)^2$ when $J \geq T_{\text{air}}$. For N sensors, a given frame survives if none of the other $N - 1$ overlaps it,

$$p_s = \left(1 - \left[1 - (1 - T_{\text{air}}/J)^2\right]\right)^{N-1} \approx e^{-2G}, \quad G = \frac{N\,T_{\text{air}}}{J} \tag{34}$$

the second form being the pure-ALOHA formula of Section IV with the offered load spread over the window J. Without jitter all N frames start together, every pair overlaps, and $p_s \approx 0$ for $N \geq 2$ at a common SF. Requiring $p_s \geq p^{\star}$ gives

$$J \;\geq\; J_{\min} = \frac{2\,N\,T_{\text{air}}}{-\ln p^{\star}}. \tag{35}$$

For a zone of $N = 250$ sensors and 12-byte frames, $T_{\text{air}} = 41.2$ ms at SF7 and 144.4 ms at SF9, and $p^{\star} = 0.9$ gives $J_{\min} = 196$ s at SF7 and 685 s (11.4 min) at SF9. Table LIV shows the effect of several window lengths at SF9. A window of 15 min, equal to the nominal heartbeat period, raises p_s from effectively zero to 0.92. Since $J_{\min}$ must be smaller than T_h, the formula also shows that a 15-minute heartbeat at SF9 leaves little margin; the sensors with good links should use SF7, where $J_{\min}$ is less than a third of that. Jitter is needed only to break synchronization: once heartbeats have freerunning phases, event-driven traffic is approximately Poisson and Eq. (34) is the same ALOHA estimate as before.

4) Failover and handover: Each fog node monitors its neighbors through the summaries. The protocol of Fig. 11 has four states per monitored zone. In NORMAL, summaries arrive each T_s. After $k = 3$ consecutive missing summaries (30 s) the node enters SUSPECTED. A fresh summary returns it to NORMAL. If none arrives within a further confirmation time $T_c = 30$ s, the node requests a *lease* for the zone from the cloud; if the cloud is unreachable, the takeover rule falls back to a fixed priority list, and the node with the lowest identifier among the live neighbors takes over, which avoids two nodes claiming the zone. The node then enters TAKEOVER, increments the epoch e, loads the zone's last replicated state, and starts processing frames from the zone's sensors. With the proposed timers a failure is acted upon

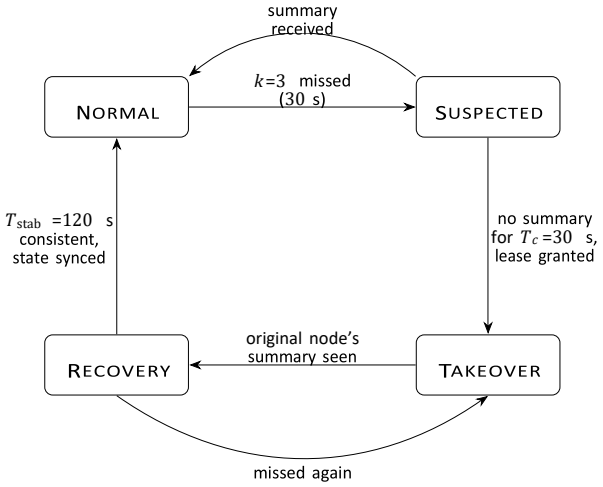


Fig. 11. Failover state machine for one monitored zone, with the proposed timer values.

after at most $3T_s + T_c = 60$ s plus the state-loading time. During TAKEOVER the node also watches for summaries from the original node; when it sees one, it enters RECOVERY, performs a state synchronization, and returns the zone only after $T_{\text{stab}} = 120$ s of consistent summaries, so that a flapping node cannot cause repeated handovers. A relapse during recovery returns to TAKEOVER.

5) What is stored where: Raw sensor frames and per-bay state are kept only at the zone's fog node, and are replicated to one designated neighbor at each summary period in compact form (the occupancy bitmap of the zone's bays, 250 bits or about 32 bytes, plus outstanding reservations), which is what a successor loads. The cloud stores the latest summary per zone, the reservation ledger, accounts, price history, and the model registry. Neighbors store the other zones' summaries only, never their raw frames. A cloud outage therefore stops cross-lot views and new account operations but not the local bay selection in a zone, and a fog outage costs at most one handover delay.

H. Design Detail: Camera-Assisted Congestion Estimation

1) Counting definitions: Two ways of turning detections into counts are common, and we use both. A *counting line* is a virtual segment across a lane; a tracked vehicle that crosses it increments an entry or exit counter E_ℓ or X_ℓ for lane ℓ. It needs tracking across frames, but it is robust to partial occlusion because only the crossing matters. A *counting zone* is a polygon covering a lane stretch of length L_ℓ; the count $n_\ell(t)$ is the number of detected vehicles whose reference point lies inside it at time t. It needs no tracking but is sensitive to occlusion and to queues that extend beyond the polygon. A detector of the single-pass kind [53] provides the boxes, and a lightweight tracker associates them over frames. The lane capacity is $N_\ell = \lfloor L_\ell/\ell_v \rfloor$, where ℓ_v is the space per queued vehicle including the gap. With an assumed $\ell_v = 7.5$ m and $L_\ell = 180$ m, $N_\ell = 24$. The raw density is $\rho^{\text{raw}}{}_\ell(t) = \min\{1, n_\ell(t)/N_\ell\}$, which is the congestion c_e used by the routing model of Section III.

2) Smoothing and hysteresis: Detections fluctuate from frame to frame, and a routing cost that jumps on every frame would make recommendations unstable. We smooth with an exponential moving average (EMA) sampled every $\Delta = 5$ s:

$$\hat{\rho}_\ell(t) = \alpha \rho^{\text{raw}}{}_\ell(t) + (1-\alpha)\hat{\rho}_\ell(t-\Delta),\ 0 < \alpha \le 1. \quad (36)$$

The effective time constant is $\tau_{\text{ema}} = -\Delta/\ln(1-\alpha)$, which is 14.0 s for $\alpha = 0.3$. Starting from $\hat{\rho} = 0.5$ and a step to $\rho^{\text{raw}} = 0.9$, the estimate is 0.62, 0.704, 0.763, and 0.804 after four samples. Discrete congestion levels, for example *free*, *busy*, and *congested*, use two thresholds each, an upward threshold $\theta^{\uparrow}$ and a lower downward threshold $\theta^{\downarrow} = \theta^{\uparrow} - h$ with band h. With $\theta^{\uparrow} = 0.75$ and $h = 0.10$ for *congested*, the example crosses the threshold at the third sample (15 s after the step). If the raw value then falls to 0.3, the estimate goes to 0.653 and then 0.547; the state stays *congested* after the first sample, since $0.653 > 0.65$, and leaves it at the second. A single threshold at 0.75 would flip on any noise around that value, while the band tolerates noise of up to $\pm h/2$. The values $\alpha = 0.3$, $h = 0.10$, and the thresholds are proposals; the audit in Section V-J would tune them.

3) Counter proxy and fusion: When the camera is unavailable we estimate the vehicle count in a lane group from the entry and exit counters. Conservation gives, from a reference count $n(t_0)$ taken when the camera last worked or the area was observed empty,

$$\hat{n}_{\text{p}}(t) = n(t_0) + E(t_0,t) - X(t_0,t),$$
$$\sigma^2_{\text{p}}(t) = \sigma_0^2 + \varepsilon(E + X). \quad (37)$$

The variance uses the same independent-miss assumption as the conservation check of the preceding security analysis: each crossing is missed with probability ε, so errors accumulate with the number of events, and σ_0^2 is the variance of the reference count. The proxy drifts and must be re-anchored. The proxy for a group with no counters is the number of occupied bays nearby, scaled by a ratio fitted from the audit; we mention it but do not model it.

When both estimates are available, they are combined by inverse-variance weighting. For independent unbiased estimates x_1, x_2 with variances σ_1^2, σ_2^2, the linear combination $\hat{x} = wx_1 + (1-w)x_2$ has variance $w^2\sigma_1^2 + (1-w)^2\sigma_2^2$; setting the derivative with respect to w to zero gives

$$w = \frac{1/\sigma_1^2}{1/\sigma_1^2 + 1/\sigma_2^2}, \qquad \sigma^2_{\hat{x}} = \left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right)^{-1} \le \min(\sigma_1^2, \sigma_2^2). \quad (38)$$

Example. The camera counts $x_1 = 18$ vehicles with $\sigma_1^2 = 4$ (taken from the audit error, an assumption), and the proxy gives $x_2 = 22$ with $\sigma_0^2 = 9$, $\varepsilon = 0.02$, $E = 40$, and $X = 30$, so $\sigma_2^2 = 9 + 0.02 \cdot 70 = 10.4$. Then $w = 0.722$, $\hat{x} = 19.1$ and $\sigma^2_{\hat{x}} = 2.89$ ($\sigma = 1.70$, against 2 for the camera alone). With $N_\ell = 24$ the density is 0.796 against 0.75 from the camera alone, which lies above the *congested* threshold of 0.75 that the camera alone only reaches. Eq. (38) is optimal only for unbiased and independent errors; a camera with a systematic undercount at night is biased, and the variances must then come from audits per lighting condition, not from one pooled value.

TABLE LV PROPOSED RETENTION FOR CAMERA-DERIVED DATA

Data	Content	Retention
Frames, crops	Not stored; in memory only	< 1 s
Lane counts	n_ℓ, E_ℓ, X_ℓ every 5 s, flags	30 days at fog
Lane aggregates	5-min means, per-lane, no identifiers	13 months at cloud
Audit sample	Manual counts, no images	Study duration

4) Degraded-mode flags: Each lane estimate carries a flag byte, copied into the summary (Table LIII): CAM_OK, CAM_OBSTRUCTED (the detector sees no change in image content or a sharply falling number of detections for longer than a set time), PROXY_ONLY, FUSED, STALE (no update for $> 3T_s$), and RESET (the proxy was re-anchored). Routing uses the fused or proxy value but widens the congestion term when

TABLE LVI
PROPOSED FORECAST FEATURES AND THEIR AVAILABILITY AT PREDICTION TIME t

Feature	Definition	Source	Availability at t
occ_lag_k	Occupied bays in the zone at $t - k \cdot 5$ min, $k = 0,\ldots,12$	Fog bay state	Yes: past values only
in_15, out_15	Gate entries and exits in $(t - 15$ min$,t]$	Gate counters	Yes; delayed counters are excluded
hod, dow	Hour of day and day of week, as sine/cosine pairs and an index	Clock	Yes: known in advance
hijri_month, holiday	Hijri calendar month; public or religious holiday flag	Calendar table	Yes, if the table is fixed before t
event_flag, event_size	Scheduled event in the catchment area; expected attendance class	Operator entry	Only entries published before t
res_due_h	Accepted reservations that start within h of t	Reservation ledger	Yes: ledger state as of t
cong_ema	Smoothed lane density at the zone entrance, Eq. (36)	Camera/proxy	Yes; carries a quality flag
weather_fc	Forecast temperature and rain for the target hour	External service	Forecast issued at or before t; not the observation
qflags	Share of sensors online; degraded-mode bits	Fog health	Yes

flagged: the edge cost is evaluated at $\hat{\rho} + z\sigma$ for a proposed $z = 1$, which makes a route over an uncertain lane slightly less attractive. The modes are logged, so the share of time in each can be reported.

5) Data minimization and retention: Only counts leave the camera pipeline. Frames are processed in memory and overwritten; no frame, crop, or plate is written to disk. The detector class is restricted to vehicle categories, and pedestrians are not counted. Table LV lists the proposed retention. Retention periods are proposals, to be fixed with the operator and reviewed against the applicable data-protection law [66], [67].

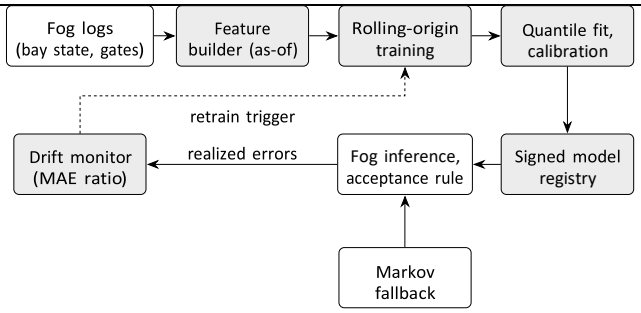


Fig. 12. Proposed forecasting pipeline. Shaded boxes run in the cloud, unshaded boxes at the fog node.

I. Design Detail: Learning-Based Forecasting Pipeline

1) Features and leakage control: A forecast issued at time t for horizon h may use only information that exists at t. Table LVI lists the proposed features. The commonest leakage in practice is not the target itself but covariates joined by their *event time* instead of their *as-of time*: an event flag that the operator entered after the event began, or the observed weather of the target hour instead of the forecast that was available at t. Features known in advance (calendar, announced events) are therefore joined as of the issue time and stored with their publication time.

2) Training and deployment pipeline: Fig. 12 shows the data flow. Fog nodes upload logs to the cloud; a feature builder performs the as-of joins; models are trained and calibrated in the cloud, signed, and published to a registry; fog nodes download a model, run inference locally on features computed from live data, and fall back to the Markov baseline of Section III when no valid model is present. Both candidate models of the paper, the gradient-boosted trees [73] and the LSTM [74], are trained per horizon or with horizon as an input, and the Markov and persistence baselines are run in parallel so that every deployment decision is logged against them.

3) Rolling-origin retraining and drift trigger: Evaluation and retraining both follow the rolling-origin principle: the model is always fitted on data before the forecast origin and tested on data after it [46], [71]. For monitoring, define for a window w (one day) the mean absolute error $\mathrm{MAE}_w(h)$ of the deployed model at horizon h against realized occupancy, and the drift statistic

$$D_w(h) = \frac{\mathrm{MAE}_w(h)}{\mathrm{MAE}_{\mathrm{ref}}(h)}, \tag{39}$$

where $\mathrm{MAE}_{\mathrm{ref}}$ is the error on the held-out period at deployment time. A retraining is triggered when $D_w > \theta$ for m consecutive windows; we propose $\theta = 1.3$ and $m = 3$ as placeholders. If exceedances were independent with probability $q = 0.05$ per window under no drift, a false trigger would start a run with probability $q^m = 1.25 \times 10^{-4}$; real errors are autocorrelated, so the actual rate is higher and must be measured. Algorithm 3 states the loop.

The shadow period and the promotion test (lines 9–12) stop a retraining on corrupted data from replacing a good model, which matters because the integrity attacks of the preceding security analysis target exactly this loop.

Algorithm 3 Rolling-Origin Retraining with Drift Trigger

Require: data stream, window W, calibration length C, θ, m
1: train model M_0 on the first W days; set $\mathrm{MAE}_{\mathrm{ref}}$ on the last C days
2: $c \leftarrow 0$
3: for each day w do
4: compute MAE_w for M and for the Markov baseline
5: if $\mathrm{MAE}_w/\mathrm{MAE}_{\mathrm{ref}} > \theta$ then $c \leftarrow c + 1$
6: else $c \leftarrow 0$
7: end if
8: if $c \geq m$ or scheduled retraining is due then
9: fit candidate M' on the latest W days; calibrate on the last C
10: run M' in shadow mode for K days
11: if $\mathrm{MAE}(M') \leq \mathrm{MAE}(M)$ and $\leq$ baseline then
12: sign and publish M'; $M \leftarrow M'$; reset $\mathrm{MAE}_{\mathrm{ref}}$; $c \leftarrow 0$
13: end if
14: end if
15: end for

4) Prediction intervals and the acceptance rule: We state one method as primary. Quantile regression [47] trains one model per level q on the pinball loss $\rho_q(y,\hat{y}) = \max\{q\,(y-\hat{y}),\,(q-1)(y-\hat{y})\}$, (40)

whose minimizer is the q-quantile of the conditional distribution. Gradient-boosted trees support this loss directly and an LSTM can have one output head per quantile. As a secondary step, split conformal prediction [48] adjusts the quantile: on n recent calibration residuals, the correction is the $\lceil (n+1)(1-\delta) \rceil$-th smallest score, which is the 181st of $n = 200$ scores for $\delta = 0.1$. Because the exchangeability assumption fails for time series, the calibration set is a rolling window and the realized coverage is monitored alongside the drift statistic.

Let A_{t+h} be the number of free bays in the zone at the arrival time, $R_{t,h}$ the accepted reservations that will occupy bays at that time, and $p^{\square} = c_{FP}/(c_{FP}+c_{FN})$ the cost-based threshold of Section IV-J. A booking is accepted if

$$Q_{1-p^{\star}}\big(A_{t+h}\big) - R_{t,h} \;\geq\; 1 + s, \tag{41}$$

where Q_α is the predicted α-quantile of free bays and $s \geq 0$ is a safety margin in bays, set from the conformal correction and from the share of degraded sensors. With the illustrative costs of that subsection ($c_{FP} = 5$, $c_{FN} = 1$), $p^{\square} = 5/6$ and the lower quantile level is $1 - p^{\square} = 1/6$. If the predicted 1/6-quantile is 14 free bays, $R = 9$ and $s = 2$, then $14 - 9 = 5 \geq 3$ and the booking is accepted; with $R = 12$ it is rejected. Eq. (41) replaces the state-only rule of the reference description by one that counts outstanding reservations, and it reduces to a single-threshold rule on a bound whose meaning ($1-p^{\square}$ of the probability mass lies below it) is fixed by costs.

5) Edge deployment and fallback: The tree ensembles are small enough to run on a single-board computer, because inference is a sequence of comparisons and additions, while LSTM inference costs matrix products per time step but remains light for the input length of 13 lags. The cost is therefore modest for both, and the dominant risks are operational: model file integrity, version mismatch between model and feature code, and missing inputs. Models are signed, the feature code and model carry a joint version number, and the fog node checks the version, feature completeness, and the qflags feature before running inference. If the model is absent or invalid, or if an input is missing, the node uses the Markov baseline, widens the safety margin s, and raises a degraded-mode flag in the summary. Quantized or distilled models may be considered if measured latency requires it; we make no claim about their accuracy.

J. Design Detail: Staged Field Validation

1) Stage gates: Each stage ends with a gate. Table LVII gives entry and exit criteria. All thresholds are *proposed placeholders*, to be fixed, together with the statistical tests that decide them, before the study starts and not after the data are seen.

2) Pilot design: The pilot instruments one zone of about 250 bays, matching the zone size derived above. The zone is chosen for three properties: it includes both covered and open bays, it has a junction with a camera line of sight over at least two lanes, and it lies in the overlapping coverage of two planned gateways so that failover can be exercised on purpose. We propose six weeks, so that weekday and weekend cycles repeat several times, plus a short run-in period for installation checks that is excluded from the analysis. Operation is supervised: staff can switch the zone back to the reference behavior at any time.

Audit accuracy. A manual audit compares the system's bay state against a human observation at the same moment. If $\hat{p}$ is the proportion of bays the system reports correctly, the number of independent audit observations needed for a confidence half-width e at confidence z is

$$n = \frac{z^2\,p(1-p)}{e^2}, \qquad z = 1.96 \text{ for } 95\%. \tag{42}$$

For $p = 0.90$ and $e = 0.05$, $n = 138.3$, so 139 observations; for $p = 0.95$ and $e = 0.03$, $n = 203$; for the worst case $p = 0.5$ and $e = 0.05$, $n = 385$; and for $p = 0.95$ and $e = 0.02$, $n = 457$. A single snapshot of all 250 bays is a finite population, and the finite-population correction Algorithm 4 Counter-Acceptance Window per Sender

Require: frame with counter c and MAC; state f (highest accepted), bitmap B of W bits; limit G

```
1: if MAC invalid for c then count MAC failure; return reject
 2: if c > f then
3:         if c − f > G then return reject                    ▯ gap too large
4:     end if
5:         B ← B ≪ (c − f); set bit 0; f ← c; return accept
 6: else if f − c < W then
7:         if bit (f − c) of B is set then return reject      ▯ replay
8:     end if
9:         set bit (f − c); return accept                     ▯ late frame
 10: else
11:     return reject                                         ▯ too old
 12: end if
```

in use, a named contact for questions, and a dataprotection assessment against the applicable law [66], [67]; where required, an institutional ethics review. Safety rules are simple: no installation at height without trained staff, no change to marked emergency or access bays, and an immediate rollback if drivers are misdirected. Failures and rollbacks are reported with the results, not omitted.

K. Design Detail: Layered Security and Privacy

1) Key hierarchy: Table LVIII proposes the keys, where each lives, and how it changes. LoRaWAN already separates a per-device root key from per-session keys [34]; we add fog and cloud credentials and explicit revocation.

2) Authentication and replay window: Uplinks use a counter that must strictly increase, as in LoRaWAN [34]; the check is done at the network server. Messages that reach the fog and cloud afterward (fog–cloud messages, summaries, and frames delivered by more than one gateway) can be reordered or duplicated, so we use a sliding window of $W = 64$ counters per sender (Algorithm 4). It accepts late but unseen counters, rejects exact repeats, and rejects counters too old or too far ahead.

The MAC is checked first so that unauthenticated frames cannot move the window. G bounds the damage of a captured

TABLE LVII
PROPOSED STAGE GATES (ALL THRESHOLDS ARE PLACEHOLDERS TO BE FIXED BEFORE THE STUDY)

Stage	Entry criteria	Exit criteria
S1: Full-size simulation (2,500 bays)	Reference results reproduced at 150 bays within the 95% confidence interval; scenarios and seeds fixed	≥ 30 seeds per cell; improvement over nearest-free-bay guidance with the 95% CI of the difference excluding zero; no unexplained failures in outage scenarios; 95% CI half-width ≤ 1% of the mean for arrival time
S2: Bench test	S1 passed; firmware and fog software frozen; fault-injection plan written	Report delivery ratio ≥ 0.95 at 10% injected loss; failover recovery ≤ 90 s; 100% of injected replayed frames rejected; sustained zone load at 250 sensors for 24 h without fog overload
S3: Supervised pilot	S2 passed; operator permission, signage, and data-protection review complete; audit protocol approved	Bay-state accuracy with lower 95% bound ≥ 0.90 in each stratum; "free" errors (a bay reported free that is occupied) upper bound ≤ 1%; camera count MAPE ≤ 10%; uptime ≥ 95%; share of accepted bookings that find no space ≤ 2%

TABLE LVIII
PROPOSED KEY HIERARCHY AND LIFECYCLE

Key	Held by	Provisioning, rotation, revocation
Device root key (128-bit)	Sensor secure element [15]; network server	Unique per device, injected at manufacture or commissioning; replaced

$n/(1 + (n - 1)/250)$ lowers 139 to about 90, but snapshots at different times are correlated and bays in a row share conditions, so we keep 139 as a conservative per-stratum target. With three strata (peak, off-peak, night), this is 417 observations, or $\lceil 139/12 \rceil = 12$ bays per stratum in each of 12 audit sessions (two per week over six weeks), 432 observations in all. For the safety-critical "free" error, an audit with no failures in n trials supports an upper bound of $1 - 0.05^{1/n} \approx 3/n$ at 95%; to show ≤ 1% requires $n \geq \ln 0.05/\ln 0.99 = 298.1$, i.e., 299 audits of bays reported free with zero errors.

3) Data management and ethics: Pilot data are stored under pseudonymous identifiers; the mapping from account to pseudonym lives in the cloud and is not exported to analysts. Raw frames are not recorded, even for debugging. Audit sheets record bay identifier, time, observed state, and system state, and no personal data. The study needs the operator's written permission, visible signs that sensing and counting are

[1] *) Reservation safeguards:* A token bucket limits the request rate of each account, device, and network address (Algorithm 6). With capacity b and refill rate r, the number of requests that can be accepted in any interval of length t is at most $b+rt$. For $b = 3$ and $r = 1$ per hour, an account can place at most 4 requests in its first hour and 3 + 24 = 27 per day, which is generous for a legitimate driver and tiny for a flood. Account limits alone are weak against many accounts, so we add a cap of $c_h = 2$ concurrent unpaid holds per account with a hold time of 10 min (placeholder), a small deposit that is refunded on arrival, and a global per-zone cap on held bays. An attacker who wants to hold 5% of a 250-bay zone ($\lceil 12.5 \rceil = 13$ bays) needs at least $\lceil 13/2 \rceil = 7$ accounts, and

		at service or after suspected capture; revoked by deletion at the server
Session keys	Sensor RAM; network server	Derived at each join; rejoin at least every 30 days (placeholder) or when the counter passes a set fraction of its range
Fog identity	Fog node	Certificate issued by an operator CA at commissioning; short lifetime (placeholder: 90 days); revocation list pushed to peers
TLS session keys	Fog, cloud	Ephemeral per connection, TLS 1.3 [80]; no early data
Summary MAC key	Neighbor fog nodes	Pairwise, derived from the identity keys; rotated with the certificates
Payment keys	Payment gateway only	Never held by parking servers

future counter (we propose $G = 2^{14}$), and the rate of MAC failures per device is itself a signal: a sustained failure rate is the footprint of forgery attempts, whose per-frame success probability is 2^{-32} for a 32-bit tag. The 16-bit on-air counter is extended to 32 bits at the server by tracking the high half; the wrap-around risk is discussed in Section IV-L5.

3) Plausibility checks: Algorithm 5 applies cheap consistency checks to each report batch. C1 compares independent sensors of the same bay. C2 is the conservation law $D = o\hat{} - o_0 - E + X$ of the security analysis in Section IV: with the counting-noise model used there ($E = X = 100$ per hour, $\varepsilon = 0.01$), a threshold $\tau = 3$ gives a false-alarm probability of 0.0169 per hourly window, recomputed here from the Skellam distribution, which corresponds to roughly 0.4 false alarms per day per zone. C3 and C4 bound the rate and the dwell time of reported changes. All thresholds are proposals.

4) Alert severity and response: Table LIX gives the proposed levels. The reference workflow lists security alerting without defining it [2]; here an alert is a typed event with a response. The booking-related responses connect to the forecasting design: a higher level raises the safety margin s of Eq. (41).

Algorithm 5 Plausibility Checks on a Report Batch

Require: bay reports, second-sensor readings, $o_0, E, X, \tau, f_{\max}, d_{\min}$
$S \leftarrow \emptyset$
for each bay b do
  if sensors A and B disagree in 2 consecutive samples then add (b, C1) to S
  end if
  if b changed more than $f_{\max}$ times in 1 h then add (b, C3) to S
  end if
  if b re-occupied within $d_{\min}$ of its last change then add (b, C4) to S
  end if
end for
$D \leftarrow \sum_b o\hat{}_b - o_0 - E + X$
if $|D| > \tau$ then raise zone alert (C2)
end if
quarantine bays in S: use the second sensor or the last good state
return alerts with severity from Table LIX

TABLE LIX
PROPOSED ALERT SEVERITY LEVELS AND RESPONSES

Level	Trigger (examples)	Response
L1	Single-bay C1, C3, or C4 hit	Quarantine bay report; log
L2	Zone C2 alert; MAC failures above a set rate for one device	Raise s by a set amount; flag zone in the summary; notify operator in the next report
L3	Several devices with replay or MAC failures; large share of bays quarantined	Stop automatic booking in the zone; use the conservative fallback; page the operator
L4	Suspected key compromise, rogue gateway, or payment anomaly	Revoke keys, isolate the node, require manual re-commissioning

Algorithm 6 Token Bucket for Reservation Requests

Require: key k (account, device, or address), time t, cost κ; state (x_k, t_k); capacity b, rate r
$x_k \leftarrow \min\{b, x_k + r(t - t_k)\}$; $t_k \leftarrow t$
if $x_k \geq \kappa$ then
  $x_k \leftarrow x_k - \kappa$; return accept
else
  return reject with retry after $(\kappa - x_k)/r$
end if

TABLE LX PROPOSED RETENTION SCHEDULE (PLACEHOLDERS)

Data	Retention	Treatment afterward
Raw radio frames	7 days	Deleted
Bay events (no user link)	13 months	Aggregated to hourly counts
Reservation records	90 days	Pseudonym removed; totals kept
Position during a trip	Trip end + 24 h	Deleted
Security logs	180 days	Deleted, unless an incident is open
Payment records	Gateway	Held by the gateway, not by the system

each account must pass registration and the payment step.

6) Privacy retention, transport security, and payment separation: Table LX lists proposed retention for non-camera data. Fog–cloud traffic uses mutually authenticated TLS 1.3 [80], with early data disabled because replay protection does not cover it. The payment path is kept apart from the parking servers: the app sends card details directly to a third-party gateway [9], the gateway returns a signed result containing an opaque token, amount, and status, and the parking cloud stores only that. The parking system therefore never sees card numbers, and a compromise of it cannot leak them.

L. Design Detail: Demand-Calibrated Pricing

1) Occupancy-aware multiplier: For short stays, the base price $r_0\,(1-\delta(h))$ of the reference ladder is multiplied by

$$m(\hat{\rho}) = \text{clip}\,\big(1+\gamma(\hat{\rho}-\rho_{\text{target}}),\ m_{\min},\ m_{\max}\big), \tag{43}$$

where $\hat{\rho}$ is the forecast occupancy at the start of the stay, ρ_{target} the occupancy at which the lot should ideally operate (a value such as 0.8 leaves some bays free for newcomers), and $\gamma > 0$ the sensitivity. With the placeholders $\gamma = 2$, $\rho_{\text{target}} = 0.8$, $m_{\min} = 0.8$, and $m_{\max} = 1.5$, and $r_0 = 2.00$ per hour (an arbitrary unit), the multiplier is 0.8 for $\hat{\rho} \le 0.70$ (the unclipped values at 0.5 and 0.6 are 0.4 and 0.6), 1.0 at 0.8, 1.2 at 0.9, 1.3 at 0.95, and 1.4 at 1.0, giving prices 1.60, 2.00, 2.40, 2.60, and 2.80.

The formula hides a feedback loop that a designer must face. The forecast $\hat{\rho}$ is made from data that were themselves produced under earlier prices. Suppose demand responds as $\rho = \rho_0\, m^{\epsilon}$ with $\rho_0 = 0.95$ and $\epsilon = -0.5$ (illustrative). If the rule uses the price-neutral forecast ρ_0, then $m = 1+2(0.15) = 1.30$ and occupancy settles at $0.95 \times 1.3^{-0.5} = 0.833$. If instead it uses the observed occupancy, the fixed point solves $\rho = 0.95 m^{-0.5}$ with $m = 1 + 2(\rho - 0.8)$, which gives

Algorithm 7 Pricing Engine

Require: $\hat{\rho}$, validity flag, r_0, $\delta(\cdot)$, γ, ρ_{target}, $m_{\min}$, $m_{\max}$, step cap Δ_m, previous m_{prev}; bids B, spaces k, reserve b_0

1: if forecast invalid or zone degraded then $m \leftarrow 1$
2: else
3: $m \leftarrow \text{clip}\,\big(1+\gamma(\hat{\rho}-\rho_{\text{target}}),\ m_{\min},\ m_{\max}\big)$
4: $m \leftarrow \text{clip}(m,\ m_{\text{prev}}-\Delta_m,\ m_{\text{prev}}+\Delta_m)$
5: end if
6: quote $\leftarrow r_0\, m(1-\delta(h))$ for stay length h; lock until arrival
7: drop bids below b_0; sort remaining in decreasing order
8: allocate $\le k$ places by the configured bid rule
9: return quotes, allocation, m

$\rho = 0.881$ and $m = 1.162$: the price is lower because the surcharge itself removes part of the signal. The same occupancy target would require $m = (0.8/0.95)^{-2} = 1.41$ at this elasticity. The choice of γ therefore depends on the elasticity and on whether the input is the price-neutral or the price-affected forecast; we propose a price-neutral input (features without the current price, or the price as an explicit feature set to 1), which makes γ interpretable.

2) The pricing engine: Algorithm 7 combines the forecast state, the reference ladder, the bids, and the caps. Short-stay quotes come from Eq. (43); the multiplier is limited in how fast it can change; the fallback is the reference ladder when the forecast is invalid or the zone is in a degraded mode. Long-term places use the bid rule chosen for the study.

Fairness caps. The proposals are: the bounds $m_{\min}$, $m_{\max}$; a limit on the change per update, $\Delta_m = 0.05$ per 15 min (placeholder), so that a price never jumps; the quote locked at the time of booking and valid until the arrival time; displayed prices shown before the user commits; and a fixed share of bays outside the multiplier (for example accessible bays and those for long-term subscribers).

3) Bid collection and sealed-bid variant: The reference rule as restated in Section V sets the price at the j-th highest bid that maximizes revenue, which gives bidders a reason to shade their bids. A sealed-bid variant instead uses the highest *rejected* bid as the uniform price. For unit-demand bidders and identical places, this extends the second-price idea of Vickrey [50] and makes bidding one's true value a dominant strategy [51]. Using the example bids of the reference paper (20 bids of 200, three of 150, seven of 90), the revenuemaximizing rule with $k = 22$ spaces earns $20\times200 = 4{,}000$, the sealed-bid rule prices at the 23rd bid (150) and earns $22 \times 150 = 3{,}300$; for $k = 25$ the figures are $4{,}000$ and $25\times90 = 2{,}250$; for $k = 10$ both give $2{,}000$. The comparison takes the stated bids as true values, which is exactly what the first rule cannot guarantee, so the gap is the price of truthfulness and not a forecast of revenue loss. The protocol is: (1) the cloud opens a window with a published reserve and number of places; (2) each bidder authenticates and submits one sealed bid, with a refundable deposit; (3) bids are encrypted or committed so the operator cannot see them before closing; (4) the window closes at a fixed time; (5) the allocation and the uniform price are computed and published, with ties broken by commit time; (6) winners pay through the payment gateway and unsold places move to the short-term pool.

4) Calibration workflow: We propose five steps. (1) Collect occupancy, price, and booking logs per zone and time block. (2) Estimate a price elasticity ϵ and its interval from a log– log regression of demand on price with controls for hour, weekday, and event days, using price variation that is not driven by occupancy itself (planned step changes in alternating time blocks). (3) Run a stated-preference survey to estimate the distribution of willingness to pay. (4) Fit a demand curve and choose γ, ρ_{target}, and the bounds by simulation so that predicted occupancy stays within a band around the target. (5) Test the setting in the pilot and re-estimate. The survey is a choice experiment: each respondent sees eight choice sets with a price level (four levels), a walking distance, and whether the bay is guaranteed, and picks the preferred option or "none". The attribute combinations come from a fractional factorial design. Because a share near 0.5 is the worst case, estimating a segment share to within ±5 percentage points at 95% needs $n = 385$ respondents per segment, by the same formula as Eq. (42). Stated choices tend to overstate willingness to pay, so the survey is used for the shape of the distribution and the logs for its level.

M. Integration, Degradation, and Roadmap

1) End-to-end booking sequence: Fig. 13 traces one booking through the components. The cloud rate-limits and authenticates the request and forwards it to the fog node of the requested zone, which applies the acceptance rule of Eq. (41) using its own bay state, camera-derived congestion, and plausibility flags. The pricing engine then returns a quote. Payment goes from the app straight to the gateway; the cloud receives only the signed result and confirms the hold at the fog node. At arrival, the fog node computes the route with the camera-based congestion values.

2) Failure and degradation matrix: Table LXI lists singlecomponent failures, how the system should behave, and what a driver sees. The principle is that failures degrade service in steps, towards the reference behavior, and never silently.

3) Implementation roadmap: Table LXII orders the work packages by dependency only; we give neither dates nor costs, because they depend on a team and a site we cannot specify.

VI. Evaluation Plan

This section specifies how the improvements of Section V are to be tested in a follow-up study. No experimental results are claimed in the present paper; the plan is written so that another group could carry it out and obtain comparable numbers.

A. Metrics and Comparators

Table LXIII lists, for each improvement, the quantities to be measured and the comparator against which the improvement is judged. The two metrics of the reference paper, arrival time and searching time [2], are retained so that results remain comparable, and are complemented by measures of accuracy, robustness, and cost.

B. Scenarios

Each configuration is run at two lot sizes: the 150-space lot of the reference experiments, which allows a like-for-like reproduction, and a lot of about 2,500 spaces matching the real case study. Six scenarios are defined: (i) a normal weekday; (ii) an event surge with arrival rates several times the normal level; (iii) partial sensor outage; (iv) failure of one fog node; (v) degraded sensing, for example a blocked camera or noisy magnetic readings; and (vi) injected attacks, namely spoofed and replayed occupancy reports and a burst of reservation requests.

C. Protocol

The study starts by reproducing the reference results with the published parameters (150 spaces, 10 arrivals and 5 departures per minute, and a maximum initial availability of 90%) [2], as a sanity check on the simulation setup. Improvements are then enabled one at a time and finally together, so that the contribution of each can be separated in an ablation. Every configuration is repeated with at least 30 random seeds, and results are reported as means with 95% confidence intervals and compared pairwise. Forecasting models are trained and tested on chronologically separated data to avoid leakage, using logged lot data where available and open occupancy datasets of the kind used in earlier studies [12] otherwise.

D. Ethical Considerations and Validity

The pilot stage involves real drivers, so it requires the operator's permission, visible notices that sensing is in use, and data handling consistent with the privacy measures of Section V: pseudonymous identifiers, no stored video, and limited retention of location traces. Several threats to validity remain. Simulated traffic may differ from real driver behavior, open datasets may not represent the target city, and camera performance can vary with site geometry and lighting. Results from one lot should therefore be generalized cautiously, and a second site is desirable if resources permit.

The four subsections above state what is to be measured and against what. This part supplies the operational detail that a second group would need: the simulation model, exact metric definitions, the experimental design and its run count, the statistical analysis, the forecasting protocol, the pilot protocol, the threats to validity, and the work plan. Everything here is a plan. No run has been made, and every number that looks like a result is either taken from the reference paper [2] or is an illustrative value of ours, labeled as such.

E. Simulation Model Specification

1) Network and vehicles: The 150-space reference lot is rebuilt in SUMO [7], [40] as a directed graph of junction nodes and lane edges, with one entrance, one exit, and bay groups attached to aisle edges as SUMO parking areas. The 2,500-space lot is built as Z zones of roughly equal capacity, each with its own aisle network and gateway position, joined by shared access roads. A Python layer attached through

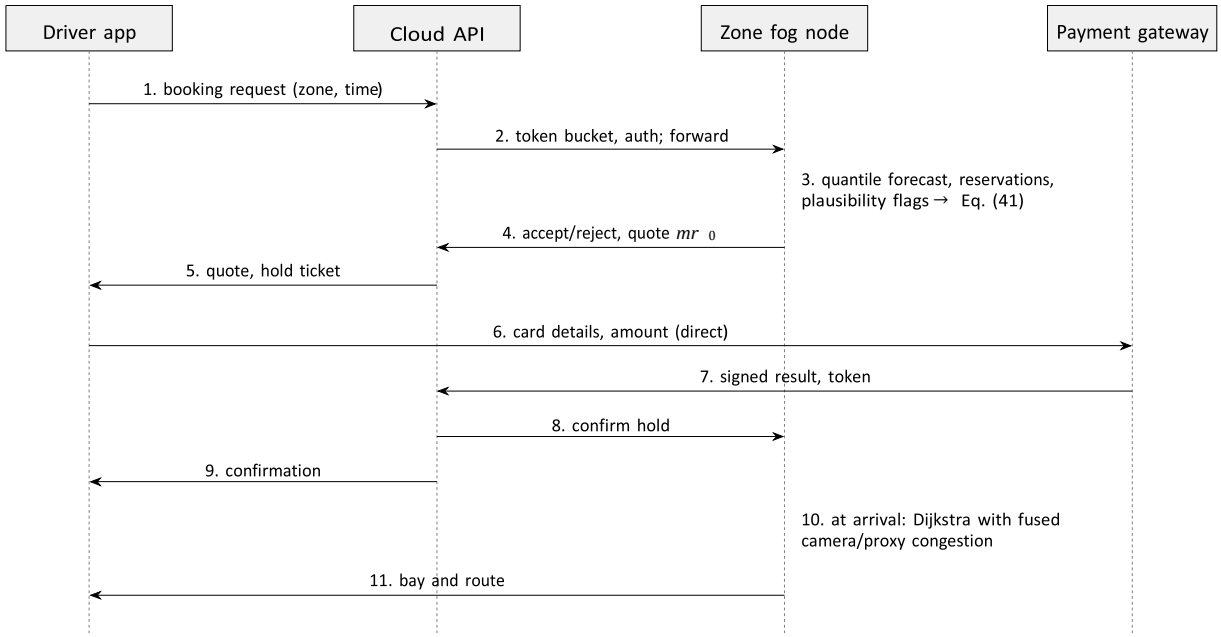


Fig. 13. Message sequence for one booking, from request to routing at arrival.

TABLE LXI
PROPOSED FAILURE AND DEGRADATION MATRIX

Component failure	System behavior	User impact
Camera blocked or off	Counter proxy; PROXY_ONLY; route cost uses $\rho\hat{}+z\sigma$	Slightly less precise routes
Bay sensor dead or quarantined	Use second sensor, else last good state; reduce the bay's weight in the forecast; alert L1	None, or rare wrong bay guidance
Gateway down	Neighbor with overlapping coverage adopts the sensors; otherwise zone marked STALE	Bookings in the zone may pause
Fog node down	Failover (Fig. 11); epoch increment; handback after stability	Up to about 60 s of stale data plus load time
Fog–cloud link down	Local bay selection continues; bookings from cached ledger only	No new accounts or payments
Cloud down	Fog nodes work on cached state; no cross-lot view	Walk-in guidance only
Forecast model unavailable	Markov baseline; larger margin s	More rejected bookings
Payment gateway down	Booking held as unpaid until timeout; no card data on parking servers	Cannot confirm booking

TABLE LXII
WORK PACKAGES AND DEPENDENCIES

ID	Work package	Depends on
WP1	Simulation model at 2,500 bays, baselines, scenarios	–
WP2	Zone partitioning, summaries, jitter, radio plan	WP1
WP3	Failover protocol	WP2
WP4	Camera counting, proxy, fusion	WP1
WP5	Feature builder and forecasting pipeline	WP1
WP6	Security stack: keys, replay window, plausibility, alerts	WP2
WP7	Pricing engine and calibration survey	WP5
WP8	Integration and bench test (S2)	WP3–WP7
WP9	Pilot (S3) and analysis	WP8

SUMO's TraCI interface holds everything SUMO does not model: bay state, the sensing and radio layers, the fog nodes, the forecaster, and the pricing engine. At every decision point (entrance, re-route, reservation request) the layer queries the fog model, which answers from its *view* of the lot, not from ground truth. That separation is what lets stale reports, lost frames, and wrong camera counts degrade the decisions.

Vehicles are of three classes: passenger cars, coaches, and two-wheelers. The base configuration uses a single passengercar type, so that the reproduction run is not confounded. Mixed traffic then enters as a sensitivity factor. Classes differ in length, acceleration, and minimum gap, in the bay size they need (capacity is counted in car-equivalent spaces), and in lateral behavior. SUMO's default lane-following model assumes lane discipline, which is weak in heterogeneous traffic; its sublane model with a finite lateral resolution relaxes this and is used whenever two-wheelers are

present. We treat the choice as a validity threat, not a settled matter, and compare the two modes in the sensitivity runs.

TABLE LXIII
EVALUATION METRICS AND COMPARATORS

Improvement	Metrics	Comparator
Fog coordination and radio	End-to-end delay, fog CPU load, packet delivery ratio, failover recovery time	Single fog node of the reference design
Congestion estimation	Vehicle-count error against manual counts; arrival time with measured versus assumed congestion	Fixed, assumed congestion values
Forecasting	MAE and RMSE at 15 min, 1 h, and 3 h; share of accepted bookings that find no space	Markov chain; persistence forecast
Field validation	Sensor accuracy against manual audit, uptime, arrival and searching time with 95% confidence intervals	Blind search; nearest-free-bay guidance
Security	Detection and false-alarm rates for injected spoofed or replayed reports; added delay and energy per message	Reference design without checks
Pricing	Revenue, utilization, and share of rejected demand under several bid distributions	Fixed-rate pricing; reference hybrid pricing

2) Arrival and stay-time models: Three arrival processes are used. (a) A homogeneous Poisson process with rate λ reproduces the reference setting. (b) A non-homogeneous Poisson process has rate $\lambda(t) = \lambda_0\, m_b$ for t in 15-minute bin b, where the multipliers m_b have mean one over the day and are fitted to logged entry times when these exist; arrivals are generated by thinning a dominating process. (c) A surge multiplies the rate by κ over a window $[t_0, t_0 + W]$, giving $\lambda_0 \kappa W$ expected arrivals in the window. Poisson counts have variance equal to their mean; because real event traffic is burstier, a Gamma-mixed (negative binomial) variant with a stated dispersion parameter is run as a robustness check.

For stay times we compare the exponential distribution, whose memorylessness matches the Markov assumption, with a lognormal distribution of the same mean m and coefficient of variation c. The lognormal parameters follow from $\sigma^2 = \ln(1 + c^2)$ and $\mu = \ln m - \sigma^2/2$. For the mean of 30 min implied by our reading of the reference rates (a full lot of 150 spaces emptying at 5 departures per minute gives $150/5 = 30$ min; reading the 5 per minute at a half-full lot, as in Section IV-J, gives 15 min, so the mean stay is not identified by the reported rates), $c = 1$ gives $\sigma = 0.833$ and a median stay of 21.2 min, against an exponential median of $30\ln 2 = 20.8$ min; $c = 1.5$ gives $\sigma = 1.086$ and a median of 16.6 min. The two shapes differ in the tails. The probability of a stay beyond 90 min is $e^{-3} = 0.050$ for the exponential, 0.041 for the lognormal with $c = 1$, and 0.060 with $c = 1.5$. This matters for forecasting, since long stays create the autocorrelation that a memoryless model ignores. It matters little for blocking. With Poisson arrivals and a fixed capacity, the Erlang loss probability is insensitive to the stay distribution beyond its mean, so differences in blocking between the two shapes are a check on the simulator, not a finding.

TABLE LXIV
SIMULATION PARAMETERS. BASE VALUES ARE FROM THE REFERENCE PAPER WHERE MARKED †; OTHER ENTRIES ARE OUR PROPOSALS

Symbol	Meaning	Base ($K=150$)	Proposed ($K=2500$)
K	Parking spaces	150†	2,500
λ	Arrival rate (per min)	10†	167 (100–250)
d	Departure rate (per min)	5†	83 (50–125)
a_0	Initial availability (share of K)	≤0.90†	≤0.90
c	Lane congestion	0.1–0.9†	0.1–0.9, or measured
m	Mean stay (min)	30 (ours)	30 (20–60)
CV	Stay-time coefficient of variation	1	1 (0.5–1.5)
κ	Surge multiplier	–	3 (2–4)
W	Surge window (min)	–	180 (60–240)
Z	Zones (fog nodes)	1†	10 (4–16)
T_h	Heartbeat period (min)	–	15 (5–60)
s	Fog service time (ms)	10† (our reading)	10 (10–250)
π_{2w}	Two-wheeler share	0	0 (0–0.4)

With $\lambda = 167$ per minute and $\kappa = 3$ over $W = 180$ min, the surge window alone brings $167 \times 3 \times 180 \approx 9.0 \times 10^4$ expected arrivals to a 2,500-space lot. Most of them must be turned away, and the share of requests that the guidance, reservation, and pricing logic rejects or diverts is part of what is measured. *4) Radio and fog layers:* Each state change of a bay, and each heartbeat, creates a frame whose time on air $T_{\text{air}} = (n_{\text{pre}} + 4.25 + n_{\text{pay}})2^{SF}/BW$ follows the standard LoRa formula. For SF9, 125 kHz, and 12 payload bytes this is 144.4 ms. A packet-level model schedules each frame as an interval on a channel, chosen at random among C channels. A frame is

3) Parameters: Table LXIV lists the parameters. The base values are those of the reference paper where it reports them [2]. The 2,500-space entries are proposals. Rates scale with capacity ($10/150 = 0.067$ arrivals per space per minute gives 167 per minute at 2,500 spaces) so that the load per space stays comparable, and ranges bracket this by roughly a factor of two.

lost if another frame on the same channel and spreading factor overlaps it in time (pure ALOHA, no capture) and, in a variant, if the signal-to-interference margin falls below a capture threshold. A gateway is a loss system with a fixed number of demodulation paths, and a device that would exceed its duty-cycle limit holds the frame. Frame counters, retries for confirmed uplinks, and downlink acknowledgments are modeled explicitly, because these are the quantities the follow-up study must log [34], [35]. Per-device link quality can be drawn from a path-loss model with shadowing, so that spreading-factor assignment has something to adapt to.

A fog node is a single-server queue with a service time s drawn from a lognormal distribution of mean 10 ms in the base case, plus a network delay to its gateway and to the cloud. Zoning replaces one node by Z nodes with a statesummary exchange every few seconds. Failure is modeled by silencing a node at a chosen instant. The neighbor detects it after a missed-heartbeat timeout and adopts its zone's sensors,

so that failover time has a measurable lower bound set by the timeout.

5) Verification and validation: Following Sargent [59], we separate the two activities. *Verification* asks whether the model implements the specification. It uses (i) unit tests of the arrival generators (sample mean, variance, and stay-time quantiles against theory), of the ToA function against published values, and of the routing function on hand-solved graphs; (ii) conservation checks at every step, namely vehicles entered = vehicles inside + vehicles exited, the sum of bay states equals the occupancy counter, and frames sent = delivered + lost by collision + lost by gateway saturation + held by duty cycle; (iii) degenerate cases, in which zero congestion must reduce the routing to shortest path and a lossless radio must give a delivery ratio of one; (iv) analytical cross-checks, such as the simulated blocking probability against the Erlang loss formula

(for $A = \lambda m$ and 150 servers, $A = 135$ gives 0.016 and $A = 150$ gives 0.062) and the simulated collision rate against $1-e^{-2G}$; and (v) reproduction of the reference scenario with the published parameters. The last is a sanity check on the setup, not a numerical target, because the reference paper omits details such as the layout and driver model. We require the same qualitative ordering of methods and an effect of the same order of magnitude, and we report any discrepancy.

Validation asks whether the model represents the real system for the intended purpose. SUMO's car-following and routing models are documented and have been widely used [7], [40], but that does not validate our lot. Validation uses logged entry and exit times to fit and then hold out the arrival and staytime models, a comparison of simulated and audited search times in the pilot (Section VI-J), and a face-validity review of animations and summary statistics with the lot operator.

F. Metric Definitions

Table LXV gives each metric a definition, unit, and level of measurement; the formulas below fix the less obvious ones. Let vehicle i enter at $t^{\text{in}}{}_i$, occupy its bay at $t^{\text{park}}{}_i$, and travel a distance D_i. Let D_i^* and T_i^* be the shortest-path distance and free-flow travel time from the entrance to the bay finally used. The description of the reference system available to us does not define its two metrics precisely, so we adopt the following reading and state it in every report.

$$T_{i\text{arr}} = t_{\text{park}i} - t_{\text{in}i}, \qquad T_{i\text{srch}} = T_{i\text{arr}} - T_{i*}, \tag{44}$$

$$D_{i\text{cruise}} = D_i - D_{i*}, \qquad \text{PDR} = \frac{N_{\text{deliv}}}{N_{\text{sent}}}. \tag{45}$$

The delivery ratio counts unique application messages, not frames, and is estimated from frame-counter gaps. End-to-end delay is $\Delta_j = t^{\text{decision}}{}_j - t^{\text{event}}{}_j$, from the bay change to the first routing or availability update that reflects it. Its p-th percentile is the $\lceil pn/100 \rceil$-th smallest of n values, reported for $p \in \{50, 95, 99\}$ and averaged over seeds. Fog utilization is $\rho = B/T_{\text{obs}}$, with B the busy time of the server. Failover time is $t_{\text{rec}} - t_{\text{fail}}$, where t_{rec} is the first instant after which the adopted zone's P95 delay stays below a stated bound for a full minute.

For forecasts with errors $e_t = y_t - \hat{y}_t$ over T test points, MAE $= T^{-1} \sum |e_t|$, RMSE $= \sqrt{T^{-1} \sum e_t^2}$, and the skill score against persistence is $\text{SS} = 1 - \text{MSE}_{\text{model}}/\text{MSE}_{\text{pers}}$.

A quantile forecast $q_t^{(\tau)}$ is scored by the pinball loss [47], [70]

$$L_\tau(y, q) = \begin{cases} \tau\,(y - q), & y \geq q, \\ (1-\tau)(q - y) & y < q, \end{cases} \tag{46}$$

averaged over test points, and an interval $[l_t, u_t]$ with nominal level $1 - \alpha$ by its empirical coverage $T^{-1} \sum \mathbf{1}[l_t \leq y_t \leq u_t]$ together with its mean width. Reservation outcomes are FAR $= n_{\text{acc,nospace}}/n_{\text{acc}}$ and FRR $= n_{\text{rej,spacefree}}/n_{\text{rej}}$; the second needs an oracle that knows whether a bay would have been free, which is available in simulation and only approximately in the pilot. Utilization is occupied space-hours divided by $KT_{\text{obs}}/60$ (with T_{obs} in minutes), and revenue per space-hour is total revenue divided by the same denominator.

For attack scenarios every injected report is labeled, so the true-positive rate is TPR = TP/(TP+FN), the false-positive rate is FPR = FP/(FP + TN), and precision is TP/(TP + FP). Sweeping the alert threshold traces the ROC curve (TPR against FPR) and the precision–recall curve. Because genuine reports

vastly outnumber attacks, FPR can look small while operators still see many false alarms, so we also report false alarms per day.

Energy per message is $E_{\mathrm{msg}} = V\,(I_{\mathrm{tx}}T_{\mathrm{air}} + I_{\mathrm{rx}}T_{\mathrm{rx}} + I_{\mathrm{mcu}}T_{\mathrm{mcu}})$, and battery life is $L = \eta Q / \bar{I}$ with $\bar{I} = I_{\mathrm{sleep}} + rE_{\mathrm{msg}}/V$ for message rate r. As an illustration with assumed values, take $V = 3.3$ V, a transmit current of 40 mA, $T_{\mathrm{air}} = 144.4$ ms, and ignore receive and processor time. The transmit energy is 19.1 mJ per frame. With a sleep current of 5 μA and $r = 2$ and 5 frames per hour, $\bar{I} = 8.2$ and 13.0 μA, and for $Q = 2$ Ah with $\eta = 0.8$ the formula gives 22.2 and 14.0 years. These are upper bounds: self-discharge, receive windows, and sensor power are ignored, so the measured quantity of interest is the *difference* that security overhead makes, not the absolute life.

Fairness of the occupancy-aware pricing is measured by four quantities: the share of operating hours at the price cap, $\pi_{\mathrm{cap}} = H^{-1}\sum_h \mathbf{1}[m_h = m_{\max}]$ for price multiplier m_h; the dispersion of posted hourly price, $\mathrm{CV}_p = \mathrm{sd}(p_h)/\bar{p}$; the Gini coefficient of the price paid per stay; and the ratio of rejection rates between user classes (subscribers and short-stay visitors), where a ratio far from one signals that one group bears the scarcity.

G. Experimental Design

1) Factors and configurations: Table LXVI lists the factors. Three improvements form the core of the ablation because they act on the same decision chain (what the sensors report, how it is routed, and what is forecast): zoned fog and radio (Z), camera-based congestion (C), and learning-based forecasting (F). They are crossed in a full 2^3 factorial of eight configurations, which allows the three main effects and the interactions to be estimated. Security (S) and pricing (P) act on largely separate paths, so each is added alone to the reference configuration, and a final configuration enables all five. Two external baselines, blind search and nearest-free-bay guidance, complete the ladder.

TABLE LXVI
EXPERIMENTAL FACTORS AND LEVELS

Factor	Levels
Lot size	150 and 2,500 spaces
Scenario	Weekday, event surge, sensor outage, fog-node failure, degraded sensing, attacks
Core improvements Z,C,F	Off (reference) or on, each
Other improvements S,P	Off or on, one at a time and with all
External baselines	Blind search; nearest free bay
Seeds	30 per cell, shared across configurations
Sensitivity inputs	λ, κ, mean stay, CV, π_{2w}, T_h at low and high values

The main effect of F on a response is the mean over A4, A6, A7, A8 minus the mean over A1, A2, A3, A5, and interactions are the corresponding contrasts of products of signs [76]. Because every configuration is run with the same seeds, each contrast is a paired quantity.

TABLE LXV
METRIC DEFINITIONS, UNITS, AND LEVEL OF MEASUREMENT

Metric	Definition	Unit	Level (aggregation)
Arrival time	Entrance to bay occupied, T_i^{arr}	min	Vehicle; mean per run
Searching time	Arrival time minus free-flow time to the used bay	min	Vehicle; mean per run
Cruising distance	Distance driven minus shortest path to the used bay	m	Vehicle; mean per run
Packet delivery ratio	Unique messages delivered over sent	ratio	Sensor, zone; per run
Delay percentiles	P50, P95, P99 of bay change to decision update	ms	Message; per run
Fog utilization	Server busy time over observation time	ratio	Node; per run
Failover time	Failure instant to restored P95 delay	s	Event; per run
MAE, RMSE	Mean absolute and root-mean-square forecast error	bays	Forecast point; per horizon
Skill score	$1 - \mathrm{MSE}_{\mathrm{model}}/\mathrm{MSE}_{\mathrm{pers}}$	ratio	Series; per horizon
Pinball loss	Eq. (46), at $\tau \in \{0.05, 0.5, 0.95\}$	bays	Forecast point; per τ
Interval coverage, width	Share of outcomes inside the interval; mean width	ratio; bays	Forecast point; per level
False-accept rate	Accepted bookings finding no space over accepted	ratio	Booking; per run
False-reject rate	Rejected requests that had a free bay over rejected	ratio	Request; per run
Utilization	Occupied space-hours over available	ratio	Lot; per run
Revenue per space-hour	Total revenue over $K\,T_{\mathrm{obs}}/60$	USD	Lot; per run
TPR, FPR, precision	Detection and false-alarm measures on labeled injections	ratio	Report; per run and threshold
False alarms per day	Alerts with no injected event, per day	count/day	Lot; per day
Energy per message	E_{msg} as defined above	mJ	Frame; per device
Battery life	$\eta Q/\bar{I}$	years	Device; analytical
Price-capped share, dispersion	π_{cap}, CV_p	ratio	Hour; per run
Gini of price paid, rejection ratio	Inequality of price paid per stay; ratio of class rejection rates	ratio	Stay, class; per run

TABLE LXVII

ABLATION MATRIX (+ ON, − OFF). CONFIGURATIONS A1–A8 ARE THE 2^3 FACTORIAL IN Z, C, F; THE REMAINDER ARE ONE-AT-A-TIME AND BASELINES

ID	Z	C	F	S	P	Note
A1	−	−	−	−	−	Reference design
A2	+	−	−	−	−	
A3	−	+	−	−	−	
A4	−	−	+	−	−	
A5	+	+	−	−	−	
A6	+	−	+	−	−	
A7	−	+	+	−	−	
A8	+	+	+	−	−	
O1	−	−	−	+	−	Security alone
O2	−	−	−	−	+	Pricing alone
FULL	+	+	+	+	+	All five
B0	Blind search					Weak baseline
B1	Nearest free bay					Stronger baseline

2) Number of runs: The core grid has 2 lot sizes ×6 scenarios ×13 configurations ×30 seeds = 4,680 runs. A separate sensitivity sweep varies the six inputs of Table LXVI at a low and a high level around the base case, for the reference configuration and the full one, in two scenarios and at both lot sizes: 6×2×2×2×2×30 = 2,880 runs. The total is 7,560 runs. Since configurations share seeds, only 2×6×30 = 360 distinct random scenarios underlie the core grid, which is what makes the pairing possible. If the power analysis below calls for more seeds, doubling to 60 gives 9,360 core runs.

3) Common random numbers and seed management: Configurations are compared on the same demand. Every random stream (arrivals, stay times, driver choices, radio shadowing, sensor noise, attack injection) has its own generator, and its seed depends on a master seed, lot size, scenario, replication index, and stream name, but *not* on the configuration. Switching an improvement on or off therefore cannot shift another stream, which would destroy the synchronization on which common random numbers rely [75]. Listing 2 shows the derivation. The master seed, the list of derived seeds, and the software versions are stored with each result file.

Listing 2. Seed derivation that ignores the configuration.

```
import hashlib

def seed(master, lot, scen, rep, stream): key =
    f"{master}|{lot}|{scen}|{rep}|{stream}" d =
    hashlib.sha256(key.encode()).digest() return int.from_bytes(d[:4],
    "big")
```

H. Statistical Analysis Plan

1) Paired comparisons and intervals: For configurations A and B and a metric X, let $D_i = X_i(A) - X_i(B)$ over seeds $i = 1,\ldots,n$. The estimate is $\bar{D}$ and the interval is $\bar{D} \pm t_{n-1,1-\alpha/2}\, s_D/\sqrt{n}$ [75]. Run-level means are treated as the unit, not individual vehicles, because vehicles within a run are correlated. As a distribution-free alternative, and as a check on the t-interval for skewed metrics such as delay, we use the percentile and bias-corrected bootstrap [87]: resample the n seed pairs with replacement 10,000 times and take quantiles of the resampled $\bar{D}$. Relative improvements $\bar{X}_B/\bar{X}_A - 1$ are computed by the same resampling, since a ratio has no simple t-interval. The effect size is reported as the paired standardized difference $d_z = \bar{D}/s_D$ together with the relative improvement, and never alone as a p-value.

2) Multiple comparisons: Six hypotheses and many metrics invite false positives. The six primary tests form one family and are controlled by the Holm step-down procedure: order the p-values $p_{(1)} \le \cdots \le p_{(m)}$ and reject in turn while $p_{(i)} \le \alpha/(m - i + 1)$. For $m = 6$ and $\alpha = 0.05$ the thresholds are 0.0083, 0.0100, 0.0125, 0.0167, 0.0250, and 0.0500. Holm needs no independence assumption and is uniformly more powerful than plain Bonferroni. Secondary metrics are reported with unadjusted intervals and labeled exploratory.

Comparing several forecasters over several series is a different problem, because the series are the sampling units. For that case we follow Demsar [88]: a Friedman test on per-series ranks, followed by Nemenyi critical differences, $\mathrm{CD} = q_\alpha \sqrt{k(k+1)/(6N)}$ for k models over N series. With $k = 5$ (persistence, seasonal naive, Markov chain, boosted trees, LSTM), $q_{0.05} = 2.728$ from his table, and $N = 10$ series, $\mathrm{CD} = 1.93$ rank units; two models whose mean ranks differ by less are not distinguished. Comparisons against a single control use a Wilcoxon signed-rank test with the Holm correction instead. With few series the Friedman test has little power, and we say so when the data are few.

3) Pre-registered hypotheses: Table LXVIII fixes the hypotheses before any run, one per improvement. Each success criterion is a statement about a confidence interval, so that it can be falsified. The margins (δ values) are our proposals, chosen to be practically meaningful, and they are not tuned afterwards. A hypothesis counts as supported only if all of its listed conditions hold (an intersection of tests), so the test at the family level uses the largest of the component p-values.

h 4)= Sample size$t\sigma_D/\sqrt{n}$, so*:* The half-width of a$n \geq (t\sigma_D/h)^2$, with$1z-$replacingα interval is$t$ as

a first approximation. Take the illustrative independent-run standard deviation of 1.2 min used earlier for arrival time and a target $h = 0.25$ min. Without common random numbers the difference has $\sigma_D = \sqrt{2} \times 1.2 = 1.70$ min and the normal approximation gives $n \geq (1.96 \times 1.70/0.25)^2 = 177.0$, so 178 runs, or 180 with the t quantile. With a correlation of 0.7 between paired runs, $\sigma_D = \sqrt{2 \times 1.44 \times 0.3} = 0.93$ min and $n \geq 53.1$ by the normal approximation, so 54 runs, or 56 with t. At the planned 30 seeds the half-width is $2.045 \times 0.93/\sqrt{30} = 0.35$ min. If the interval must also survive a Holm adjustment at the strictest step ($\alpha/6 = 0.0083$), 101 seeds would be needed for $h = 0.25$, and 30 seeds give 0.48 min. Thirty seeds are therefore a floor, not a guarantee. The rule is sequential: after 30 seeds, estimate s_D from the data and add blocks of 10 seeds (preserving the pairing) until the target half-width is reached or a cap of 100 is hit, and report the stopping rule. Adding seeds in response to the observed interval, with no change to the hypotheses, is permitted for estimation of intervals but not as a way to cross a significance threshold, and any top-up is reported.

Splits are chronological: the earliest portion of the series is used for training, the next for validation, and the last for testing, with a gap of at least the longest horizon (three hours) between portions. Models are scored by rolling origin [71]: at each origin the model sees only the past and forecasts the horizons 15 min, 1 h, and 3 h. As an arithmetic illustration with a hypothetical 4-week test period of 5-minute data, there are $4\times7\times24\times12 = 8{,}064$ test steps; with hourly origins and a 3-hour horizon there are $672-3 = 669$ origins, and the weekly period of the seasonal-naive baseline is $7 \times 24 \times 12 = 2{,}016$ steps.

The baseline ladder is persistence, seasonal naive (same time one week earlier), the five-state Markov chain of the reference design, and, as a classical statistical model, exponential smoothing or ARIMA as described by Hyndman and Athanasopoulos [46]. The learned models are the boosted trees and the LSTM of Section V. A learned model that does not beat seasonal naive at the 3-hour horizon is not an improvement.

TABLE LXVIII

PRE-REGISTERED HYPOTHESES. Δ DENOTES IMPROVED MINUS COMPARATOR; INTERVALS ARE 95% HOLM-ADJUSTED PAIRED INTERVALS OVER SEEDS; MARGINS ARE PROPOSALS

ID	Claim (comparator)	Success criterion (all must hold)
H1	Zoned fog and radio improve delivery at 2,500 spaces (single node)	Lower bound of ΔPDR > 0.02 in the weekday and surge scenarios; upper bound of ΔP95 delay < 0; failover time reported with its interval
H2	Camera-based congestion helps routing (assumed congestion)	Upper bound of $\Delta T^{\text{arr}} < 0$ against assumed values; upper bound of the relative gap to oracle congestion < 5% (non-inferiority)
H3	Learned forecasters beat the Markov chain at 1 h	Upper bound of ΔRMSE < 0 over rolling origins; SS against persistence has lower bound > 0; coverage of the 90% interval in $[0.85, 0.95]$
H4	Staged validation yields a faithful simulator	Interval of (simulated − audited) mean search time lies within $\pm 20\%$ of the audited mean (equivalence); lower 95% bound of sensor–audit bay agreement ≥ 0.90, as in the S3 gate
H5	Security layer detects injected reports at tolerable cost (reference without checks)	Lower bound of TPR ≥ 0.90; upper bound of false alarms ≤ 2 per day; upper bound of added P95 delay ≤ 10%
H6	Calibrated pricing raises revenue (reference hybrid pricing)	For every bid distribution tried, lower bound of Δrevenue per space-hour > 0; upper bound of utilization loss < 2 points; $\pi_{\text{cap}} \leq 0.20$

I. Forecasting Evaluation Protocol

1) Data, splits, and baselines: Two data sources are used. Logged lot data, if the operator grants access, are the target, with the fields listed in Table LXX. Open occupancy datasets of the kind used by Zheng et al. [12] serve for development and as a cross-check on a second city. Open data are never pooled with the logged data in training unless the pooled variant is a declared experiment.

TABLE LXIX

LEAKAGE CHECKLIST FOR THE FORECASTING STUDY

No.	Check
1	Train, validation, and test periods are disjoint and in time order, with a gap ≥ the longest horizon
2	No random shuffling or random k-fold anywhere in tuning
3	Scalers, encoders, and imputers are fitted on training data only
4	Every feature at origin t uses data from t or earlier (including rolling means and exit counts)
5	Calendar and event flags are known ahead of time, not derived from the outcome
6	Reservation outcomes are not used as features for the forecast they decide
7	Markov matrices are estimated on training data only
8	Test data were not used to choose the model, features, horizon, or stopping point
9	The same origins and horizons are used for every model

2) Tuning, calibration, and leakage: Hyperparameters are tuned only on the validation portion, with the same search

budget for every model (for example, 50 random configurations each), and every trial is logged, not just the best. The test portion is evaluated once per final model. For probabilistic outputs we draw reliability diagrams: for each nominal level τ, the observed share of outcomes at or below $q^{(\tau)}$ is plotted against τ, and a calibrated forecaster lies on the diagonal [69]. If intervals are miscalibrated, a split-conformal correction fitted on the validation portion is the first remedy [48]. Table LXIX lists the leakage checks that must be answered before any forecast result is reported.

J. Pilot Protocol

1) Site, instrumentation, and audit: A site qualifies if (i) the operator gives written permission and a named contact, (ii) the section chosen has a bounded entrance and exit that can be counted, (iii) a gateway can be mounted with mains or protected power and a clear radio path, (iv) demand varies enough in a week to give both busy and idle periods, and (v) a manual audit can be done safely from pedestrian paths. The instrumentation is: bay sensors of at least two types per bay group; entrance and exit counters; a gateway and a fog node with local storage; optionally a junction camera for counts only; and a clock synchronized to a common source, since delay metrics are meaningless otherwise.

TABLE LXX
PILOT DATA LOGGING SCHEMA (ONE ROW PER UPLINK MESSAGE)

Field	Type	Description
ts event	UTC time	Sensor-side time of bay change
ts gw, ts fog	UTC time	Gateway and fog arrival times
dev pseudo	Hash	Pseudonymous device identifier
zone, bay group	Integer	Location of the bay, no plate data
fcnt	Integer	Frame counter, for loss estimates
sf, rssi, snr	Int, float	Radio parameters per frame
state, sensor type	Enum	Occupied or free; sensor kind
health	Bit field	Battery, tamper, and fault flags
decision id	Integer	Link to the resulting routing update
alert	Enum	None, spoof, replay, or inconsistent

Ground truth comes from manual audits. Two observers walk the section independently at the top of the hour during audit sessions and record each bay as occupied or free; a third person tallies entering and leaving vehicles at the entrance over the same period. Sessions cover weekdays and weekends, morning to evening, with additional sessions at known busy periods. Search time is measured on a sample of arrivals by a stopwatch observer who follows consenting volunteers or notes the time from entrance to parking for visible vehicles, and no vehicle is stopped or approached. Inter-rater agreement for the bay state is Cohen's $\kappa = (p_o - p_e)/(1 - p_e)$. As a made-up example, with 200 bays, 120 judged occupied and 72 judged free by both, and 5 and 3 disagreements in either direction, $p_o = 0.96$, $p_e = 0.529$, and $\kappa = 0.915$. For counts, Bland–Altman limits of agreement between observers are reported. A κ below 0.8 triggers observer retraining and a repeat session.

2) Safety, consent, incidents, and rollback: Installation is done outside peak hours with the operator's staff; sensors are flush or low-profile and fixed so that they do not create a trip or puncture hazard; auditors wear high-visibility clothing and stay on pedestrian paths; and fieldwork is paused in extreme heat or storms.

The notice posted at entrances, and repeated in any app screen, states in plain language the purpose (research on parking guidance), what is sensed (bay occupancy, vehicle counts, and no images stored), what is retained and for how long, who is the responsible party, and how to ask questions or object. Under India's Digital Personal Data Protection Act [67] the notice and, where personal data are processed on the basis of consent, a clear affirmative consent are central; under the GDPR [66] the same material appears as transparency, data minimization, purpose limitation, and storage limitation. Because the design collects no plates or faces, most of the data are not personal, but volunteer drivers who share trips are, and they consent separately. This outline is not legal advice and needs review by the operator and the institution.

Incidents (safety events, a suspected data exposure, a radiointerference complaint, or guidance that sends drivers to blocked bays) are logged, reported to the operator on the same day, and reviewed before work resumes. Rollback is triggered by any of: a safety incident attributable to the system; a privacy breach; guidance errors above an agreed share of audited arrivals; or sensor uptime below an agreed floor for a full audit week. Rollback means disabling live guidance, keeping passive logging only if it is safe and lawful, restoring the operator's usual signage, and preserving logs for review. The thresholds are set with the operator before the pilot starts and recorded in the pre-registration.

K. Threats to Validity and Reporting

Table LXXI lists the threats under the usual four headings and the mitigation planned for each.

The follow-up paper would report the structure shown in Table LXXII, which is deliberately empty: its cells are for a future study to fill, and nothing here implies an outcome. Planned figures are: delay and delivery ratio versus number of sensors for both lot sizes; arrival-time distributions per configuration; main-effect and interaction plots from the 2^3 factorial; reliability diagrams and error-versus-horizon curves for forecasters; ROC and precision–recall curves for the attack scenarios; and a simulated-versus-audited comparison from the pilot.

Negative results. We commit to reporting every preregistered hypothesis, whether supported or not. A

hypothesis whose interval includes zero, or whose gain vanishes against the stronger baseline B1, will be reported as such, together with the conditions (for example, high initial availability) under which an improvement shrinks. Deviations from this plan, including changes of margins, seed counts, or scenarios, will be listed with reasons in the follow-up paper.

L. Work Plan

Table LXXIII orders the work in phases without dates or costs. A phase may start when the phases it depends on have delivered, and the pilot never starts before its simulation and bench predecessors have passed their verification gates.

VII. Discussion: From Prototype to Operation

TABLE LXXI Threats to Validity and Planned Mitigations

Type	Threat	Mitigation
Internal	Implementation faults in the co-simulation	Unit tests, conservation checks, analytical cross-checks, code review
Internal	Warm-up bias and seed-specific artifacts	Warm-up analysis or defensible start state; 30 or more shared seeds; seed list published
Internal	Tuning on the test data	Test portion used once; all trials logged; leakage checklist
External	One lot, one city, one season	Second site if possible; open data from other cities; sensitivity ranges stated
External	Simulated drivers differ from real ones	Mixed-traffic and driver-compliance sensitivity runs; pilot comparison
Construct	Metrics differ from the reference paper's meaning	Definitions stated; the reference reading and ours both reported where they differ
Construct	Sensor and camera ground truth is itself noisy	Two observers, agreement statistics, audits in varied light
Conclusion	Multiple testing and underpowered samples	Holm control; pre-registered margins; sequential top-up stated openly
Conclusion	Assumed distributions (bids, stays) drive results	Several distributions; report the worst case along with the mean

The preceding sections examined whether the reference system's evidence supports its claims and what would strengthen that evidence. This section asks a different question: supposing the design were sound, what else would have to be true before an operator in a real city could rely on it? We cover the interpretation of the headline numbers, the conditions of an Indian deployment, the energy budget of battery-powered sensors, the interests of the parties involved, system-level effects, daily operation, interoperability, and the reporting practices that would make future papers easier to compare. As elsewhere, the discussion is analytical; every number below follows from stated assumptions and none is a measurement.

A. Reading the Headline Numbers Responsibly

The reference paper reports that arrival time falls by 16–46% and revenue rises by 10–15% [2]. Taken at face value these are large effects. Three features of the evidence determine how much weight they can bear.

First, relative reductions say nothing about absolute benefit until the baseline is known. If a blind-search driver needed, purely for illustration, 10 minutes to reach a bay, a 16–46% reduction would save 1.6–4.6 minutes; if the baseline were already 3 minutes, the saving would be well under two. The range itself is wide, and the text of the reference paper does not let a reader tell which scenario produces which end of it. A reader planning a deployment needs the baseline's absolute value, its spread across drivers, and the demand level at which each figure was obtained.

Second, the revenue figure is a blend. As Section IV-F noted, short-term revenue under the tiered discount was lower than under fixed pricing, so the whole gain must come from the long-term component. That has an arithmetic consequence (the decomposition of Eq. (32)). If long-term places account for a share s of baseline revenue and short-term revenue does not fall, a 10–15% total gain requires a $10/s$ to $15/s$ percent gain in the long-term part: 20–30% for $s = 0.5$. The smaller the long-term share, the more extraordinary the effect that the synthetic bids must be producing, which is why an independent demand model matters more than the headline.

Third, each figure answers a narrow question: how an algorithm behaves in one simulated lot under inputs the authors chose, against one baseline. It does not answer whether drivers will follow the guidance, whether the sensors will stay accurate for a year, or whether the operator can run the platform. The earlier sections of this paper were largely about these gaps, which the evidence ladder of Fig. 2 organizes: each rung supports a more demanding claim than the one below, and a result obtained at a given rung should be worded accordingly. The reference system's results sit on rungs 1 and 2, with a small part of rung 3, and the evaluation plan of Section VI is designed to climb to the pilot rung.

TABLE LXXII
Mock-Up of the Main Results Table (Empty by Design)

Config.	T_{arr}	T_{srch}	PDR	P95 delay	FAR	Rev./sp.-h
B0						
B1						
A1						
A8						
FULL						

Table LXXIV turns this into a checklist of conditions that should accompany any performance claim about a parking system. The list is deliberately phrased as things a reader should be able to find in a paper, not things a paper should have proved.

B. Deployment Context in India

The authors study in Greater Noida, and the reference system was designed for a large organized lot elsewhere. Moving such a design to Indian cities changes several background assumptions. We note them in general terms, because conditions differ considerably between a mall basement, a campus, a market street, and a transit station, and none of the points below applies to all of them.

Vehicle mix is the first. Two-wheelers form a large part of the fleet in many Indian cities and share space with cars, autorickshaws, and light commercial vehicles. A bay-level design that assumes one vehicle per rectangular bay fits poorly with lots where two-wheelers are parked in rows without individual stalls. Small vehicles also present a smaller magnetic or ultrasonic signature than a car, so detection margins should be checked per vehicle class instead of being tuned on cars alone. For such lots, counting at entrances and exits and reporting zone-level availability may be both cheaper and more reliable than a sensor per space, a point that connects to the zoning argument of Section IV-A.

Informal and unmarked parking is the second. Much onstreet parking has no painted bays, no barrier, and no defined entry. The reference design needs a discrete set of spaces and a controlled entrance to compute occupancy states and apply a tariff. Where these are absent, the first operational step is to define spaces, which is a municipal task as much as a technical one.

Power and connectivity are the third. Outages and voltage fluctuations are possible at fog nodes and gateways, mobile backhaul can degrade at busy times, and a fog node that loses mains power also loses the local autonomy that justified placing it there. This strengthens the case for a small battery or UPS-backed gateway, for local buffering with store-andforward, and for defined behavior when the cloud link is lost.

Climate is the fourth. Heat, dust, and heavy monsoon rain stress enclosures, seals, adhesives, and batteries; direct sun on a sensor housing raises internal temperature above ambient, which shortens battery life and shifts ultrasonic readings. Ingress protection, vented-but-filtered enclosures, and thermal margin in the battery model are therefore design requirements, not refinements.

The fifth is the supplier and maintenance ecosystem. A deployment depends on who can install, repair, and replace units locally and on how quickly spares arrive. Hardware choices that lock the operator to a distant single supplier add risk that does not appear in a simulation.

Payment habits are the sixth. Cash is still common at many lots, and instant account-to-account payments through UPIstyle interfaces and tag-based electronic payment of the kind used at toll plazas are widely used. A tariff design that assumes card-on-file billing or an app-only reservation excludes drivers who do not use it; attendants, QR codes, and tag readers can be options that coexist. Cash handling also leaves revenue data incomplete, which affects any claim about revenue gain.

Finally, data protection law applies. India's Digital Personal Data Protection Act, 2023 [67] regulates the processing of digital personal data, and a parking platform that stores number plates, locations, and payment references is within its practical reach. We make no legal claim here, and Section V gives the technical measures (counts-only edge processing, short reten-

TABLE LXXIII WORK PLAN BY PHASE

No.	Phase	Deliverables	Depends on
P0	Pre-registration	Frozen hypotheses, margins, metrics, analysis scripts stub	–
P1	Simulator build and verification	Co-simulation, unit and conservation tests, reproduced reference scenario	P0
P2	Data acquisition	Operator agreement, logged lot data if available, open datasets cleaned	P0
P3	Forecasting study	Baselines, tuned models, rolling-origin results, calibration plots	P1 (interfaces), P2
P4	Full-scale simulation	Core grid and sensitivity sweep, ablation, hypotheses H1–H3, H5, H6 in simulation	P1, P3
P5	Bench test	Several nodes with injected loss and faults; radio and fog logs	P1
P6	Pilot	Instrumented section, audits, logs, incident record, H4	P4, P5, P2
P7	Analysis and reporting	Tables, figures, negative results, published code and seeds	P4, P6

TABLE LXXIV CONDITIONS THAT MUST ACCOMPANY A PERFORMANCE CLAIM

Condition	What the reader should find stated

Baseline	Policy compared against, including at least one simple informed policy, with its absolute performance
Inputs	Which inputs were measured, which were assumed, and the source of each
Demand regime	Utilization range tested, including near-full operation
Uncertainty	Number of independent runs or days, and intervals rather than single values
Faults	Sensor error, packet loss, outages, and stale data included or explicitly excluded
Scale	Number of bays, sensors, and gateways, and how the metric changes with them
Duration	Length of observation, including at least one change of season or demand pattern for field work
Independence	Whether anyone other than the designers produced the ground truth
Cost	Energy, maintenance, and operating effort needed to obtain the result

TABLE LXXV
DESIGN ASSUMPTIONS, INDIAN-CONTEXT ISSUES, AND RESPONSES

Assumption	Issue	Design response
One car per marked bay	Many two-wheelers; unmarked rows	Zone counts; per-class detection tests
Controlled entry	Open or informal lots	Define spaces and access first; attendant-assisted counting
Mains power at fog node	Outages, fluctuations	Power backup, buffering, local fallback mode
Benign environment	Heat, dust, monsoon	Sealed enclosures, thermal derating, inspections
Card or app payment	Cash, UPI, tag-based payment	Multiple payment paths; reconcile cash records
Single vendor stack	Local support varies	Open interfaces; spares plan; local service contract
Few privacy constraints	Statutory data duties	Data minimization, notice, retention limits

tion, pseudonymous identifiers) that reduce exposure under any regime. Table LXXV lists how each point shifts an assumption of the reference design and the response we suggest.

C. Energy and Maintenance Budget for Bay Sensors

Battery-powered bay sensors are attractive because they avoid trenching and mains wiring, but their cost is paid in visits. A visit is a technician, a vehicle, and often a lane closure; an unnoticed dead sensor is a bay that silently reports stale state. This subsection extends the energy model of Section IV-L8 with two derating factors that matter in practice and uses it to relate reporting rate and spreading factor to a maintenance schedule.

1) Model: Per uplink, the sensor spends $E_{\text{msg}} = V\, I_{\text{tx}} T_{\text{air}} + E_{\text{fix}}$, where T_{air} follows from the standard LoRa time-on-air formula (Section IV-H) and E_{fix} lumps wake-up, sensing, and receive windows. With n_d uplinks per day and sleep power P_{sl}, the daily load is

$$E_{\text{day}} = n_d E_{\text{msg}} + P_{\text{sl}}\,(86400 - n_d T_{\text{air}}). \tag{47}$$

Two further effects reduce life. Self-discharge removes charge even when the sensor is idle; we model it as a constant leakage power $P_{\text{sd}} = \sigma E_b/(365.25 \cdot 86400)$ for an annual selfdischarge fraction σ. A usable-capacity factor $\eta < 1$ covers cutoff voltage, low-temperature capacity loss, and the margin at which the operator wants the cell replaced. The derated life is then

$$L_{\text{bat}} = \frac{\eta E_b}{E_{\text{day}} + 86400 P_{\text{sd}}} \ \text{(days)}. \tag{48}$$

Both derating parameters enter only as assumptions here. We use the energy parameters of Section IV-L8 ($V = 3.3$ V, $I_{\text{tx}} = 40$ mA, $E_{\text{fix}} = 20$ mJ, $5\ \mu$A sleep current, a 2.4 Ah cell, so $E_b = 28{,}512$ J), a 17-byte physical payload (13 + 4 bytes), and, for the derated case only, $\sigma = 2\%$ per year and $\eta = 0.8$. The resulting leakage power is $18.1\ \mu$W, slightly more than the $16.5\ \mu$W sleep power: at low reporting rates the cell is lost to leakage about as fast as to the electronics. Without derating the model reproduces the values of the battery table of Section IV-L8 (14.77, 10.49, and 2.66 years at 144 uplinks per day for SF7, SF9, and SF12).

2) Illustrative results: Table LXXVI gives ideal and derated lives for four reporting rates, computed by the same code as the earlier listing extended with the two derating terms. At SF7 the 144-per-day derated life of 9.1 years is only 62% of the ideal; at SF12 the derating costs less in relative terms (76% retained) because transmission dominates the load. Fig. 14 shows the

TABLE LXXVI
ILLUSTRATIVE BATTERY LIFE IN YEARS, IDEAL / DERATED ($\sigma = 2\%$/YR, $\eta = 0.8$; ASSUMED PARAMETERS)

	Uplinks per day n_d			
SF	24	48	144	288
7	37.7 / 17.2	28.8 / 14.6	14.8 / 9.1	8.5 / 5.8
9	32.2 / 15.7	22.8 / 12.5	10.5 / 6.9	5.8 / 4.2
12	12.8 / 8.2	7.3 / 5.1	2.7 / 2.0	1.4 / 1.1

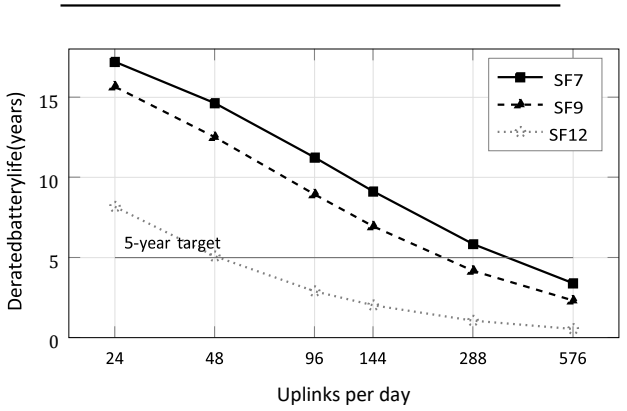


Fig. 14. Derated battery life versus reporting rate for three spreading factors (assumed parameters of Table LXXVI).

derated curves for three spreading factors on a logarithmic reporting axis.

Three observations follow. First, the highest values in the table, such as 37.7 years, exceed what any cell or enclosure seal can be trusted to deliver, so they should be read as statements about the load, not forecasts; the derated column is already more conservative, and any value beyond a decade should be capped by shelf life and enclosure aging. Second, reporting rate and spreading factor interact: for a 5-year derated target the model allows about 355 uplinks per day at SF7, 228 at SF9, but only 49 at SF12. A cell-edge bay forced to SF12 can therefore afford roughly one report every half hour, which constrains the freshness argument of Section IV-H. Third, the model does not include retransmissions, confirmed uplinks, join procedures, or the extra airtime of security bytes, all of which shorten life; the 16-byte case in the earlier table already shows a 24% drop at SF12.

3) Implications for maintenance: Battery life converts to a visit rate. Replacing cells at the end of the derated life implies, for 1,000 bays, about 68 replacements per year at SF7 with 48 uplinks per day and 110 at 144 per day; at SF9 with 144 per day it is 144, and at SF12 it is about 495. These are arithmetic consequences of the illustrative lives above, not field figures. They show that the spreading-factor assignment is a maintenance decision: moving a handful of marginal bays to a better-placed gateway can remove most of the visits. Because the failure of a battery sensor is silent, replacement should be driven by measured battery voltage trends and not by calendar alone, and visits should be batched by route so that one trip covers several replacements, inspections, and re-calibrations.

TABLE LXXVII
TOTAL-COST-OF-OWNERSHIP STRUCTURE

Category	Capital items	Operating items
Sensing	Bay sensors, mounting, commissioning	Battery replacement, recalibration, damaged units
Network	Gateways, antennas, backup power	Backhaul, spectrum or network fees, gateway upkeep
Compute	Fog hardware, enclosure	Electricity, cloud hosting, software updates
Payments	Terminals, tag or QR readers	Transaction fees, reconciliation, dispute handling
People	Installation team, training	Field technicians, monitoring staff, support desk
Compliance	Privacy assessment, audit	Retention management, incident response

TABLE LXXVIII
STAKEHOLDER BENEFITS, COSTS, AND RISKS

Party	Benefit	Cost	Risk
Operator	Higher utilization, revenue data	Capital and upkeep	Stale data, disputes, lock-in
Drivers	Less search time, predictable fees	Data shared, fees may rise	Misleading availability
City author-ity	Less cruising, better data	Oversight effort	Induced demand, equity complaints
Payment provider	Transaction volume	Integration	Fraud, failed settlements
Nearby residents	Less circling traffic	Spillover parking	Noise, overflow into streets

Cost drivers can be described without prices. The capital items are sensors, gateways, fog hardware, enclosures, mounting, and commissioning; the operating items are connectivity, power, cloud and fog hosting, software maintenance, battery replacement, field visits, replacement of vandalized or failed units, calibration, payment fees, and staff. Table LXXVII lists these as the structure of a total-cost-of-ownership comparison.

A comparison that prices only sensors may favor battery nodes at purchase, while a mains-powered or camera-based design may look better over its lifetime; which holds depends on figures that only a pilot can supply.

D. Stakeholders, Incentives, and Fairness

A parking platform sits among parties whose interests only partly coincide. Table LXXVIII summarizes what each gains, what it pays, and what can go wrong.

1) Fairness in dynamic pricing: Demand-responsive prices are justified by the argument that a small share of vacant spaces should be kept to shorten search [17], [19]. The same mechanism can burden drivers who cannot choose their

time or place of travel: workers on fixed shifts, people visiting hospitals, and low-income users with few alternatives. We see four safeguards. A price cap bounds the worst case. A limit on the rate of change keeps prices from jumping within a visit; in generic notation, $p_{t+1} = \min\{p_{\max}, p_t + \delta, f(\rho_t)\}$, where ρ_t is occupancy and δ the permitted step. Concession tariffs or monthly passes preserve access for defined groups. And a cash or non-app payment path ensures the price is payable by everyone. The reference scheme's bid-based longterm places raise a related point: auctions allocate to the highest willingness to pay, which is not the same as the highest need, so a share of long-term capacity may need to be allocated by other rules.

2) Transparency and explainability: A driver who is quoted a price should be able to see what produced it. A pricing rule whose inputs are occupancy and time of day can be stated in a sentence, shown at the entrance, and audited; a blackbox learned policy cannot. We therefore recommend that any learned component propose a price within a published rulebased envelope, and that the operator log the inputs, the rule applied, and the resulting price for each quote. Logs of this kind also support dispute handling and regulator review.

3) Trust depends on data accuracy: The service's value is a promise about the world: this bay is free now. A false promise is more costly than silence, because the driver spends time reaching a bay that is taken and then searches from a worse position. Stale availability is the main source (Section IV-A), alongside sensor faults and reservations that outlive a noshow. Trust compounds: if each recommendation is wrong with probability q independently, the chance that a regular user meets at least one failure in k uses is $1 - (1 - q)^k$. For an illustrative $q = 0.05$ that is 40% in 10 uses and 64% in 20. A system that is right 95% of the time will therefore be experienced as unreliable by its most frequent users. Operators should publish an availability confidence indicator, withdraw guidance for zones whose data are stale, and treat measured false-positive rate as a primary acceptance metric. Drivers should also be told when the display is a forecast and not an observation.

E. Sustainability and System-Level Effects

The environmental case for smart parking rests on reducing the distance driven while searching, which has been estimated to be a meaningful share of urban traffic in some settings [1]. Two cautions apply.

The first is rebound. If guidance makes parking easier, the generalized cost of driving to the area falls, and some trips that would have been made by other means, or not at all, may be induced. This is the kind of reasoning associated with Jevons: an efficiency gain per use can be offset by higher use. We do not claim the effect is large for parking, only that the net benefit is a difference between avoided search distance and induced distance, and that only the first term appears in a typical simulation. A related effect operates through pricing: where parking was previously free or cheap, charging reduces some demand independently of guidance [18]. The two effects should be separated in any evaluation.

The second is measurement. A credible estimate of emissions impact needs, at minimum: observed search distance or time before and after, for a comparable set of trips; the

TABLE LXXIX
HEALTH METRICS AND ALERT RULES (THRESHOLDS ARE PARAMETERS)

Metric	Alert rule
Heartbeat age	Sensor silent for more than k expected report intervals
Delivery ratio	Rolling packet delivery per gateway below θ_{pdr}
Battery trend	Voltage slope predicts cutoff within the visit horizon
Stuck sensor	No state change over h hours while neighbors change
Count balance	Gate entries minus exits differs from sum of bay states by more than ϵ
Fog queue	Backlog above θ_q for m consecutive minutes
Backhaul	Cloud link down longer than θ_b; switch to local mode
Clock skew	Device or node time offset above θ_c
Frame counters	Counter regression or jump per device (possible replay)
Price drift	Quoted price outside the published envelope
Firmware drift	Device version differs from approved release

number of trips and their purposes; vehicle-class mix, since two-wheelers and cars differ in emissions per kilometer; the idling and low-speed share of the search, where fuel use per kilometer is higher; the sensor, gateway, and server energy added by the system; and traffic on neighboring roads, to catch displaced circulation. With these, the net change can be written in generic form as

$$\Delta E = \sum_{v} f_v \left(\Delta D_{v\text{search}} + \Delta D_{v\text{induced}}\right) + \Delta E_{\text{system}}, \tag{49}$$

where f_v is the emission factor of vehicle class v and the ΔD terms are changes in distance. Each term needs a measured or sourced value; none is available in the reference paper.

Computation carries its own footprint. Fog nodes shift processing from distant data centers to small local devices that run continuously; a fleet of them draws power all year, and video analytics draws far more than counting packets. Cloud processing is efficient per operation but adds network transfer and idle capacity. We do not think the balance can be settled in general. It should be reported per deployment, as part of the energy item in Table LXXVII, so that a camera-heavy design is not credited with savings that its own electricity partly cancels.

F. Operating the System

Most of the effort in a long-lived deployment goes to staying correct after commissioning. This subsection lists what an operator must watch, update, and recover.

1) Monitoring and alerting: Observability should be designed with the sensors and not retrofitted. Table LXXIX lists health metrics and alert rules, with thresholds left as parameters to be set from pilot data.

The count-balance rule is the most valuable because it is independent of any one sensor: the entrance and exit counts and the sum of bay states are two views of the same quantity, and persistent disagreement flags either a failed sensor or an uncounted entry. It extends naturally to the conservation argument of Section IV-L6.

2) Firmware updates: Over-the-air updates are indispensable, since a flaw in a fleet of thousands of devices cannot be fixed by visits, and also dangerous. An update channel is a path to run code on every device, so images must be signed and verified before installation, keys must be protected, and rollback to a previous version must be possible so that a faulty image does not strand the fleet. Low-power wide-area links carry updates slowly because of small frames and duty-cycle limits, so updates should be staged by group with a canary subset first, and delta images preferred. Energy is a further risk: a long update drains the battery that the schedule above assumed, and an interrupted update must leave the device bootable. Update orchestration belongs in the threat model of Section IV-L.

3) Calibration drift and re-calibration: Sensors drift. Ultrasonic ranging, for example, depends on the speed of sound, which rises by about 0.6 m/s per kelvin near room temperature. Using the common approximation $c \approx 331.3 + 0.606\vartheta$ with ϑ in °C, a sensor calibrated at 20 °C and used without temperature compensation at 45 °C reads a distance too small by about 4.2% of the true range, i.e., about 8.5 cm at a true range of 2 m. That is enough to shift a thresholdbased occupancy decision. Magnetic sensors drift with their baseline when neighboring metal changes, and cameras drift when mounts shift. A re-calibration schedule should combine periodic checks against manual ground truth for a sample of bays, automatic baseline updates when a bay is known to be empty (for example at the quietest hour), and eventdriven checks when the monitoring rules flag a stuck or noisy sensor. The calibration interval should be set from pilot drift measurements.

4) Incident handling and data retention: Operations need defined degraded modes. When data are stale, the system should show last-known values with a clear marker, disable reservations for affected zones, and fall back to a static tariff; when the cloud link fails, the fog node should keep serving local decisions. Incidents should be classified by impact on drivers, money, and personal data, with a named owner, a runbook, an after-action review, and a route to notify affected parties when personal data are involved. Retention should be defined per data class: raw radio frames and debug logs for a short period, aggregated occupancy counts for long-term planning, payment records for as long as financial obligations require, and no retained camera frames in the counts-only design of Section V. Deletion must be executed and verified, which is itself an operational task.

G. Interoperability, Standards, and Roadmap to Multi-Lot

Sotres et al. [10] ran the same parking service over different IoT platforms in two cities, which shows that portability is achievable and also that it needs deliberate design. For an operator, open interfaces matter for three reasons: a vendor can be replaced without rebuilding the service; third-party navigation and payment apps can integrate; and a city can combine data from several lots. At the radio layer, the LoRaWAN specification [34] fixes the device-to-network contract, and at the messaging layer MQTT [39] is a common publish–subscribe choice. Above these, the application needs a generic data model. A minimal one has lots, bays or zones, observations (state, timestamp, quality), tariffs, reservations, and devices (type, firmware, battery), with stable identifiers, explicit time zones, and an observation quality field so that stale or estimated values are never presented as measurements. A versioned REST or streaming interface, documented and tested against sample payloads, completes it.

Lock-in is not only a hardware matter. It arises when payloads use proprietary encodings, when history cannot be exported, or when the pricing logic lives inside an opaque service. Contracts should require export in a documented format. A roadmap toward several lots follows the evidence ladder: first one lot with ground truth (the pilot rung of Fig. 2); then a second lot with different geometry to test transfer; then a shared platform that adds tenancy, per-lot tariffs, and crosslot guidance, where the main new risk is a recommendation that moves drivers between lots and shifts load onto the less-instrumented one. The cooperating-fog-node design that the reference paper mentions only as a possibility becomes necessary at this stage and should be validated in the same way as the single-lot design.

H. Lessons for Authors and Reviewers

The reporting checklist in the appendices lists items to be completed per study. Here we state the recommendations behind it in short form.

1) State the rung of the evidence ladder (Fig. 2) at which each result was obtained.
2) Mark every input as measured, assumed, or generated, and give its source.

3) Include a baseline that a practitioner would actually use, in addition to a weak one.
4) Report absolute values with the relative improvement.
5) Repeat experiments with independent seeds or days and report intervals.
6) Inject faults (loss, drift, outages) or say plainly that none were modeled.
7) Report scale: bay counts, device counts, and how the metric changes when they increase.
8) Report the energy, maintenance, and staffing needed to obtain the result.
9) Separate revenue effects that come from pricing from those that come from guidance.
10) Describe privacy handling and data retention, not only accuracy.
11) Release code, configurations, and anonymized traces where possible.
12) Phrase conclusions to match the evidence: a simulated result is evidence that an idea may work, not that a product does.

For reviewers, the first two items are the quickest screening test: if a paper cannot say which inputs it measured, the remaining numbers cannot be interpreted.

I. Limitations of This Work

This paper is a desk analysis. All statements about the reference system come from its published description, and we did not obtain its code, data, or hardware; where that description is silent we have said so instead of guessing, and some of our criticisms may reflect omissions of presentation instead of omissions of work. The calculations in Sections VII-A and VII-C use assumed parameters: the energy model uses datasheet-style round numbers, the derating factors are illustrative, and no battery, sensor, or radio was measured. The reasoning on fairness, sustainability, and operations is qualitative and draws on general knowledge, not on interviews with operators or drivers. Our remarks on the Indian context are general and drawn from the authors' awareness of local conditions, not from a survey; conditions vary across cities and lots, and legal points are not legal advice. Finally, the proposed improvements and evaluation plan remain untested.

Several findings would change our conclusions. A field study showing that a fixed-rate Markov predictor matches learned models on real occupancy traces would weaken our forecasting argument. Measured packet loss and node load at full scale well below our estimates would reduce the importance of zoning. Observed sensor drift and battery performance better than our assumed derating would relax the maintenance burden. Pricing outcomes from real demand data that reproduce the reported gain would rehabilitate the revenue claim. Conversely, field data worse than our assumptions would strengthen every concern raised here. Either way, the way forward is the same: climb the evidence ladder and report each rung honestly.

VIII. Conclusion

This paper examined a recent fog-based IoT smart parking system [2] and found six limitations that separate its promising simulation results from dependable deployment: a small prototype, an unexercised camera-based congestion input, a fixedrate Markov forecaster, validation that is mostly simulated, a thin treatment of security and privacy, and pricing tests built on synthetic bids. For each limitation we proposed a concrete improvement, namely zoned fog coordination, camera-assisted congestion counting, learning-based forecasting with safety margins, staged validation, a layered security and privacy design, and demand-calibrated pricing with a precisely stated rule. An evaluation plan with metrics, scenarios, baselines, and ethical safeguards was defined so that these proposals can be tested.

The main limitation of the present work is that it is analytical: it reports no new experiments. The immediate next step is therefore to implement the proposed components, run the evaluation plan in simulation at full lot size, and then confirm the findings in a supervised pilot. We hope that the analysis also helps other researchers judge how much weight small-scale, simulation-based smart parking results can bear.

Acknowledgment

The authors thank Prof. Apoorv Jain for his guidance and support.

TABLE LXXX
Abbreviations: Networking, Radio, and Platform

Term	Definition
IoT	Internet of Things: networked sensors and actuators
LPWAN	Low-power wide-area network; long range at a low data rate
LoRa	Chirp-spread-spectrum radio modulation for LPWANs
LoRaWAN	MAC and network protocol layered on LoRa [34]
ISM band	Radio bands for industrial, scientific, and medical use, open under local rules
SF	Spreading factor, 7 to 12; higher means longer range and longer airtime
BW	Channel bandwidth, here 125 kHz
CR	Coding rate: 4/5 for $\mathrm{CR} = 1$ up to 4/8 for $\mathrm{CR} = 4$
ToA	Time on air: duration of one transmitted frame
ADR	Adaptive data rate: the network adjusts a device's SF and power
PDR	Packet delivery ratio: delivered over transmitted frames
RSSI	Received signal strength indicator, in dBm
SNR	Signal-to-noise ratio, in dB
OTAA	Over-the-air activation: session keys derived at join
ABP	Activation by personalization: keys and counters preset

FCnt	Frame counter; rises with each uplink and defeats replay
Gateway	Relay between LoRa radio and an IP back end
Fog node	Edge computer near the lot that filters and decides locally
MQTT	Lightweight publish–subscribe messaging protocol [39]
NB-IoT	Narrowband IoT: a cellular LPWAN technology
RFID	Radio-frequency identification, used for credentials
SPOF	Single point of failure
MTTR	Mean time to repair

APPENDIX A
GLOSSARY AND NOTATION

This appendix collects the abbreviations used in the paper and the symbols of the analytical sections. Symbols are reused across sections where the field's convention is strong (for example, τ and ρ both have several uses), so Table LXXXII states the meaning in each context. Unless a row says otherwise, definitions follow the cited standards and textbooks in the main text and make no claim about the reference system beyond what Section III reports.

APPENDIX B
CALCULATION CODE

Every number in the analytical sections that comes from a formula can be regenerated with the short programs below. They use only the Python standard library and numpy, were run with Python 3.13 and numpy 2.5 to produce the printed output, and contain no random numbers, so the output is identical on every run. The programs are teaching code: they favor readability over speed and do no input validation beyond what is stated. All inputs that describe the reference system are the ones used in Section III; every other parameter is an illustrative assumption.

A. LoRa Time on Air

The time on air determines both energy per message and channel load, so it is the first quantity to compute. Listing 3 implements the formula of the transceiver data sheet for explicit header, CRC on, and a configurable preamble. Lowdata-rate optimization is switched on automatically for SF11 and SF12 at 125 kHz, as the radio requires. The table generator prints the nominal bit rate and the airtime for 12-, 25-, and 51byte physical payloads. The 12-byte column reproduces 41.22, 144.38, and 1,155.07 ms for SF7, SF9, and SF12; the 25-byte column corresponds to a 12-byte application payload plus 13 bytes of LoRaWAN overhead.

TABLE LXXXI
ABBREVIATIONS: STATISTICS, SECURITY, AND LEGAL

Term	Definition
SUMO	Simulation of Urban MObility, an open traffic simulator [7]
ALOHA	Random-access scheme with no channel sensing before sending
Erlang B	Blocking probability of an M/M/K/K loss system
MAE	Mean absolute error
RMSE	Root mean squared error
MAPE	Mean absolute percentage error
MASE	Mean absolute scaled error [68]
CRPS	Continuous ranked probability score [70]
SS	Skill score: $1 - \mathrm{MSE}_{\mathrm{model}}/\mathrm{MSE}_{\mathrm{baseline}}$
CI	Confidence interval
ACF	Autocorrelation function
TV	Total variation distance between distributions
TLS	Transport Layer Security [80]
AES	Advanced Encryption Standard; LoRaWAN uses 128-bit keys
MIC	Message integrity code, 32 bits in LoRaWAN
STRIDE	Threat taxonomy: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege
PII	Personally identifiable information
DPIA	Data protection impact assessment
GDPR	EU General Data Protection Regulation [66]
DPDP	India's Digital Personal Data Protection Act, 2023 [67]

Listing 3. LoRa time on air (milliseconds) and nominal bit rate

```
import math

def toa_ms(pl, sf, bw=125e3, cr=1, npre=8, crc=1, ih=0,
           de=None):
    """Semtech SX127x time on air, in ms."""
    if de is None:
        de = 1 if sf >= 11 and bw == 125e3 else 0
    ts = 2**sf / bw  # symbol time (s)
    num = 8*pl - 4*sf + 28 + 16*crc - 20*ih
    den = 4*(sf - 2*de)
    npay = 8 + max(math.ceil(num/den)*(cr + 4), 0)
    return 1000 * (npre + 4.25 + npay) * ts

def bitrate(sf, bw=125e3, cr=1):
    return sf * bw / 2**sf * 4/(cr + 4)

print("SF Rb(b/s) " + " ".join( f"PL={p:>2}" for p in (12, 25, 51)))
for sf in range(7, 13):
    row = " ".join(f"{toa_ms(p, sf):7.2f}" for p in (12, 25, 51))
    print(f"{sf:>2} {bitrate(sf):7.0f} {row}")
```

TABLE LXXXII
NOTATION USED ACROSS THE ANALYTICAL SECTIONS

Symbol	Meaning
N	Number of sensors or bays in a lot or zone
K	Bay capacity in the Erlang loss model
$N(t)$	Occupied bays at time t
λ, μ	Arrival rate; per-car departure rate (also the state-change rate of a bay)
ρ	Offered load λ/μ in Erlangs; in the attack model, the true occupied fraction
ρ, $\hat{\rho}^*$	Estimated occupied fraction; acceptance limit $1 - m/N$
m	Safety margin in bays (in the elasticity model, a price multiplier)
λ_e, T_h	State changes per bay per hour; heartbeat interval
r, Λ	Packets per second per sensor; $\Lambda = Nr$
PL	Physical payload length in bytes
T_s, T_{air}	LoRa symbol time $2^{SF}/BW$; time on air
G	Offered load: mean frames in the air, ΛT_{air}

p_s	ALOHA success probability, e^{-2G}
$N_{\max}$, Z	Sensors per channel at a target p_s; number of zones
Δ_x	Longest report gap with change probability below x
$B(\rho,K)$	Erlang B blocking probability
$G = (V,E)$	Junction graph (context separates it from the load G)
L_e, c_e, w_e	Length, congestion in $[0,1]$, and cost of edge e
v_{free}, α, β	Free-flow speed and shape parameters of w_e
$\mathbf{P}$, π	Transition matrix; stationary distribution
X_m	Occupancy state after m five-minute steps
θ, τ	State limit and probability threshold of the acceptance rule. τ is also a turnover rate, a quantile level, or an alarm threshold where stated
k	Spaces offered in the long-term auction
$b_{(j)}$, $R(j)$	j-th highest bid; revenue $j\,b_{(j)}$
$j^{\star}$	Revenue-maximizing number of winners
r_0	Default subscription rate used as a fixed-rate benchmark
φ, δ	Attacker-controlled bay fraction; reporting shift (also a stay-length discount)
E, X, $o(t)$	Gate entries, exits, and occupancy; $o(t) = o(0)+E-X$
c_{FP}, c_{FN}	Costs of a wrongly accepted and a wrongly rejected booking
$p^{\star}$	Acceptance threshold $c_{FP}/(c_{FP}+c_{FN})$
e_t, ℓ_τ	Forecast error $y_t - \hat{y}_t$; pinball loss at level τ
E_{msg}	Energy per uplink, $P_{\text{tx}}T_{\text{air}} + E_{\text{fix}}$

Output:

SF	Rb(b/s)	PL=12	PL=25	PL=51
7	5469	41.22	61.70	102.66
8	3125	82.43	113.15	184.83
9	1758	144.38	205.82	328.70
10	977	288.77	411.65	616.45
11	537	577.54	823.30	314.82
12	293	1155.07	1482.75	2465.79

The formula covers the physical layer only. It ignores the receive windows of class A devices, retransmissions of confirmed uplinks, and any regional limits on transmit time, all of which change the energy and capacity picture.

B. ALOHA Success Probability and Sensor Capacity

Under the pure-ALOHA assumptions of the capacity analysis (Poisson packet starts, any overlap destroys both frames, no capture), a frame survives with probability $p_s = e^{-2G}$, where $G = N\,rT_{\text{air}}$. Listing 4 computes G and p_s for a 2,500-sensor lot and inverts the formula to find the largest N that keeps $p_s \geq 0.9$ on one channel. The inverse is $N_{\max} = \lfloor -\ln(p^{\star})/(2rT_{\text{air}}) \rfloor$. The airtimes are entered as constants taken from Listing 3 so that the program is selfcontained. Output:

Listing 4. Offered load, success probability, and capacity

```
import math

def p_success(g):
    """Pure ALOHA: no overlap in a 2*ToA window."""
    return math.exp(-2*g)

def offered_load(n, rate_h, toa_ms):
    return n * rate_h/3600 * toa_ms/1000

def n_max(rate_h, toa_ms, target=0.9):
    """Largest N with p_s >= target."""
    per_sensor = rate_h/3600 * toa_ms/1000
    return math.floor(-math.log(target) / (2*per_sensor))

# ToA in ms for PL = 12 B (Listing 1)
toa = {7: 41.216, 9: 144.384, 12: 1155.072}
print("N=2500 sensors, rate 2/h")
for sf, t in toa.items():
    g = offered_load(2500, 2, t)
    print(f"SF{sf:<2} G={g:.4f} " f"ps={p_success(g):.3f}")
print("max N for ps >= 0.9")
for rate in (2, 5):
    print(f"rate {rate}/h:",
          {sf: n_max(rate, t) for sf, t in
           toa.items()})
g = 0.5
print(f"peak throughput {g*p_success(g):.4f}")
```

```
N=2500 sensors, rate 2/h
SF7 G=0.0572 ps=0.892
SF9  G=0.2005  ps=0.670  SF12  G=1.6043  ps=0.040
max N for ps >= 0.9 rate 2/h: {7: 2300, 9: 656, 12: 82}
rate 5/h: {7: 920, 9: 262, 12: 32}
peak throughput 0.1839
```

The model is a first approximation. It treats all sensors as one population at one spreading factor, ignores the capture effect (which would raise p_s) and quasi-orthogonality between spreading factors (which would lower the benefit of mixing them), and assumes that loss comes from collisions alone. Fading, obstruction, and external interference are not modeled, so the capacities are upper bounds for a given channel plan.

C. Erlang B and Markov Chain Helpers

Listing 5 contains three helpers. The Erlang B function uses the stable recursion $B_0 = 1$, $B_i = \rho B_{i-1}/(i+\rho B_{i-1})$, which avoids the factorial overflow of the closed form at $K = 150$. The n-step function raises a transition matrix to a power, and the stationary function solves $\pi\mathbf{P} = \pi$ together with $\sum_i \pi_i = 1$ as an overdetermined least-squares system. The matrix is the illustrative five-state matrix of Section III; it is not estimated from data. The program prints the stationary vector scaled by 99, which gives $(14,23,25,23,14)$, the probability of staying at or below state 3 at four horizons, the largest total-variation distance to π after 36 steps (three hours), and the secondlargest eigenvalue modulus, which sets the mixing rate.

Listing 5. Erlang B recursion and Markov chain helpers

```
import numpy as np

def erlang_b(rho, k):
    """Stable recursion for blocking probability."""
    b = 1.0
    for i in range(1, k + 1):
        b = rho*b / (i + rho*b)
    return b

P = np.array([
```

```
        [0.80, 0.18, 0.02, 0.00, 0.00],
        [0.10, 0.74, 0.14, 0.02, 0.00],
        [0.02, 0.12, 0.72, 0.12, 0.02],
        [0.00, 0.02, 0.14, 0.74, 0.10],
        [0.00, 0.00, 0.02, 0.18, 0.80]])

def n_step(P, n): return np.linalg.matrix_power(P, n)

def stationary(P):
        """Solve pi P = pi with sum(pi) = 1."""  m = len(P) a = np.vstack([P.T
        - np.eye(m), np.ones(m)]) rhs = np.zeros(m + 1); rhs[-1] = 1 return
        np.linalg.lstsq(a, rhs, rcond=None)[0]

for rho in (100, 130, 150, 180):
        print(f"rho={rho:>3} B={erlang_b(rho, 150):.3e}")
pi = stationary(P) print("pi*99 :", np.round(pi*99, 4)) print("sum
pi[:3] =", round(pi[:3].sum(), 3)) for n in (1, 6, 12, 36):
        acc = n_step(P, n)[:, :3].sum(axis=1) print(f"n={n:>2}",
        np.round(acc, 3))
tv = 0.5*np.abs(n_step(P, 36) - pi).sum(axis=1) print("max TV at
n=36:", round(tv.max(), 4)) print("|eig2| =", round(sorted(
abs(np.linalg.eigvals(P)))[-2], 3))
```

Output:

```
rho=100 B=6.511e-07 rho=130 B=7.823e-03
rho=150 B=6.240e-02 rho=180 B=1.881e-01 pi*99 :
[14. 23. 25. 23. 14.] sum pi[:3] = 0.626 n= 1 [1.
                0.98 0.86 0.16 0.02] n= 6 [0.923 0.829
0.659 0.433 0.256] n=12 [0.801 0.729 0.631 0.526
0.439] n=36 [0.641 0.634 0.626 0.618 0.612] max TV
at n=36: 0.0145 |eig2| = 0.9
```

The rows for $n = 36$ differ by less than 0.03, which is the numerical form of the observation that a time-homogeneous chain forgets its starting state within three hours. The helpers assume a time-homogeneous, irreducible chain; a product of period-specific matrices would need its own loop.

D. Uniform-Price Auction Revenue

The long-term pricing rule sorts bids in descending order and chooses the number of winners $j \le \min(k,n)$ that maximizes $j\, b_{(j)}$, with price $b_{(j)}$. Listing 6 implements this and runs it on the worked example of the reference paper (20 bids of USD 200, three of USD 150, seven of USD 90) for five values of k. The optimal price is USD 200 in every case, with revenues of 1,000, 2,000, 4,000, 4,000, and 4,000, which is the basis of the statement in Section IV-F that USD 150 is not revenue-maximizing for this example.

Listing 6. Revenue-maximizing uniform price

```
def best_uniform_price(bids, k): """Maximize R(j) = j*b_(j), j <= min(k, n)."""
        b = sorted(bids, reverse=True) best_j, best_r = 0, 0 for j in range(1,
        min(k, len(b)) + 1): r = j * b[j - 1] if r > best_r:                # first max
        wins ties best_j, best_r = j, r
        price = b[best_j - 1] if best_j else 0 return price, best_j, best_r

bids = [200]*20 + [150]*3 + [90]*7 print(len(bids), "bids; k: price, winners,
revenue") for k in (5, 10, 20, 23, 30):
        p, j, r = best_uniform_price(bids, k) print(f"k={k:>2}: price={p} "
        f"winners={j:>2} rev={r}")
```

Output:

```
30 bids; k: price, winners, revenue k= 5: price=200
winners= 5 rev=1000 k=10: price=200 winners=10
rev=2000 k=20: price=200 winners=20 rev=4000
k=23: price=200 winners=20 rev=4000 k=30:
price=200 winners=20 rev=4000
```

The function evaluates reported bids as if they were true values. It does not model bid shading, ties beyond the first maximum, or bidders who would not accept the price, so it is a calculator for the stated objective and not a prediction of what bidders would do.

E. Congestion-Weighted Shortest Path

Listing 7 runs Dijkstra's algorithm with a binary heap (heapq) on the seven-junction graph of Section III. Edge costs follow $w_e = (L_e/v_{\text{free}})(1 + \alpha c^{\beta}_e)$ with $v_{\text{free}} = 5$ m/s, $\alpha = 1.5$, and $\beta = 2$; the lengths and congestion values are those of the example and are hypothetical. Stale heap entries are skipped instead of being decreased in place, which is the usual idiom with heapq. The program prints the leastcost route (S–B–D–T, 70.05 s), the route when congestion is ignored (S–A–C–T, 60.00 s), and the regret of a wrong belief: if the system believes that edge D–T has $c = 0.3$ while the true value is 0.9, it sends the vehicle along the cheaper-looking route, whose true cost is 89.49 s against an optimum of 71.28 s. Output:

```
route S-B-D-T cost 70.05 s c=0          S-A-C-T cost 60.00 s
true cost of chosen 89.49, optimum 71.28 regret 18.21 s
(25.5%)
```

The listing is a snapshot planner: costs are fixed during the search, vehicles do not influence one another, and the cost function is a stand-in because the reference description does not give one.

F. Replay Window and Token Bucket

Two small defensive checks are common to the security measures of Section V. The class ReplayWindow in Listing 8 keeps the highest counter seen and a bitmap of the counters in a window behind it. A new highest counter is accepted and shifts the window; an older counter is accepted once, if it is still inside the window, and rejected if it was

Listing 7. Dijkstra with congestion-weighted edges

```
import heapq

V_FREE, ALPHA, BETA = 5.0, 1.5, 2.0
# (u, v, length_m, congestion)
```

```
EDGES = [("S","A",100,.2), ("S","B",140,.1),
         ("A","B",60,.3), ("A","C",120,.2),
         ("B","D",100,.2), ("C","D",70,.4),
         ("C","T",80,.6), ("D","T",90,.3),
         ("D","E",50,.1), ("E","T",60,.3)]

def cost(length, c): return length / V_FREE * (1 + ALPHA * c**BETA)

def build(edges, override=None): g = {} for u, v,
    L, c in edges:
        if override and {u, v} == set(override[0]):
            c = override[1]
        w = cost(L, c)
        g.setdefault(u, []).append((v, w))
        g.setdefault(v, []).append((u, w))
    return g

def dijkstra(g, s, t): dist, pred, heap = {s: 0.0}, {}, [(0.0, s)] while heap:
        d, u = heapq.heappop(heap) if d >
        dist.get(u, 1e18):
            continue            # stale entry if u == t: break for v, w
        in g[u]:
            if d + w < dist.get(v, 1e18): dist[v], pred[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    path = [t] while path[-1] != s:
        path.append(pred[path[-1]])
    return path[::-1], dist[t]

def path_cost(g, path): legs = zip(path, path[1:]) return
    sum(dict(g[a])[b] for a, b in legs)

p, d = dijkstra(build(EDGES), "S", "T") print("route", "-".join(p), f"cost
{d:.2f} s") flat = [(u, v, L, 0.0) for u, v, L, c in EDGES] p0, d0 =
dijkstra(build(flat), "S", "T") print("c=0 ", "-".join(p0), f"cost {d0:.2f} s")
# belief c_DT=0.3, truth 0.9 true_g = build(EDGES, (("D","T"), 0.9))
chosen, _ = dijkstra(build(EDGES), "S", "T") best, db = dijkstra(true_g,
"S", "T") dc = path_cost(true_g, chosen) print(f"true cost of chosen
{dc:.2f}, " f"optimum {db:.2f}")
print(f"regret {dc-db:.2f} s ({(dc-db)/db:.1%})")
```

seen or has fallen out. A window of size one reduces to the strict "greater than the last counter" rule that LoRaWAN servers normally apply; wider windows suit a backhaul that can reorder frames, at the price of accepting late frames. The class TokenBucket limits requests per account: tokens accrue at a fixed rate up to a burst size, and each request spends one. Time is an explicit argument so that the test is deterministic. In the test run, counters 3 and 4 arrive late and are accepted once, the duplicate 2 and the second 3 are rejected, and counter 5, which reappears after the window has moved to 20, is rejected as too old; the bucket admits a burst of three, throttles the next requests, and recovers after idle time.

Listing 8. Replay-window check and token-bucket limiter

```
class ReplayWindow: """Sliding-window replay check.""" def __init__(self,
    size=16): self.size, self.top, self.mask = size, -1, 0

    def check(self, c):
        full = (1 << self.size) - 1
        if c > self.top:            # new highest s = c - self.top m =
            (self.mask << s) | 1 self.mask = m & full self.top = c
            return True
        d = self.top - c            # age of counter seen = (self.mask >>
        d) & 1 if d >= self.size or seen:
            return False            # too old or seen
        self.mask |= 1 << d return True

class TokenBucket:
    """Rate limiter; time is passed in explicitly.""" def __init__(self, rate,
    burst): self.rate, self.cap = rate, burst self.tokens, self.t = burst, 0.0

    def allow(self, now, cost=1.0): add = (now - self.t) * self.rate self.tokens =
        min(self.cap, self.tokens + add) self.t = now if self.tokens >= cost:
        self.tokens -= cost return True
        return False

w = ReplayWindow(size=8)
seq = [0, 1, 2, 2, 5, 3, 3, 4, 20, 5, 19]
print("counters:", seq) print("accepted:", [int(w.check(c)) for c in seq])

tb = TokenBucket(rate=0.5, burst=3) # 0.5/s times = [0, .1, .2, .3, 1, 2.5, 2.6,
2.7, 10] print("times  :", times) print("allowed :", [int(tb.allow(t)) for t in
times])
```

Output:

```
counters: [0, 1, 2, 2, 5, 3, 3, 4, 20, 5, 19] accepted: [1, 1, 1, 0, 1, 1, 0, 1, 1,
0, 1] times  : [0, 0.1, 0.2, 0.3, 1, 2.5, 2.6, 2.7, 10] allowed : [1, 1, 1, 0, 0, 1,
0, 0, 1]
```

Neither class authenticates anything: the replay check is meaningful only for counters that have already passed the message integrity check, and the bucket must be keyed by an authenticated account, not by a spoofable address. Counter handling across the 16-bit wrap of LoRaWAN 1.0.x frames, and persistence across a server restart, are left out.

G. Forecast Metrics

Listing 9 defines the metrics used in the evaluation plan: MAE, RMSE, the pinball loss at quantile level τ, the empirical coverage of a prediction interval, and the skill score against a baseline, $1-\mathrm{MSE}_{\mathrm{model}}/\mathrm{MSE}_{\mathrm{baseline}}$. The eight occupancy values and both forecasts are *invented* to exercise the functions; they are not data from any lot and carry no information about forecasting accuracy. The baseline is persistence (the previous value, with an invented first value). The interval is the forecast plus or minus three bays, and the upper quantile forecast is the point forecast plus three bays.

Listing 9. Forecast error, interval, and skill metrics

```
import numpy as np

def mae(y, f): return float(np.mean(np.abs(y - f)))

def rmse(y, f): return float(np.sqrt(np.mean((y - f)**2)))

def pinball(y, q, tau): u = y - q
    return float(np.mean(u*(tau - (u < 0))))

def coverage(y, lo, hi): return float(np.mean((y >= lo) & (y <= hi)))
```

```
def skill(y, f, base): """1 - MSE_model / MSE_baseline."""
    mse = lambda e: np.mean(e**2) return 1 - mse(y -
    f)/mse(y - base)

# Hypothetical occupancy (bays); invented numbers y = np.array(
    [112, 118, 125, 131, 138, 142, 145, 140.]) fcst = np.array(
    [110, 120, 124, 133, 135, 143, 141, 142.]) pers =
np.concatenate(([108.], y[:-1])) # lag 1 lo              = fcst - 3 hi   = fcst +
3 q90 = fcst + 3

print("MAE model, persist:", round(mae(y, fcst), 3), round(mae(y, pers), 3))
print("RMSE model, persist:", round(rmse(y, fcst), 3), round(rmse(y, pers), 3))
print("pinball(0.9):", round(pinball(y, q90, 0.9), 3)) print("coverage of [f-3,
f+3]:", coverage(y, lo, hi)) print("mean width:", float(np.mean(hi - lo))) sk =
skill(y, fcst, pers) print("skill vs persistence:", round(float(sk), 3))
```

Output:

```
MAE model, persist: 2.125 5.25 RMSE model,
persist: 2.318 5.431 pinball(0.9): 0.387
coverage of [f-3, f+3]: 0.875 mean width: 6.0
skill vs persistence: 0.818
```

Coverage is reported with the mean interval width, since an interval can reach any coverage by widening. With only eight points, no value above has statistical meaning; real evaluations need the sample sizes and confidence intervals described in Section VI.

Appendix C

Reporting Checklist for Smart Parking Studies

The checklist in Table LXXXIII lists what a smart parking paper should state so that a reader can judge and repeat its claims. It is derived from the gaps that Section IV found in the reference study and is meant for authors, reviewers, and the follow-up study of Section VI. An item is satisfied by stating the information, not by having a particular value.

Appendix D

Risk Register for a Pilot Deployment

A supervised pilot (Section V) should start with a written list of what can go wrong. Table LXXXIV is a starting register; the likelihood (L) and impact (I) ratings are qualitative judgments of the authors, on a three-level scale of low (L), medium

TABLE LXXXIII
REPORTING CHECKLIST WITH RATIONALE

ID	Report	Why it matters
System description		
S1	Tiers and where each decision runs (sensor, fog, cloud)	Latency and failure claims depend on placement
S2	Sensor types, count per bay, and sampling or reporting rule	Event-driven and periodic reporting load the radio differently
S3	Radio parameters: SF, bandwidth, payload bytes, confirmed or unconfirmed uplinks	Airtime and collision estimates cannot be reproduced without them
S4	Lot size, number of zones, gateways, and fog nodes	Results at 150 spaces do not transfer to 2,500
Data		
D1	Source of every input: measured, public dataset, or assumed	Separates evidence from assumption
D2	Period, resolution, missing-data share, and cleaning rules	Imputation and filtering change forecast error
D3	How ground truth was obtained and its own error	A model cannot be judged more precisely than its labels
Simulation setup		
M1	Simulator version, network, demand model, and all parameter values	Hidden defaults decide outcomes
M2	Warm-up length, run length, and initial conditions	Transient and steady-state results differ
M3	Which inputs were fixed by hand (for example congestion levels)	Hand-set inputs make the benefit partly assumed
M4	Stress scenarios: surge, outage, node failure, degraded sensing, attack	Normal-day results hide fragility
Statistics		
T1	Number of independent replications and the seed policy	One run shows a single sample path
T2	Means with confidence intervals, not point values alone	Shows whether a difference exceeds noise
T3	Paired comparison on common random numbers and the test used	Reduces variance when comparing policies
Baselines		
B1	A naive baseline (blind search, persistence)	Establishes the floor
B2	A strong simple baseline (nearest free bay, seasonal persistence)	Shows the value of the added complexity
B3	Ablation with each component enabled alone	Attributes gains to the right component
Forecasting		
F1	Horizons evaluated, including the booking lead time	Skill falls with horizon
F2	Chronological train, validation, and test split	Random splits leak future information
F3	Point error (MAE, RMSE) and interval coverage with width	Decisions need uncertainty, not only a point
F4	Error in the decision metric: bookings accepted that found no space	Links accuracy to cost
Pricing		
P1	Bid or demand distribution and how it was calibrated	Synthetic bids can guarantee the outcome
P2	Objective stated (revenue, occupancy, welfare)	“Best price” is undefined without it
P3	Tie-break rule, price cap, and sensitivity to shading	Rules can be gamed and may disadvantage users
Security and privacy		
X1	Threat model: assets, attackers, trust boundaries	Defines what the design claims to resist
X2	Key provisioning, counter handling, and rotation	Most practical failures are here
X3	Personal data collected, retention period, and legal basis	Needed for compliance and minimization
X4	Overhead of protection: bytes, airtime, energy, delay	Security is not free on LPWAN
Reproducibility		
R1	Code, configuration, and software versions released or described	Allows reruns and audits
R2	Data schema and sample records (Appendix E)	Lets others build comparable logs
R3	Hardware list with firmware versions and calibration dates	Sensor behavior drifts
Ethics		
E1	Permissions, notices, and who can access the data	Pilots involve real drivers
E2	Fairness checks: price distribution by time and place, access for drivers without smartphones	Pricing can exclude users

(M), and high (H), and must be re-rated for each site with the operator. None of the ratings comes from measurement. Mitigations refer to measures proposed in Section V, and owners are roles, not named persons.

Appendix E
Data Dictionary for a Pilot Log

Table LXXXV defines a minimal schema for the five log families of a pilot: sensor uplinks, fog events, bookings, payments, and audit counts. The schema is our proposal and not the reference system's. Each field carries a privacy class. N is non-personal operational data. Q is quasi-identifying: harmless alone but linkable to a person in combination, such as booking times. P is personal or pseudonymous personal data, which is likely to count as personal data under the GDPR and the DPDP Act. R is restricted: secrets or payment references whose disclosure causes direct harm. The payment family stores only a token issued by the gateway; card numbers never reach the parking servers. We suggest keeping class N for the study period, deleting class P and Q records after a short stated retention period, and keeping class R only in the payment provider's system.

Pilot Risk Register (Qualitative, Illustrative Ratings)

ID	Risk	Cause	L	I	Mitigation	Owner
R1	Radio saturation and packet loss	Uncoordinated access; bursts when a section empties together	M	H	Zoned gateways; jittered heartbeats; per-device SF; alarm on falling PDR	Network engineer
R2	Coverage holes	Obstruction by vehicles, buildings, and shelving; poor gateway siting	H	M	Pre-pilot RSSI and SNR survey; relocate or add a gateway; log link quality	Network engineer
R3	Wrong or drifting occupancy readings	Magnetic disturbance; temperature effect on ultrasonic range; sensor ageing	H	H	Two sensor types per bay; scheduled manual audits; drift alarms; recalibration	Field technician
R4	Physical damage to sensors	Vehicle strikes; water ingress; theft or tampering	M	M	Rated enclosures; mounting guide; tamper flag in payload; spares	Field technician
R5	Poor camera counts	Night, rain, glare, occlusion, or camera shift	H	M	Acceptance test per condition; entry and exit counter fallback; degraded-mode flag	Vision engineer
R6	Fog node outage or corruption	Power cuts; storage wear; overheating; software fault	M	H	UPS; read-only system image; watchdog; remote restart; standby node	Systems engineer
R7	Unreliable logs	Clock drift; duplicate or missing records; schema changes	H	M	Network time source; sequence numbers; versioned schema; daily quality report	Data engineer
R8	Spoofing, replay, or key theft	Weak key handling; counter reset; compromised gateway or device	M	H	Per-device keys; strict counter check; plausibility checks; patch plan	Security lead
R9	Privacy or legal breach	Plates or faces captured; retention beyond purpose; unclear data roles	M	H	Counts-only cameras; impact assessment; signage; retention schedule; legal review	Data protection lead
R10	Unfair or unstable prices	Surcharges at peaks; bid collusion; rules unclear to users	M	M	Price cap; published rules; monitor price distribution; ethics review	Pricing analyst
R11	Loss of stakeholder support	Operator withdraws; users distrust bookings; liability for a failed booking unclear	M	H	Written agreement; defined scope; refund rule; regular reporting	Project manager
R12	Batteries drain early	Cold or heat; retransmissions; sensing duty higher than planned	M	M	Energy budget with margin; battery voltage in payload; replacement schedule	Hardware engineer
R13	Environmental damage to equipment	Heat, dust, monsoon rain, lightning surge	H	M	Ingress-rated cases; surge protection; temperature logging	Hardware engineer
R14	Schedule overrun	Procurement or permit delay; too few audit staff	H	M	Milestones with buffer; early permits; stage gates before expansion	Project manager

Appendix F
Illustrative Bay-Sensor Payload Layout

The capacity analysis uses 12 bytes as a round working figure for the whole physical payload; the 12 application bytes of the layout below would, with 13 bytes of LoRaWAN overhead, form a 25-byte frame (Section IV-H), and the 6byte layout of Table VI gives a 19-byte frame; the reference description does not give the content of its frames. Table LXXXVI shows one *illustrative* layout of 12 bytes that carries the fields needed for the log schema above. It is our own design for explanation, not the reference system's format and not a LoRaWAN requirement. Multi-byte fields are bigendian. Security is left to the LoRaWAN layer, which adds its own header and message integrity code, so the layout contains no key material or checksum. The age byte, the minutes since the last state change (capped at 255), lets the server estimate the true event time when a frame is delayed or sent as a heartbeat.

Listing 10 packs and unpacks this layout with the struct module. The format string ">BHhHBbHB" encodes one byte, two bytes, a signed two-byte value, two bytes, one byte, a signed byte, two bytes, and one byte, which is 12 bytes in total. The decoder rejects frames of the wrong length or an unknown version, the two cheapest checks against malformed input. The sample frame is invented: sequence 513, a field change of $-12.3\ \mu$T, an ultrasonic distance of 480 mm, 3.34 V, 31 °C, bay 1042, and a state change 7 minutes ago with the occupied flag set. Output:

```
12 bytes: 11 02 01 ff 85 01 e0 a7 1f 04 12 07 flags  1 seq
          513
```

TABLE LXXXV

PILOT LOG SCHEMA (PROPOSED)

Field	Type	Unit	Class	Description
Sensor uplink (one row per received frame)				
rx_time	timestamp	UTC, ms	N	Time the gateway received the frame
dev_id	string	—	N	Pseudonymous device identifier
gw_id	string	—	N	Receiving gateway; frames heard by several gateways give several rows
f_cnt	uint32	—	N	Frame counter after server-side extension
bay_id	uint16	—	N	Bay the device is mounted on
occupied	bool	—	N	Reported bay state
battery_v	float	V	N	Battery voltage at transmission
rssi, snr	float	dBm, dB	N	Link quality at the gateway
sf	uint8	—	N	Spreading factor of the frame
Fog event (state changes, alerts, and degraded modes)				
event_id	uuid	—	N	Unique event key
ts	timestamp	UTC, ms	N	Time of the event on the fog node
zone_id	string	—	N	Zone served by the node
event_type	enum	—	N	State change, plausibility alert, failover, camera degraded, rate limit
veh_count	uint16	vehicles	N	Vehicles counted per lane in the interval; no images stored
congestion	float	0 to 1	N	Estimated lane congestion c
free_fcst, free_lo	float	bays	N	Forecast free bays and lower prediction bound at the booking horizon
cpu_load	float	%	N	Fog node processor load at the event
Booking booking_id				
	uuid	—	Q	Unique booking key
acct_pid	string	—	P	Pseudonymous account identifier, rotated per retention period
plate_hash	string	—	P	Keyed hash of the number plate, only if plate matching is used
start, end	timestamp	UTC, min	Q	Booked interval
status	enum	—	N	Held, confirmed, cancelled, expired, no-space
price_quoted	decimal	INR	Q	Price shown at booking time
Payment (token only)				
pay_token	string	—	R	Opaque token issued by the payment gateway
amount	decimal	INR	Q	Amount charged
pay_status	enum	—	N	Authorized, captured, refunded, failed
Audit counts				
audit_id	uuid	—	N	Unique audit key
audit_time	timestamp	UTC, min	N	Time of the manual count
manual_occ	uint16	bays	N	Occupied bays counted by hand in the zone
sensor_occ	uint16	bays	N	Occupied bays according to sensors at the same time
gate_in, gate_out	uint32	vehicles	N	Cumulative entry and exit counts since the reference time
auditor	string	—	P	Pseudonymous auditor identifier

```
    mag_ut      -12.3
    dist_mm       480
    batt_v       3.34
    temp_c 31
    bay       1042 age_min 7
rejected: bad length rejected: unknown
 version
```

Quantization limits what the frame can express: the battery value has 20 mV resolution, the field change has 0.1 μT resolution, and the sequence number wraps. A real design would add a field-by-field range check and would test against the actual sensor ranges.

REFERENCES

[1] D. C. Shoup, "Cruising for parking," *Transport Policy*, vol. 13, no. 6, pp. 479–486, Nov. 2006.

[2] M. Balfaqih, W. Jabbar, M. Khayyat, and R. Hassan, "Design and development of smart parking system based on fog computing and Internet of Things," *Electronics*, vol. 10, no. 24, p. 3184, Dec. 2021.

[3] C. Biyik *et al.*, "Smart parking systems: Reviewing the literature, architecture and ways forward," *Smart Cities*, vol. 4, pp. 623–642, 2021.

TABLE LXXXVI

ILLUSTRATIVE 12-BYTE BAY-SENSOR PAYLOAD

Byte	Field	Content
0	ver, flags	High nibble: version. Low nibble: bit 0 occupied, bit 1 tamper, bit 2 low battery, bit 3 sensor fault
1–2	seq	Application sequence number, wraps at 65,536
3–4	mag	Magnetic field change, signed, 0.1 μT

TABLE LXXXVI

5–6	dist	Ultrasonic distance, mm
7	batt	Battery voltage in 20 mV steps
8	temp	Temperature, signed, ˚C
9–10	bay	Bay identifier
11	age	Minutes since last state change, capped at 255

[4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st Ed. MCC Workshop Mobile Cloud Comput.*, Helsinki, Finland, 2012, pp. 13–16.

[5] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, "A study of LoRa: Long range & low power networks for the Internet of Things," *Sensors*, vol. 16, no. 9, p. 1466, 2016.

[6] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, Dec. 1959.

[7] P. A. Lopez *et al.*, "Microscopic traffic simulation using SUMO," in *Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, Maui, HI, USA, Nov. 2018, pp. 2575–2582.

[8] J. J. Barriga *et al.*, "A smart parking solution architecture based on LoRaWAN and Kubernetes," *Applied Sciences*, vol. 10, p. 4674, 2020.

[9] J. Shi, L. Jin, J. Li, and Z. Fang, "A smart parking system based on NB-IoT and third-party payment platform," in *Proc. 17th Int. Symp. Commun. Inf. Technol. (ISCIT)*, Cairns, Australia, Sep. 2017, pp. 1–5.

[10] P. Sotres, C. L. de la Torre, L. Sanchez, S. M. Jeong, and J. Kim, "Smart´ city services over a global interoperable Internet-of-Things system: The smart parking case," in *Proc. Global Internet Things Summit (GIoTS)*, Bilbao, Spain, Jun. 2018, pp. 1–6.

[11] V. K. Sarker, T. N. Gia, I. B. Dhaou, and T. Westerlund, "Smart parking system with dynamic pricing, edge-cloud computing and LoRa," *Sensors*, vol. 20, p. 4669, 2020.

[12] Y. Zheng, S. Rajasegarar, and C. Leckie, "Parking availability prediction for sensor-enabled car parks in smart cities," in *Proc. IEEE 10th Int. Conf. Intell. Sensors, Sensor Netw. Inf. Process. (ISSNIP)*, Singapore, Apr. 2015, pp. 1–6.

[13] E. I. Vlahogianni, K. Kepaptsoglou, V. Tsetsos, and M. G. Karlaftis, "A real-time parking prediction system for smart cities," *Journal of Intelligent Transportation Systems*, vol. 20, no. 2, pp. 192–204, 2016.

[14] I. You, S. Kwon, G. Choudhary, V. Sharma, and J. T. Seo, "An enhanced LoRaWAN security protocol for privacy preservation in IoT with a case study on a smart factory-enabled parking system," *Sensors*, vol. 18, p. 1888, 2018.

[15] Y. Jeon, H.-I. Ju, and S. Yoon, "Design of an LPWAN communication module based on secure element for smart parking application," in *Proc.*

Listing 10. Encoding and decoding the illustrative payload **import** struct

```
# Illustrative 12-byte layout, big-endian:
# B ver<<4|flags H seq h mag(0.1 uT) H dist(mm)
# B batt(20 mV)                    b temp(C) H bay_id B age(min)
FMT = ">BHhHBbHB"
VER = 1
OCC, TAMPER, LOWBAT, FAULT = 1, 2, 4, 8

def encode(seq, mag_ut, dist_mm, batt_v, temp_c, bay, age_min,
           flags):
    if not 0 <= flags <= 15:
        raise ValueError("flags are 4 bits")
    return struct.pack(
        FMT, VER << 4 | flags, seq & 0xFFFF, round(mag_ut*10),
        dist_mm, round(batt_v/0.02), temp_c, bay, min(age_min,
        255))

def decode(raw):
    if len(raw) != struct.calcsize(FMT):
        raise ValueError("bad length")
    b0, seq, mag, d, bat, t, bay, age = \ struct.unpack(FMT,
    raw) if b0 >> 4 != VER:
        raise ValueError("unknown version")
    return dict(flags=b0 & 15, seq=seq, mag_ut=mag/10, dist_mm=d,
                batt_v=round(bat*0.02, 2), temp_c=t, bay=bay,
                age_min=age)

raw = encode(513, -12.3, 480, 3.34, 31, 1042, 7,
             OCC) print(len(raw), "bytes:",
raw.hex(" ")) for k, v in decode(raw).items():
    print(f" {k:<8}{v}")
for bad in (raw[:-1], b"\x21" + raw[1:]):
    try:
        decode(bad)
    except ValueError as e: print("rejected:", e)
```


IEEE Int. Conf. Consum. Electron. (ICCE), Las Vegas, NV, USA, Jan. 2018, pp. 1–2.

[16] I. Butun, N. Pereira, and M. Gidlund, "Security risk analysis of LoRaWAN and future directions," *Future Internet*, vol. 11, no. 1, p. 3, 2019.

[17] D. C. Shoup, *The High Cost of Free Parking*. Chicago, IL, USA: American Planning Association, 2005.

[18] R. Arnott and E. Inci, "An integrated model of downtown parking and traffic congestion," *Journal of Urban Economics*, vol. 60, no. 3, pp. 418–442, 2006.

[19] G. Pierce and D. Shoup, "Getting the prices right: An evaluation of pricing parking by demand in San Francisco," *Journal of the American Planning Association*, vol. 79, no. 1, pp. 67–81, 2013.

[20] T. Lin, H. Rivano, and F. Le Mouel, "A survey of smart parking¨ solutions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3229–3253, 2017.

[21] A. Fahim, M. Hasan, and M. A. Chowdhury, "Smart parking systems: Comprehensive review based on various aspects," *Heliyon*, vol. 7, no. 5, p. e07050, 2021.

[22] G. Amato, F. Carrara, F. Falchi, C. Gennaro, C. Meghini, and C. Vairo, "Deep learning for decentralized parking lot occupancy detection," *Expert Systems with Applications*, vol. 72, pp. 327–334, 2017.

[23] S. Nawaz, C. Efstratiou, and C. Mascolo, "ParkSense: A smartphone based sensing system for on-street parking," in *Proc. ACM MobiCom*, 2013.

[24] S. Mathur *et al.*, "ParkNet: Drive-by sensing of road-side parking statistics," in *Proc. ACM MobiSys*, 2010, pp. 123–136.

[25] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[26] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of things (IoT): A vision, architectural elements, and future directions," *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, 2013.

[27] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of things for smart cities," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, 2014.

[28] L. Sanchez´ *et al.*, "SmartSantander: IoT experimentation over a smart city testbed," *Computer Networks*, vol. 61, pp. 217–238, 2014.

[29] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[30] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[31] A. V. Dastjerdi and R. Buyya, "Fog computing: Helping the Internet of Things realize its potential," *Computer*, vol. 49, no. 8, pp. 112–116, 2016.

[32] C. Mouradian, D. Naboulsi, S. Yangui, R. H. Glitho, M. J. Morrow, and P. A. Polakos, "A comprehensive survey on fog computing: Stateof-the-art and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 1, pp. 416–464, 2018.

[33] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628–1656, 2017.

[34] LoRa Alliance, "LoRaWAN specification v1.1," 2017.

[35] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. MeliaSegui, and T. Watteyne, "Understanding the limits of LoRaWAN," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017.

[36] J. Haxhibeqiri, E. De Poorter, I. Moerman, and J. Hoebeke, "A survey of LoRaWAN for IoT: From technology to application," *Sensors*, vol. 18, no. 11, p. 3995, 2018.

[37] U. Raza, P. Kulkarni, and M. Sooriyabandara, "Low power wide area networks: An overview," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 855–873, 2017.

[38] M. C. Bor, U. Roedig, T. Voigt, and J. M. Alonso, "Do LoRa low-power wide-area networks scale?" in *Proc. 19th ACM Int. Conf. Model. Anal. Simul. Wireless Mobile Syst. (MSWiM)*, 2016, pp. 59–67.

[39] OASIS, "MQTT version 3.1.1, OASIS standard," 2014.

[40] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, "Recent development and applications of SUMO—simulation of urban MObility," *International Journal on Advances in Systems and Measurements*, vol. 5, no. 3–4, pp. 128–138, 2012.

[41] R. E. Barone, T. Giuffre, S. M. Siniscalchi, M. A. Morgano, and` G. Tesoriere, "Architecture for parking management in smart cities," *IET Intelligent Transport Systems*, vol. 8, no. 5, pp. 445–452, 2014.

[42] E. Sallum, N. Pereira, M. Alves, and M. Santos, "Performance optimization on LoRa networks through assigning radio parameters," in *Proc. IEEE Int. Conf. Ind. Technol. (ICIT)*, Buenos Aires, Argentina, Feb. 2020, pp. 304–309.

[43] F. Caicedo, C. Blazquez, and P. Miranda, "Prediction of parking space availability in real time," *Expert Systems with Applications*, vol. 39, no. 8, pp. 7281–7290, 2012.

[44] T. Rajabioun and P. A. Ioannou, "On-street and off-street parking availability prediction using multivariate spatiotemporal models," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 5, pp. 2913–2924, 2015.

[45] J. R. Norris, *Markov Chains*. Cambridge, U.K.: Cambridge University Press, 1997.

[46] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Melbourne, Australia: OTexts, 2018.

[47] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.

[48] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” arXiv:2107.07511, 2021.

[49] C. Lei and Y. Ouyang, “Dynamic pricing and reservation for intelligent urban parking management,” *Transportation Research Part C*, vol. 77, pp. 226–244, 2017.

[50] W. Vickrey, “Counterspeculation, auctions, and competitive sealed tenders,” *Journal of Finance*, vol. 16, no. 1, pp. 8–37, Mar. 1961.

[51] V. Krishna, *Auction Theory*, 2nd ed. San Diego, CA, USA: Academic Press, 2009.

[52] L. F. Luque-Vega, D. A. Michel-Torres, E. Lopez-Neri, M. A. CarlosMancilla, and L. E. Gonzalez-Jim´ enez, “IoT smart parking system based´ on the visual-aided smart vehicle presence sensor: SPIN-V,” *Sensors*, vol. 20, p. 1476, 2020.

[53] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 779–788.

[54] A. G. Howard *et al.*, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv:1704.04861, 2017.

[55] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018.

[56] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2016, pp. 3464–3468.

[57] X. Yang, E. Karampatzakis, C. Doerr, and F. Kuipers, “Security vulnerabilities in LoRaWAN,” in *Proc. IEEE/ACM Third Int. Conf. Internetof-Things Design and Implementation (IoTDI)*, 2018.

[58] M. Eldefrawy, I. Butun, N. Pereira, and M. Gidlund, “Formal security analysis of LoRaWAN,” *Computer Networks*, vol. 148, pp. 328–339, 2019.

[59] R. G. Sargent, “Verification and validation of simulation models,” *Journal of Simulation*, vol. 7, no. 1, pp. 12–24, 2013.

[60] S. M. Ross, *Introduction to Probability Models*, 11th ed. Academic Press, 2014.

[61] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards realtime object detection with region proposal networks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[62] W. Liu *et al.*, “SSD: Single shot multibox detector,” in *Proc. European Conf. Computer Vision (ECCV)*, 2016.

[63] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” arXiv:1804.02767, 2018.

[64] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Proc. European Conf. Computer Vision (ECCV)*, 2014.

[65] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[66] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 (General Data Protection Regulation),” Official Journal of the European Union, 2016.

[67] Government of India, “The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023),” 2023.

[68] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

[69] T. Gneiting, F. Balabdaoui, and A. E. Raftery, “Probabilistic forecasts, calibration and sharpness,” *Journal of the Royal Statistical Society: Series B*, vol. 69, no. 2, pp. 243–268, 2007.

[70] T. Gneiting and A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

[71] C. Bergmeir and J. M. Ben´ıtez, “On the use of cross-validation for time series predictor evaluation,” *Information Sciences*, vol. 191, pp. 192–213, 2012.

[72] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[73] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, San Francisco, CA, USA, Aug. 2016, pp. 785–794.

[74] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[75] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York, NY, USA: McGraw-Hill, 2015.

[76] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. Hoboken, NJ, USA: Wiley, 2017.

[77] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.

[78] D. Dolev and A. C. Yao, “On the security of public key protocols,” *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.

[79] National Institute of Standards and Technology, “Guide for conducting risk assessments,” NIST, Tech. Rep. SP 800-30 Rev. 1, 2012.

[80] E. Rescorla, “The Transport Layer Security (TLS) protocol version 1.3,” IETF RFC 8446, 2018.

[81] A. Cavoukian, “Privacy by design: The 7 foundational principles,” Information and Privacy Commissioner of Ontario, 2009.

[82] R. Roman, J. Zhou, and J. Lopez, “On the features and challenges of security and privacy in distributed Internet of Things,” *Computer Networks*, vol. 57, no. 10, pp. 2266–2279, 2013.

[83] S. Sicari, A. Rizzardi, L. A. Grieco, and A. Coen-Porisini, “Security, privacy and trust in Internet of Things: The road ahead,” *Computer Networks*, vol. 76, pp. 146–164, 2015.

[84] R. B. Myerson, “Optimal auction design,” *Mathematics of Operations Research*, vol. 6, no. 1, pp. 58–73, 1981.

[85] E. H. Clarke, “Multipart pricing of public goods,” *Public Choice*, vol. 11, no. 1, pp. 17–33, 1971.

[86] T. Groves, “Incentives in teams,” *Econometrica*, vol. 41, no. 4, pp. 617–631, 1973.

[87] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[88] J. Demsar, “Statistical comparisons of classifiers over multiple data sets,”ˇ *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.